

The **irace** Package: User Guide

Manuel López-Ibáñez, Leslie Pérez Cáceres, Jérémie Dubois-Lacoste,
Thomas Stützle and Mauro Birattari
IRIDIA, CoDE, Université Libre de Bruxelles, Brussels, Belgium

Version 4.4.3.9000, September 4, 2026

Contents

1	General information	4
1.1	Background	4
1.2	Version	4
1.3	License	4
2	Before starting	5
3	Installation	5
3.1	System requirements	5
3.2	irace installation	5
3.2.1	Install automatically within R	6
3.2.2	Manual download and installation	6
3.2.3	Local installation	6
3.2.4	Testing the installation and invoking irace	7
4	Running irace	8
4.1	Step-by-step setup guide	8
4.2	Setup example for ACOTSP	11
5	Defining a configuration scenario	13
5.1	Target algorithm parameters	13
5.1.1	Parameter types	13
5.1.2	Parameter domains	14
5.1.3	Parameter dependent domains	14
5.1.4	Conditional parameters	14
5.1.5	Forbidden parameter configurations	14
5.1.6	Global options	15
5.1.7	Parameter file format	15
5.1.8	Parameters R format	16
5.2	Target algorithm runner	18
5.2.1	Target runner as an executable program	18
5.2.2	Target runner as an R function	19
5.3	Target evaluator	20

5.3.1	Target evaluator executable program	20
5.3.2	Target evaluator R function	21
5.4	Training instances	21
5.5	Initial configurations	23
5.6	Repairing configurations	23
6	Parallelization	24
7	Testing (Validation) of configurations	26
8	Recovering irace runs	27
9	Output and results	29
9.1	Text output	29
9.2	R data file (logFile)	32
9.3	Analysis of results	41
10	Advanced topics	43
10.1	Monitoring iterations with a callback	43
10.2	Tuning budget	44
10.3	Multi-objective tuning	45
10.4	Improving anytime behavior	46
10.5	Tuning for minimizing computation time	46
10.6	Hyper-parameter optimization of machine learning methods	47
10.7	Heterogeneous scenarios	48
10.8	Choosing the statistical test	49
10.9	Complex parameter space constraints	50
10.10	Unreliable target algorithms and immediate rejection	51
10.11	Ablation Analysis	51
10.12	Post-selection race	54
10.13	Parameter importance analysis using PyImp	54
11	List of command-line and scenario options	55
11.1	General options	55
11.2	Elitist irace	56
11.3	Internal irace options	57
11.4	Target algorithm parameters	58
11.5	Target algorithm execution	58
11.6	Initial configurations	60
11.7	Training instances	60
11.8	Tuning budget	60
11.9	Statistical test	61
11.10	Adaptive capping	61
11.11	Recovery	62
11.12	Testing	62
12	FAQ (Frequently Asked Questions)	63
12.1	Is irace minimizing or maximizing the output of my algorithm?	63
12.2	Are experiments with irace reproducible?	63
12.3	Is it possible to configure a MATLAB algorithm with irace?	63

12.4	My program works perfectly on its own, but not when running under irace . Is irace broken?	64
12.5	irace seems to run forever without any progress, is this a bug?	64
12.6	My program may be buggy and run into an infinite loop. Is it possible to set a maximum timeout?	65
12.7	When using the mpi option, irace stops with an error message indicating that a function is not defined. How to fix this?	65
12.8	Error: 4 arguments passed to .Internal(nchar) which requires 3	65
12.9	Warning: In read.table(filename, header = TRUE, colClasses = "character", : incomplete final line found by	66
12.10	How are relative filesystem paths interpreted by irace ?	66
12.11	My parameter space is small enough that irace could generate all possible configurations; however, irace generates repeated configurations and/or does not generate some of them. Is this a bug?	66
12.12	On Windows and using target-runner.py (a Python file), I get the error target-runner.py is not executable	66
12.13	Error in socketConnection(...) : can not open the connection	67
12.14	irace does not print the call to the target-runner with <code>-debug-level 2</code> when using the <code>-parallel</code> option	67
12.15	Why does the column Bound in the output sometimes shows NA ?	67
13	Resources and contact information	67
14	Acknowledgements	68
	Bibliography	68
	Appendix A Installing R	70
A.1	GNU/Linux	70
A.2	OS X	70
A.3	Windows	70
	Appendix B targetRunner troubleshooting checklist	70
	Appendix C targetEvaluator troubleshooting checklist	73
	Appendix D Glossary	73
	Appendix E NEWS	74

1 General information

1.1 Background

The **irace** package implements an *iterated racing* procedure, which is an extension of Iterated F-race (I/F-Race) [3]. The main use of **irace** is the automatic configuration of optimization and decision algorithms, that is, finding the most appropriate settings of an algorithm given a set of instances of a problem. However, it may also be useful for configuring other types of algorithms when performance depends on the used parameter settings. It builds upon the **race** package by Birattari and it is implemented in R. The **irace** package is available from CRAN:

<https://cran.r-project.org/package=irace>

More information about **irace** is available at <https://mlopez-ibanez.github.io/irace/>.

1.2 Version

The current version of the **irace** package is 4.4.3.9000. Previous versions of the package can also be found in the [CRAN website](#).

The algorithm underlying the current version of **irace** and its motivation are described by López-Ibáñez et al. [12]. The **adaptive capping mechanism** available from version 3.0 is described by Pérez Cáceres et al. [16]. Details of the implementation before version 2.0 can be found in a previous technical report [11].



Versions of **irace** before 2.0 are not compatible with the file formats detailed in this document.

1.3 License

The **irace** package is Copyright © 2026 and distributed under the GNU General Public License version 3.0 (<http://www.gnu.org/licenses/gpl-3.0.en.html>). The **irace** package is free software (software libre): You can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

The **irace** package is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Please be aware that the fact that this program is released as Free Software does not excuse you from scientific propriety, which obligates you to give appropriate credit! If you write a scientific paper describing research that made substantive use of this program, it is your obligation as a scientist to (a) mention the fashion in which this software was used in the Methods section; (b) mention the algorithm in the References section. The appropriate citation is:

Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Thomas Stützle, and Mauro Birattari. The **irace** package: Iterated Racing for Automatic Algorithm Configuration. *Operations Research Perspectives*, 3:43–58, 2016. doi: [10.1016/j.orp.2016.09.002](https://doi.org/10.1016/j.orp.2016.09.002)

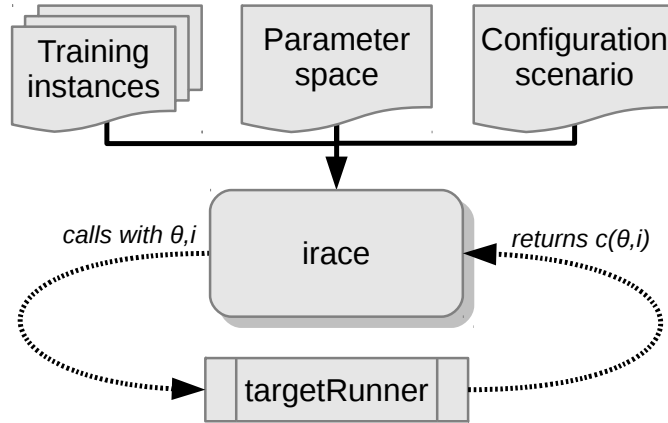


Figure 1: Scheme of **irace** flow of information.

2 Before starting

The **irace** package provides an automatic configuration tool for tuning optimization algorithms, that is, automatically finding good configurations for the parameters values of a (target) algorithm saving the effort that normally requires manual tuning.

Figure 1 gives a general scheme of how **irace** works. **Irace** receives as input a *parameter space definition* corresponding to the parameters of the target algorithm that will be tuned, a set of *instances* for which the parameters must be tuned for and a set of options for **irace** that define the *configuration scenario*. Then, **irace** searches in the parameter search space for good performing algorithm configurations by executing the target algorithm on different instances and with different parameter configurations. A **targetRunner** must be provided to execute the target algorithm with a specific parameter configuration (θ) and instance (i). The **targetRunner** function (or program) acts as an interface between the execution of the target algorithm and **irace**: It receives the instance and configuration as arguments and must return the evaluation of the execution of the target algorithm.

The following user guide contains guidelines for installing **irace**, defining configuration scenarios, and using **irace** to automatically configure your algorithms.

3 Installation

3.1 System requirements

- R (version $\geq 3.2.0$) is required for running **irace**, but you don't need to know the R language to use it. R is freely available and you can download it from the R project website (<https://www.r-project.org>). See [Section A](#) for a quick installation guide of R.
- For GNU/Linux and OS X, the command-line executable **parallel-irace** requires GNU Bash. Individual examples may require additional software.

3.2 irace installation

The **irace** package can be installed automatically within R or by manual download and installation. We advise to use the automatic installation unless particular circumstances do not allow it. The

instructions to install **irace** with the two mentioned methods are the following:

3.2.1 Install automatically within R

Execute the following line in the R console to install the package:

```
install.packages("irace")
```

Select a mirror close to your location, and test the installation in the R console with:

```
library("irace")  
q() # To exit R
```

Alternatively, within the R graphical interface, you may use the **Packages** and **data->Package installer** menu on OS X or the **Packages** menu on Windows.

3.2.2 Manual download and installation

From the **irace** package CRAN website (<https://cran.r-project.org/package=irace>), download one of the three versions available depending on your operating system:

- **irace_4.4.3.9000.tar.gz** (Unix/BSD/GNU/Linux)
- **irace_4.4.3.9000.tgz** (OS X)
- **irace_4.4.3.9000.zip** (Windows)

To install the package on GNU/Linux and OS X, you must execute the following command at the shell (replace **<package>** with the path to the downloaded file, either **irace_4.4.3.9000.tar.gz** or **irace_4.4.3.9000.zip**):

```
R CMD INSTALL <package>
```

To install the package on Windows, open R and execute the following line on the R console (replace **<package>** with the path to the downloaded file **irace_4.4.3.9000.zip**):

```
install.packages("<package>", repos = NULL)
```

If the previous installation instructions fail because of insufficient permissions and you do not have sufficient admin rights to install **irace** system-wide, then you need to force a local installation.

3.2.3 Local installation

Let's assume you wish to install **irace** on a path denoted by **<R_LIBS_USER>**, which is a filesystem path for which you have sufficient rights. This directory **must** exist before attempting the installation. Moreover, you must provide to R the path to this library when loading the package. However, the latter can be avoided by adding the path to the system variable **R_LIBS** or to the R internal variable **.libPaths**, as we will see below.¹

On GNU/Linux or OS X, execute the following commands to install the package on a local directory:

¹On Windows, see also https://cran.r-project.org/bin/windows/base/rw-FAQ.html#I-don_0027t-have-permission-to-write-to-the-R_002d3_002e3_002e1_005clibrary-directory.

```
export R_LIBS_USER="<R_LIBS_USER>"
# Create R_LIBS_USER if it doesn't exist
mkdir $R_LIBS_USER
# Replace <package> with the path to the downloaded file.
R CMD INSTALL --library=$R_LIBS_USER <package>
# Tell R where to find R_LIBS_USER
export R_LIBS=${R_LIBS_USER}:${R_LIBS}
```

On Windows, you can install the package on a local directory by executing the following lines in the R console:

```
# Replace <package> with the path to the downloaded file.
# Replace <R_LIBS_USER> with the path used for installation.
install.packages("<package>", repos = NULL, lib = "<R_LIBS_USER>")
# Tell R where to find R_LIBS_USER.
# This must be executed for every new session.
.libPaths(c("<R_LIBS_USER>", .libPaths()))
```

3.2.4 Testing the installation and invoking irace

Once **irace** has been installed, load the package and test that the installation was successful by opening an R console and executing:

```
# Load the package
library("irace")
# Obtain the installation path
system.file(package = "irace")
```

The last command must print out the filesystem path where **irace** is installed. In the remainder of this guide, the variable **\$IRACE_HOME** is used to denote this path. When executing any provided command that includes the **\$IRACE_HOME** variable do not forget to replace this variable with the installation path of **irace**.

On GNU/Linux or OS X, you can let the operating system know where to find **irace** by defining the **\$IRACE_HOME** variable and adding it to the system **PATH**. Append the following commands to **~/.bash_profile**, **~/.bashrc** or **~/.profile**:

```
# Replace <IRACE_HOME> with the irace installation path
export IRACE_HOME=<IRACE_HOME>
export PATH=${IRACE_HOME}/bin/:$PATH
# Tell R where to find R_LIBS_USER
# Use the following line only if local installation was forced
export R_LIBS=${R_LIBS_USER}:${R_LIBS}
```

Then, open a new terminal and launch **irace** as follows:

```
irace --help
```

On Windows, you need to add both R and the installation path of **irace** to the environment variable **PATH**. To edit the **PATH**, search for “Environment variables” in the control panel, edit **PATH** and add a string similar to **C:\R_PATH\bin;C:\IRACE_HOME\bin\x64** where **R_PATH** is the

installation path of **R** and **IRACE_HOME** is the installation path of **irace**. If **irace** was installed locally, you also need to edit the environment variable **R_LIBS** to add **R_LIBS_USER**. Then, open a new terminal (run program **cmd.exe**) and launch **irace** as:

```
irace.exe --help
```

Alternatively, you may directly invoke **irace** from within the **R** console by executing:

```
library("irace")
irace_cmdline("--help")
```

4 Running irace

Before performing the tuning of your algorithm, it is necessary to define a tuning scenario that will give **irace** all the necessary information to optimize the parameters of the algorithm. The tuning scenario is composed of the following elements:

1. Target algorithm parameter description (see [Section 5.1](#)).
2. Target algorithm runner (see [Section 5.2](#)).
3. Training instances list (see [Section 5.4](#)).
4. **irace** options (see [Section 11](#)).
5. *Optional*: Initial configurations (see [Section 5.5](#)).
6. *Optional*: Target algorithm evaluator (see [Section 5.3](#)).

These scenario elements can be provided as plain text files or as **R** objects. This user guide provides examples of both types, but we advise the use of plain text files, which we consider the simpler option.

For a step-by-step guide to create the scenario elements for your target algorithm continue to [Section 4.1](#). For an example execution of **irace** using the **ACOTSP** scenario go to [Section 4.2](#).

4.1 Step-by-step setup guide

This section provides a guide to setup a basic execution of **irace**. The template files provided in the package (**\$IRACE_HOME/templates**) will be used as basis for creating your new scenario. Please follow carefully the indications provided in each step and in the template files used; if you have doubts check the the sections that describe each option in detail.

1. Create a directory (e.g., **./tuning/**) for the scenario setup. This directory will contain all the files that describe the scenario. On GNU/Linux or OS X, you can do this as follows:

```
mkdir ./tuning
cd ./tuning
```

2. Initialize the tuning directory with template config files. On GNU/Linux or OS X, you can do this as follows:

```
irace --init
```

3. Define the target algorithm parameters to be tuned by following the instructions in `parameters.txt`. Available parameter types and other guidelines can be found in [Section 5.1](#).
4. *Optional*: Define the initial parameter configuration(s) of your algorithm, which allows you to provide good starting configurations (if you know some) for the tuning. Follow the instructions in `configurations.txt` and set `configurationsFile="configurations.txt"` in `scenario.txt`. More information in [Section 5.5](#). If you do not need to define initial configurations remove this file from the directory.
5. Place the instances you would like to use for the tuning of your algorithm in the folder `./tuning/Instances/`. In addition, you can create a file (e.g., `instances-list.txt`) that specifies which instances from that directory should be run and which instance-specific parameters to use. To use such an instance file, set the appropriate option in `scenario.txt`, e.g., `trainInstancesFile = "instances-list.txt"`. See [Section 5.4](#) for guidelines.
6. Uncomment and assign in `scenario.txt` only the options for which you need a value different from the default. Some common options that you might want to adjust are:

execDir (`--exec-dir`): the directory in which **irace** will execute the target algorithm; the default value is the current directory.

maxExperiments (`--max-experiments`): the maximum number of executions of the target algorithm that **irace** will perform.

maxTime (`--max-time`): maximum total execution time in seconds for the executions of `targetRunner`. In this case, `targetRunner` must return two values: cost and time. Note that you must provide either **maxTime** or **maxExperiments**.

trainInstancesDir (`--train-instances-dir`): set to `./Instances` if you put the training instances in that folder as instructed above.

For setting the tuning budget, see [Section 10.2](#). For more information on **irace** options and their default values, see [Section 11](#).

7. Modify the `target-runner` script to run your algorithm. This script must execute your algorithm with the parameters and instance specified by **irace** and return the evaluation of the execution and *optionally* the execution time (`cost [time]`). When the **maxTime** option is used, returning `time` is mandatory. The `target-runner` template is written in GNU Bash scripting language, which can be executed easily in GNU/Linux and OS X systems. However, you may use any other programming language. We provide examples written in Python, MATLAB and other languages in `$IRACE_HOME/examples/`. An example using Julia is available at <https://github.com/sbomsdorf/An-example-of-irace-using-Julia-code>. Follow these instructions to adjust the given `target-runner` template to your algorithm:
 - (a) Set the `EXE` variable with the path to the executable of the target algorithm.
 - (b) Set the `FIXED_PARAMS` if you need extra arguments in the execution line of your algorithm. An example could be the time that your algorithm is required to run (`FIXED_PARAMS="--time 60"`) or the number of evaluations required (`FIXED_PARAMS="--evaluations 10000"`).
 - (c) The line provided in the template executes the executable described in the `EXE` variable.

```
$EXE ${FIXED_PARAMS} -i ${INSTANCE} --seed ${SEED} ${CONFIG_PARAMS}
```

You must change this line according to the way your algorithm is executed. In this example, the algorithm receives the instance to solve with the flag `-i` and the seed of the random number generator with the flag `--seed`. The variable `CONFIG_PARAMS` adds to the command line the parameters that **irace** has given for the execution. You must set the command line execution as needed. For example, the instance might not need a flag and might need to be the first argument:

```
$EXE ${INSTANCE} ${FIXED_PARAMS} --seed ${SEED} ${CONFIG_PARAMS}
```

The output of your algorithm is saved to the file defined in the `$STDOUT` variable, and error output is saved in the file given by `$STDERR`. The line:

```
if [ -s "$STDOUT" ]; then
```

checks if the file containing the output of your algorithm is not empty. The example provided in the template assumes that your algorithm prints in the last output line the best result found (only a number). The line:

```
COST=$(cat ${STDOUT} | grep -e '^[[:space:]]*[+-]\?[0-9]' | cut -f1)
```

parses the output of your algorithm to obtain the result from the last line. The **target-runner** script must print **only** one number. In the template example, the result is printed with `echo "$COST"` (assuming **maxExperiments** is used) and the generated files are deleted (you may remove that line if you wish to keep them).



The **target-runner** script must be an executable file, unless you specify **targetRunnerLauncher**.

You can test the target runner from the R console by checking the scenario as explained earlier in [Section 4](#).

If you have problems related to the **target-runner** script when executing **irace**, see [Section B](#) for a check list to help diagnose common problems. For more information about the **targetRunner**, please see [Section 5.2](#),

8. *Optional*: Modify the **target-evaluator** file. This is rarely needed and the **target-runner** template does not use it. [Section 5.3](#) explains when a **targetEvaluator** is needed and how to define it.
9. The **irace** executable provides an option (`--check`) to check that the scenario is correctly defined. We recommend to perform a check every time you create a new scenario. When performing the check, **irace** will verify that the scenario and parameter definitions are correct and will test the execution of the target algorithm. To check your scenario execute the following commands:

- From the command-line (on Windows, execute **irace.bat**):

```
# $IRACE_HOME is the installation directory of irace.
$IRACE_HOME/bin/irace --scenario scenario.txt --check
```

- Or from the R console:

```
library("irace")
scenario <- readScenario(filename = "scenario.txt",
                        scenario = defaultScenario())
checkIraceScenario(scenario = scenario)
```

10. Once all the scenario elements are prepared you can execute **irace**, either using the command-line wrappers provided by the package or directly from the R console:

- **From the command-line console**, call the command (on Windows, you should execute `irace.exe`):

```
cd ./tuning/
# $IRACE_HOME is the installation directory of irace
# By default, irace reads scenario.txt, you can specify a different file
# with --scenario.
$IRACE_HOME/bin/irace
```

For this example, we assume that the needed scenario files have been set properly in the `scenario.txt` file using the options described in [Section 11](#). Most **irace** options can be specified in the command line or directly in the `scenario.txt` file.

- **From the R console**, evaluate:

```
library("irace")
# Go to the directory containing the scenario files
setwd("./tuning")
scenario <- readScenario(filename = "scenario.txt",
                        scenario = defaultScenario())
irace_main(scenario = scenario)
```

This will perform one run of **irace**. See the output of `irace --help` in the command-line or `irace_cmdline("--help")` in R for quick information on additional **irace** options. For more information about **irace** options, see [Section 11](#).



Command-line options override the same options specified in the `scenario.txt` file.

4.2 Setup example for ACOTSP

The **ACOTSP** tuning example can be found in the package installation in the folder `$IRACE_HOME/examples/acotsp`. Other example scenarios can be found in the same folder. More examples of tuning scenarios can be found in the Algorithm Configuration Library (Aclib, <https://www.aclib.net/>).

In this section, we describe how to execute the **ACOTSP** scenario. If you wish to start setting up your own scenario, continue to the next section. For this example, we assume a GNU/Linux system such as Ubuntu with a working C compiler such as `gcc`. To execute this scenario follow these steps:

1. Create a directory for the tuning (e.g., `./tuning/`) and copy the example scenario files located in the `examples` folder to the created directory:

```
mkdir ./tuning
cd ./tuning
# $IRACE_HOME is the installation directory of irace.
cp $IRACE_HOME/examples/acotsp/* ./
ls ./
# Make sure that target-runner is executable
chmod u+x target-runner
```

2. Download the training instances from <https://iridia.ulb.ac.be/supp/IridiaSupp2016-003/scenarios/acotsp/instances.tar.gz> to the `./tuning/` directory and decompress it, which creates a folder `instances`:

```
tar -xvf instances.tar.gz
ls instances/
```

If the above gives an error or does not show any files, then the files were not extract correctly. Maybe the `instances.tar.gz` file did not download correctly or maybe it is not in the correct place. It should be within the folder `tuning` correctly.

3. Download the **ACOTSP** software from <https://github.com/MLopez-Ibanez/ACOTSPQAP/archive/refs/heads/master.zip> to the `./tuning/` directory and compile the `acotsp` executable using `make`.

```
unzip master.zip
make -C ACOTSPQAP-master acotsp
./ACOTSPQAP-master/acotsp --help
```

If the above gives an error, then the `acotsp` executable failed to compile for some reason. Maybe you are missing the C compiler or some files did not extract correctly.

4. Create a directory for executing the experiments and execute **irace**:

```
mkdir ./acotsp-arena/
# $IRACE_HOME is the installation directory of irace.
$IRACE_HOME/bin/irace
```

Or you can also execute **irace** from the R console using:

```
library("irace")
setwd("./tuning/")
irace_cmdline()
```

The most usual sources of error when running the above commands are:

- The **irace** package is not correctly installed. Please make sure that installing **irace** did not give any errors.
- The location of the files is not correct. Please make sure that you have:
 - The folder `tuning` and that it contains the files `scenario.txt`, `parameters-acotsp.txt`, `target-runner`, and the folders `instances`, `ACOTSPQAP-master` and `acotsp-arena`.

- The folder `instances` should contain the TSP instance files (`*.tsp`).
- The folder `ACOTSPQAP-master` should contain the executable `acotsp`. You should be able to invoke `./ACOTSPQAP-master/acotsp --help` without an error.
- The folder `acotsp-arena` should be empty.

5 Defining a configuration scenario

5.1 Target algorithm parameters

The parameters of the target algorithm are defined by a parameter file as described in [Section 5.1.7](#). Optionally, when executing **irace** from the R console, the parameters can be specified directly as an R object (see [Section 5.1.8](#)). For defining your parameters follow the guidelines provided in the following sections.

5.1.1 Parameter types

Each target parameter has an associated type that defines its domain and the way **irace** handles them internally. Understanding the nature of the domains of the target parameters is important to select appropriate types. The four basic types supported by **irace** are the following:

- *Real* parameters are numerical parameters that can take floating-point values within a given range. The range is specified as an interval ‘(<lower bound>, <upper bound>)’. This interval is closed, that is, the parameter value may eventually be one of the bounds. The possible values are rounded to a number of *decimal places* specified by the global option `digits` ([Section 5.1.6](#)). For example, given the default number of digits of 4, the values 0.12345 and 0.12341 are both rounded to 0.1234. Selected real-valued parameters can be optionally sampled on a logarithmic scale (base e).
- *Integer* parameters are numerical parameters that can take only integer values within the given range. Their range is specified as the range of real parameters and they can also be optionally sampled on a logarithmic scale (base e).
- *Categorical* parameters are defined by a set of possible values specified as ‘(<value 1>, ..., <value n>)’. The values are quoted or unquoted character strings. Empty strings and strings containing commas or spaces must be quoted.
- *Ordinal* parameters are defined by an *ordered* set of possible values in the same format as for categorical parameters. They are handled internally as integer parameters, where the integers correspond to the indexes of the values.



Boolean (or logical) parameters are best encoded as categorical ones with just two values rather than integer ones with domain (0,1). Some boolean parameters take an explicit value (0/1 or true/false) such as:

```
d1b "--d1b " c (0, 1)
```

Others are switches whose presence activates the parameter:

```
d1b "" c ("", "--d1b")
```

5.1.2 Parameter domains

For each target parameter, an interval or a set of values must be defined according to its type, as described above. There is no limit for the size of the set or the length of the interval, but keep in mind that larger domains could increase the difficulty of the tuning task. Choose always values that you consider relevant for the tuning. In case of doubt, we recommend to choose larger intervals, as occasionally best parameter settings may be not intuitive a priori. All intervals are considered as closed intervals.

It is possible to define parameters that will have always the same value. Such “*fixed*” parameters will not be tuned but their values are used when executing the target algorithm and they are affected by constraints defined on them. All fixed parameters must be defined as categorical parameters and have a domain of one element.

5.1.3 Parameter dependent domains

Domains that are dependent on the values of other parameters can be specified only for numerical parameters (both integer and real). To do so, the dependent domain must be expressed in function of another parameter, which must be a numerical parameter. The expression that defines a dependency must be written between quotes: (value, "expression") or ("expression", value) or ("expression", "expression").

The expressions can only use the following operators and R functions: +, -, *, /, %, min, max, round, floor, ceiling, trunc. If you need to use an operator or function not listed here, please contact us.



The user must ensure that the defined domain is valid at all times since **irace** currently is not able to detect possible invalid domains based on the expressions provided.

If you have a parameter **p2** that is just a transformation of another **p1**, then instead of using a dependent domain (left-hand side of the following example), it will be better to create a dummy parameter that controls the transformation (right-hand side) and do the transformation within **target-runner**. For example:

```
# With dependent domains
p1 "" r (0, 100)
p2 "" r ("p1", "p1 + 10")
```

should be

```
# With a dummy parameter
p1 "" r (0, 100)
p2dum "" r (0, 10)
```

and **target-runner** will compute $p2 = p2dum \cdot p1$.

5.1.4 Conditional parameters

Conditional parameters are active only when others have certain values. These dependencies define a hierarchical relation between parameters. For example, the target algorithm may have a parameter **localsearch** that takes values (**sa**, **ts**) and another parameter **ts-length** that only needs to be set if the first parameter takes precisely the value **ts**. Thus, parameter **ts-length** is conditional on **localsearch == "ts"**.

5.1.5 Forbidden parameter configurations

A line containing just **[forbidden]** ends the list of parameters and starts the list of forbidden expressions. Each line is a logical expression (in R syntax) containing parameter names as defined by the **parameterFile** (Section 5.1), values and logical operators. For a list of R logical operators see:

<https://stat.ethz.ch/R-manual/R-devel/library/base/html/Syntax.html>

If **irace** generates a parameter configuration that makes any of the logical expressions evaluate to **TRUE**, then the configuration is considered forbidden and it is never evaluated. This is useful when some combination of parameter values could cause the target algorithm to crash, consume excessive CPU time or memory, or when it is known that they do not produce satisfactory results.



Initial configuration (Section 5.5) that are forbidden will be discarded with a warning.

If the forbidden constraints provided are too strict, **irace** may produce the following error:

```
irace tried 100 times to sample from the model a configuration not forbidden
without success, perhaps your constraints are too strict?
```

In that case, it may be a good idea to reformulate the forbidden constraints as conditional parameters (Section 5.1.4), parameter-dependent domains (Section 5.1.3), repairing the configurations (Section 5.6) or post-processing within the target-algorithm (Section 10.9).

5.1.6 Global options

A line containing just `[global]` starts the definition of global options. The only global option currently implemented is **digits**, which controls the number of decimal digits for real valued parameters. Its default value is 4.

5.1.7 Parameter file format

For simplicity, the description of the parameters space is given as a table. Each line of the table defines a configurable parameter

`<name> <label> <type> <domain> [ | <condition> ]`

where each field is defined as follows:

- <name>** The name of the parameter as an unquoted alphanumeric string, e.g., `'ants'`.
- <label>** A *label* for this parameter. This is a string that will be passed together with the parameter to **targetRunner**. In the default **targetRunner** provided with the package (Section 5.2), this is the command-line switch used to pass the value of this parameter, for instance `"--ants "`.
The value of the parameter is concatenated *without separator* to the label when invoking **targetRunner**, thus *any whitespace in the label is significant*. Following the same example, when parameter **ants** takes value 5, the default **targetRunner** will pass the parameter as `"--ants 5"`.
- <type>** The type of the parameter, either *integer*, *real*, *ordinal* or *categorical*, given as a single letter: `'i'`, `'r'`, `'o'` or `'c'`. Numerical parameters can be sampled using a natural logarithmic scale with `'i,log'` and `'r,log'` (without spaces) for integer and real parameters, respectively.
- <domain>** The range or set of values of the parameter delimited by parentheses, e.g., `(0,1)` or `(a,b,c,d)`. See also parameter dependent domains (Section 5.1.3).

<condition> An optional *condition* that determines whether the parameter is enabled or disabled, thus making the parameter conditional. If the condition evaluates to false, then no value is assigned to this parameter, and neither the parameter value nor the corresponding label are passed to **targetRunner**. The condition must follow the same syntax as those for specifying forbidden configurations (see below), that is, it must be a valid R logical expression². The condition may contain the name of other parameters as long as the dependency graph does not contain any cycle. Otherwise, **irace** will detect the cycle and stop with an error.



Categorical and ordinal parameters are always treated as strings. Given a parameter like:

```
a " c (0, 5, 10, 20)
```

then, a condition like `a > 10` will be true when `a` is 5, because comparisons between strings are lexicographic and "10" is sorted before "5". As a work-around, you can convert the string to numeric in the condition with `as.numeric(a)`.

As an example, Figure 2 shows the parameters file of the **ACOTSP** scenario.

```
# name      switch      type values      [conditions (using R syntax)]
algorithm    "--"          c  (as,mmas,eas,ras,acs)
localsearch  "--localsearch " c  (0, 1, 2, 3)
alpha        "--alpha "    r  (0.00, 5.00)
beta         "--beta "    r  (0.00, 10.00)
rho          "--rho "      r  (0.01, 1.00)
ants         "--ants "     i  (5, 100)
nnls         "--nnls "   i  (5, 50)      | localsearch %in% c(1, 2, 3)
q0           "--q0 "    r  (0.0, 1.0)   | algorithm == "acs"
dlb          "--dlb "    c  (0, 1)      | localsearch %in% c(1,2,3)
rasrank      "--rasranks " i  (1, "ants")  | algorithm == "ras"
elitistants  "--elitistants " i  (1, 750)     | algorithm == "eas"

[forbidden]
## Examples of valid logical operators are:
## == != >= <= > < & | ! %in%
(alpha == 0.0) & (beta == 0.0)
```

Figure 2: Parameter file (`parameters.txt`) for tuning **ACOTSP**.

5.1.8 Parameters R format

The target parameters are stored in an R list that you can obtain from the R console using the following command:

```
parameters <- readParameters(file = "parameters.txt")
```

See the help of the `readParameters` function (`?readParameters`) for more information. The structure of the parameter list that is created is as follows:

²For a list of R operators see: <https://stat.ethz.ch/R-manual/R-devel/library/base/html/Syntax.html>

names	Vector that contains the names of the parameters.
types	Vector that contains the type of each parameter 'i', 'c', 'r', 'o'.
switches	Vector that contains the labels of the parameters. e.g., switches to be used for the parameters on the command line.
domain	List of vectors, where each vector may contain two values (minimum, maximum) for real and integer parameters, or a set of values for categorical and ordinal parameters.
conditions	List of R logical expressions, with variables corresponding to parameter names.
isFixed	Logical vector that specifies which parameter is fixed and, thus, it does not need to be tuned.
transform	Vector that contains the transformation of each parameter. Currently, it can take values '' (no transformation, default) of 'log' (natural logarithmic transformation).
nbParameters	An integer, the total number of parameters.
nbFixed	An integer, the number of parameters with a fixed value.
nbVariable	Number of variable (i.e., to be tuned) parameters.
isDependent	Logical vector that specifies which parameter defines a dependent domain.
forbidden	List of R logical expressions that cannot evaluate to TRUE for any evaluated configuration.

The following example shows the structure of the **parameters** R object for the **algorithm**, **ants** and **q0** parameters of the **ACOTSP** scenario:

```
> str(parameters, vec.len = 10)

Classes 'ParameterSpace', 'R6' <ParameterSpace>
Public:
 .params: list
 as_character: function ()
 clone: function (deep = FALSE)
 conditions: list
 depends: list
 domains: list
 forbid_configurations: function (x)
 forbidden: NULL
 get: function (x)
 get_ordered: function ()
 hierarchy: 1 1 2
 initialize: function (... , forbidden = NULL, verbose = 0L)
 isFixed: FALSE FALSE FALSE
 names: algorithm ants q0
 names_fixed:
 names_numeric: ants q0
 names_variable: algorithm ants q0
 nbFixed: 0
 nbParameters: 3
```

```
nbVariable: 3
switches: -- --ants --q0
types: c i r
```

5.2 Target algorithm runner

The evaluation of a candidate configuration on a single instance is done by means of a user-given auxiliary program or, alternatively, a user-given R function. The function (or program name) is specified by the option **targetRunner**. The **targetRunner** must return the cost value (e.g., cost of the best solution found) of the evaluation; unless computing the cost requires information from all the configurations evaluated on an instance, e.g., when evaluating multi-objective algorithms with unknown normalisation bounds (see [Section 5.3](#) for details).



The objective of **irace** is to minimize the cost value returned by the target algorithm. If you wish to maximize, you can multiply the cost by **-1** before returning it to **irace**.

5.2.1 Target runner as an executable program

When **targetRunner** is an auxiliary executable program, it is invoked for each candidate configuration, passing as arguments:

```
<id_configuration> <id_instance> <seed> <instance> [bound] <configuration>
```

id_configuration	an alphanumeric string that uniquely identifies a configuration;
id_instance	an alphanumeric string that uniquely identifies an instance;
seed	seed for the random number generator to be used for this evaluation, ignore the seed for deterministic algorithms;
instance	string giving the instance to be used for this evaluation;
bound	optional execution time bound. Only provided when the boundMax option is set in the scenario, see Section 10.5 ;
configuration	the pairs parameter label-value that describe this candidate configuration. Typically given as command-line switches to be passed to the executable program.

The experiment list shown in [Section 5.2.2](#), would result in the following execution line:

```
target-runner 1 113 734718556 /home/user/instances/tsp/2000-533.tsp \
--eas --localsearch 0 --alpha 2.92 --beta 3.06 --rho 0.6 --ants 80
```

The command line switches that describe the candidate configuration are constructed by appending to each parameter label (switch), *without separator*, the value of the parameter, following the order given in the parameter table. The program **targetRunner** must print a real number, which corresponds to the cost measure of the candidate configuration for the given instance and optionally its execution time (mandatory when **maxTime** is used and/or when the **capping** option is enabled). The working directory of **targetRunner** is set to the execution directory specified by the option **execDir**. This allows the user to execute independent runs of **irace** in parallel using different values for **execDir**, without the runs interfering with each other.

5.2.2 Target runner as an R function

When `targetRunner` is an R function, it is invoked for each candidate configuration as:

```
targetRunner(experiment, scenario)
```

where `experiment` is a list that contains information about configuration and instance to execute one experiment, and `scenario` is the scenario list. The structure of the `experiment` list is as follows:

<code>id_configuration</code>	an alphanumeric string that uniquely identifies a configuration;
<code>id_instance</code>	an alphanumeric string that uniquely identifies an instance;
<code>seed</code>	seed to be used for this evaluation;
<code>instance</code>	string giving the instance to be used for this evaluation;
<code>bound</code>	optional execution time bound;
<code>configuration</code>	1-row data frame with a column per parameter name;

The following is an example of an experiment list for the **ACOTSP** scenario:

```
> print(experiment)

$id_configuration
[1] 1

$id_instance
[1] 48

$seed
[1] 1498426593

$configuration
  algorithm localsearch alpha beta rho ants nnls q0 dlb rasrank
1      as           0     1    1 0.95  10  NA NA <NA>    NA
  elitists time
1      NA     5

$instance
[1] "./instances/2000-918.tsp"

$switches
  algorithm      localsearch      alpha      beta
  "--" "--localsearch " "--alpha " "--beta "
      rho      ants      nnls      q0
"--rho " "--ants " "--nnls " "--q0 "
      dlb      rasrank  elitists      time
"--dlb " "--rasranks " "--elitists " "--time "
```

If you need the `switches` of each parameter (or any other information about the parameters), you can obtain it from `scenario$parameters`.

You can find an example that calls MATLAB from R using this approach here: <https://github.com/MLopez-Ibanez/irace/blob/master/inst/examples/matlabr/scenario.txt>.

If `targetEvaluator` is NULL, then the `targetRunner` function must return a list with at least one element "cost", the numerical value corresponding to the evaluation of the given configuration on the given instance. A cost of `Inf` is accepted and results in the immediate rejection of the configuration (see Section 10.10).

If the scenario option `maxTime` is non-zero or if the `capping` option is enabled, then the list must contain at least another element "time" that reports the execution time for this call to `targetRunner`.

The return list may also contain the following optional elements that are used by `irace` for reporting errors in `targetRunner`:

`error` is a string used to report an error;
`outputRaw` is a string used to report the raw output of calls to an external program or function;
`call` is a string used to report how `targetRunner` called an external program or function;

5.3 Target evaluator

Normally, `targetRunner` returns the cost of the execution of a candidate configuration (see Section 5.2). However, there are cases when the cost evaluation must be delayed until all candidate configurations in a race have been executed on a instance.

The `targetEvaluator` option defines an auxiliary program (or an R function) that allows postponing the evaluations of the candidate configurations. For each instance seen, the program `targetEvaluator` is only invoked after all the calls to `targetRunner` for all alive candidate configurations on the same instance have already finished.



When using `targetEvaluator`, `targetRunner` must not return the evaluation of the configuration. If `maxTime` is used, `targetRunner` must return only execution time.

As an example, `targetEvaluator` may be used to dynamically find normalization bounds for the output returned by an algorithm for each individual instance. In this case, `targetRunner` will save the output of the algorithm, then the first call to `targetEvaluator` will examine the output produced by all calls to `targetRunner` for the same instance, update the normalization bounds and return the normalized output. Subsequent calls to `targetEvaluator` for the same instance will simply return the normalized output.

A similar need arises when using quality measures for multi-objective optimization algorithms, such as the hypervolume, which typically require specifying reference points or sets. By using `targetEvaluator`, it is possible to dynamically compute the reference points or sets while `irace` is running. Examples are provided at `examples/hypervolume`. See also Section 10.3 for more information on how to tune multi-objective algorithms.

5.3.1 Target evaluator executable program

When `targetEvaluator` is an auxiliary executable program, it is invoked for each candidate with the following arguments:

```
<id_configuration> <id_instance> <seed> <instance> <num_configurations> <all_conf_id>
```

<code>id_configuration</code>	an alphanumeric string that uniquely identifies a configuration;
<code>id_instance</code>	an alphanumeric string that uniquely identifies an instance;
<code>seed</code>	seed to be used for this evaluation;
<code>instance</code>	string giving the instance to be used for this evaluation;
<code>num_configurations</code>	number of alive candidate configurations;
<code>all_conf_id</code>	list of IDs of the alive configurations separated by whitespace.

The `targetEvaluator` executable must print a numerical value corresponding to the cost measure of the candidate configuration on the given instance.

5.3.2 Target evaluator R function

When `targetEvaluator` is an R function, it is invoked for each candidate configuration as:

```
targetEvaluator(experiment, num_configurations, all_conf_id, scenario,
               target_runner_call)
```

where `experiment` is a list that contains information about one experiment (see [Section 5.2.2](#)), `num_configurations` is the number of configurations alive in the race, `all_conf_id` is the vector of IDs of the alive configurations, `scenario` is the scenario list and `target_runner_call` is the string of the `targetRunner` execution line.

The function `targetEvaluator` must return a list with one element "cost", the numerical value corresponding to the cost measure of the given configuration on the given instance.

The return list may also contain the following optional elements that are used by `irace` for reporting errors in `targetEvaluator`:

<code>error</code>	is a string used to report an error;
<code>outputRaw</code>	is a string used to report the raw output of calls to an external program or function;
<code>call</code>	is a string used to report how <code>targetEvaluator</code> called an external program or function;

5.4 Training instances

The `irace` options `trainInstancesDir` and `trainInstancesFile` specify where to find the training instances. If you only set the value of `trainInstancesDir` (e.g., to `./Instances`), `irace` will consider all files within that directory as training instances.

Otherwise, the value of `trainInstancesFile` may specify a text file. The format of this file is one instance per line. Within each line, elements separated by white-space will be parsed as separate arguments to be supplied to `targetRunner`. This allows defining instance-specific parameter settings. Quoted strings will be parsed as a single argument. The following example shows a training instance file for the **ACOTSP** scenario:

```
# Example training instances file
100/100-1_100-2.tsp --time 1
100/100-1_100-3.tsp --time 2
100/100-1_100-4.tsp --time 3
```

Figure 3: Training instances file for tuning **ACOTSP**.

The value of `trainInstancesDir`, if set, is always prefixed to the instance name, that is, the instances names are treated as relative to this directory. For example, given the above file as `trainInstancesFile` and `trainInstancesDir="./Instances"`, then a possible invocation of `targetRunner` would be:

```
target-runner 1 4 5718 ./Instances/100/100-1_100-2.tsp --time 1 --alpha 2.92 ...
```

Training instances do not need to be files, **irace** just passes the elements of each line as arguments to `targetRunner`, thus each line may denote the name of a benchmark function or a label, plus instance-specific settings, that the target algorithm understands. Each line may even be the command-line parameters required to call an instance generator within `targetRunner`. When the instances do not represent actual files, then `trainInstancesDir` is usually set to the empty string (`--train-instances-dir=""`). For example,

```
# Example training instances file
rosenbrock_20 --function=12 --nvar 20
rosenbrock_30 --function=12 --nvar 30
rastrigin_20 --function=15 --nvar 20
rastrigin_30 --function=15 --nvar 30
```

Optionally, when executing **irace** from the R console, the list of instances might be provided explicitly by means of the variable `scenario$instances`. Thus, the previous example would be equivalent to:

```
scenario$instances <- c("rosenbrock_20 --function=12 --nvar 20",
  "rosenbrock_40 --function=12 --nvar 30",
  "rastrigin_20 --function=15 --nvar 20",
  "rastrigin_40 --function=15 --nvar 30")
```

By default, **irace** assumes that the target algorithm is stochastic (the value of the option `deterministic` is 0), thus, the same configuration can be executed more than once on the same instance and obtain different results. In this case, **irace** generates pairs (`instance,seed`) by generating a random seed for each instance. In other words, configurations evaluated on the same instance use the same random seed. This is a well-known variance reduction technique called *common random numbers* [15]. If all available pairs are used within a run of **irace**, new pairs are generated with different seeds, that is, a configuration evaluated more than once per instance will use different random seeds.

If `deterministic` is set to 1, then each instance will be used at most once per race. This setting should only be used for target algorithms that do not have a stochastic behavior and, therefore, executing the target algorithm on the same instance several times with different seeds does not make sense.



If `deterministic` is active and the number of training instances provided to **irace** is less than `firstTest` (default: 5), no statistical test will be performed on the race.

Finally, **irace** randomly re-orders the sequence of instances provided. This random sampling may be disabled by using the option `sampleInstances` (`--sample-instances 0`) if keeping the order provided in the instance file is important.



We advise to always sample instances to prevent biasing the tuning due to the instance order. See also [Section 10.7](#)

5.5 Initial configurations

The scenario option `configurationsFile` allows specifying a text file that contains an initial set of configurations to start the execution of **irace**. If the number of initial configurations supplied in the file is less than the number of configurations required by **irace** in the first iteration, additional configurations will be sampled uniformly at random.

The format of the configurations file is one configuration per line, and one parameter value per column. The first line must give the parameter name corresponding to each column (names must match those given in the parameters file). Each configuration must satisfy the parameter conditions (NA should be used for those parameters that are not enabled for a given configuration) and not be forbidden by the constraints that define forbidden configurations (Section 5.1.5), if any.

Figure 4 gives an example file that corresponds to the **ACOTSP** scenario.

```
## Initial candidate configuration for irace
algorithm localsearch alpha beta rho ants nnls dlb q0 rasrank elitstants
as          0          1.0  1.0  0.95 10  NA  NA  0  NA      NA
```

Figure 4: Initial configuration file (`default.txt`) for tuning **ACOTSP**.

We advise to use this feature when a default configuration of the target algorithm exists or when different sets of good parameter values are known. This will allow **irace** to start the search from those parameter values and attempt to improve their performance.

5.6 Repairing configurations

In some problems, the parameter values require complex constraints that cannot be implemented by constraints defined in the parameter space (Section 5.1.5). The scenario option `repairConfiguration` can be set to a user-defined R function that takes a single configuration generated by **irace** and returns a “*repaired*” configuration, thus allowing the implementation of any rules necessary to satisfy arbitrary constraints on parameter values. The `repairConfiguration` function is called after generating a configuration and before checking for forbidden configurations. The first argument is a 1-row `data.frame` with parameter names as the column names and the second argument is the `parameters` list (Section 5.1.8). An example that makes all real-valued parameters sum up to one would be:

```
repairConfiguration = function(configuration, parameters)
{
  isreal <- names(which(parameters$types[colnames(configuration)] == "r"))
  # This ignores 'digits'
  c_real <- unlist(configuration[isreal])
  c_real <- c_real / sum(c_real)
  configuration[isreal] <- c_real
  return(configuration)
}
```

The following example forces three specific parameters to be in increasing order:

```
repairConfiguration = function(configuration, parameters)
{
  columns <- c("p1", "p2", "p3")
```

```
# cat("Before"); print(configuration)
configuration[columns] <- sort(unlist(configuration[columns], use.names=FALSE))
# cat("After"); print(configuration)
return(configuration)
}
```

The above code can be specified directly in the `scenarioFile`, by default `scenario.txt`.

6 Parallelization

A single run of **irace** can be done much faster by executing the calls to **targetRunner** (the runs of the target algorithm) in parallel. There are four ways to parallelize a single run of **irace**:

1. **Parallel processes:** The option `parallel` executes multiple calls to **targetRunner** in parallel within a single computer, by means of the **parallel** R package. For example, adding `--parallel N` to the command line of **irace** will launch in parallel up to N calls of the target algorithm. When using this option within a computing cluster, **irace** will be submitted as a *job* in some way that tells the cluster to “reserve” N CPUs (or tasks depending on the cluster) within a single cluster node (a single machine).
2. **MPI:** By enabling the option `mpi`, calls to **targetRunner** will be executed in parallel by using the message passing interface (MPI) protocol (requires the **Rmpi** R package). In this case, the option `parallel` controls the number of slave nodes used by **irace**. For example, adding `--mpi 1 --parallel N` to the command-line will create N slaves + 1 master, and execute up to N calls of **targetRunner** in parallel.

The user is responsible for setting up the required MPI environment. MPI is commonly available in computing clusters and requires launching **irace** in some particular way. An example script for using MPI mode in a SGE cluster is given at `$IRACE_HOME/bin/parallel-irace-mpi`.

By default, **irace** dynamically balances the load among nodes, however, this may significantly increase communication overhead in some parallel environments and disabling `loadBalancing` may be faster.

3. **Batch queue mode:** Some computing clusters work by submitting jobs to a batch queue and waiting for the jobs to finish. With the option `batchmode` (`--batchmode [sgelpbs|torque|slurm]`), **irace** will launch in parallel as many calls of **targetRunner** as possible and use a cluster-specific method to wait for jobs to finish. In this mode, `parallel` controls how many jobs are queued and should be set to the queue limit of your cluster. If your cluster type is not supported or not working as expected, please submit a pull request (<https://github.com/MLopez-Ibanez/irace/pulls>) adding support to your cluster type. See the examples in `$IRACE_HOME/examples/batchmode-cluster/`.



In batchmode, **irace** runs in the submission node of the cluster, hence, **irace** is not submitted to the cluster as a job (that is, neither `qsub` nor `squeue` should be used to invoke **irace** itself). The user must call the appropriate job submission command (e.g., `qsub` or `squeue`) from **targetRunner** with the appropriate settings for their cluster, that is, **targetRunner** submits one job to the cluster and prints a single string: The job ID that allows **irace** to determine the status of the running job. Moreover, the use of a separate **targetEvaluator** script is required to collect the results of **targetRunner** and return them to **irace**.

4. `targetRunnerParallel`: This option allows users to fully control the parallelization of the execution of `targetRunner`. Its value must be an R function that will be invoked by `irace` as follows:

```
targetRunnerParallel(experiments, exec_target_runner, scenario, target_runner)
```

where `scenario` is the list describing the configuration scenario (Section 5); `experiments` is a list that describes the configurations and instances to be executed (see Section 5.2 for a description); `target_runner` is the function that calls the target algorithm and it is the same as `targetRunner`, if the latter is a function, or it is a call to `target_runner_default`, if `targetRunner` is the path to an executable; and `exec_target_runner` is an internal function within `irace` that takes care of executing `target_runner`, check its output and, possibly, retry in case of error (see `targetRunnerRetries`). The `targetRunnerParallel` function should call the given `target_runner` function for each element in the `experiments` list, possibly using `exec_target_runner` as a wrapper. A trivial example would be:

```
targetRunnerParallel <- function(experiments, exec_target_runner, scenario,
                                target_runner)
{
  lapply(experiments, exec_target_runner, scenario = scenario,
         target_runner = target_runner)
}
```

However, the user is free to set up the calls in any way, perhaps implementing their own replacement for `target_runner` and/or `exec_target_runner`. The user may load and call other R packages, such as `batchtools` (<https://cran.r-project.org/package=batchtools>).

The only requirement is that the `targetRunnerParallel` function must return a list of the same length as `experiments`, where each element is the output expected from the corresponding call to `targetRunner` (see Section 5.2).

The following is an example of the output of a call to `targetRunnerParallel` with 2 experiments, in which the execution time is not reported:

```
print(output)

## [[1]]
## [[1]]$cost
## [1] 42220233
##
## [[1]]$time
## numeric(0)
##
##
## [[2]]
## [[2]]$cost
## [1] 34446448
##
## [[2]]$time
## numeric(0)
```

The best option will depend on the resources available to you. Option 1 is usually the fastest and simplest to setup. Running on a node (machine) with 128 CPUs will be faster than running on 8 nodes with 16 CPUs because the communication between nodes required by MPI can be very slow depending on the cluster. Option 2 may be faster if `irace` generates more configurations per iteration than the number of CPUs of a single node. However, depending on the configuration of your cluster, requesting many CPUs may require waiting in the queue a long time. Option 3 may be the slowest since `irace` has to check the queue frequently. However, `irace` will start running experiments as soon as 1 CPU is available, thus option 3 may actually finish earlier than the other options if there is always some CPUs available in the cluster but the queue for requesting many CPUs at once is very long.

As a rule-of-thumb, if you only have access to a single machine, then you only need option 1. If you have access to a computing cluster with multiple machines, then use option 1 with the maximum number of CPUs that a single node has in your computing cluster. If that number is 64 or more, it should be enough unless a single run of `irace` evaluates thousands of configurations. Otherwise, investigate option 2. If option 2 does not work, then investigate option 3.

7 Testing (Validation) of configurations

Once the tuning process is finished, **`irace`** returns a set of configurations corresponding to the elite configurations at the end of the run, ordered from best to worst. In order to evaluate the generality of these configurations without looking at their performance on the training set, **`irace`** offers the possibility of evaluating these configurations on a test instance set, typically different from the training set used during the tuning phase. These evaluations will use the same settings for parallel execution, `targetRunner` and `targetEvaluator`.

The test instances can be specified by the options `testInstancesDir` and/or `testInstancesFile`, or by setting directly the variable `scenario$testInstances`. These options behave similarly to their counterparts for the training instances (Section 5.4). In particular, each test instance is assigned a different seed in the same way as done for the training instances. In principle, **`irace`** evaluates each configuration on each testing instance just once, because evaluating one run on n instances is always better than evaluating n' runs on n/n' instances [2]. However, if the number of instances is limited, one can always duplicate instances as needed in the `testInstancesFile`, and **`irace`** will assign a different random seed to each instance. An example of the output produced by **`irace`** when testing is shown in Fig. 5.

The options `testNbElites` and `testIterationElites` control which configurations are evaluated during the testing phase. In particular, setting `testIterationElites = 1` will test not only the final set of elite configurations (those returned at the end of the training phase), but also the set of elites at the end of each race (iteration). The option `testNbElites` limits the maximum number of configurations considered within each set. Some examples:

- `testIterationElites = 0; testNbElites = 1` means that only the best configuration found during the run of **`irace`**, the final best, will be used in the testing phase.
- `testIterationElites = 1; testNbElites = 1` will test, in addition to the final best, the best configuration found at each iteration.
- `testIterationElites = 1; testNbElites = 2` will test the two best configurations found at each iteration, in addition to the final best and second-best configurations.

The testing can be also (re-)executed at a later time by using the following R command (but you may need to override `testNbElites` and `testIterationElites`):

```
testing_fromlog(logFile = "./irace.Rdata", testNbElites = 1)
```

The above line will load the scenario setup from `logFile` to perform the testing. The testing results will be stored in the R object `iraceResults$testing`, which is saved in the file specified by `scenario$logFile`. The structure of the object is described in [Section 9.2](#). For examples on how to analyse the results see [Section 9.3](#).

Another alternative is to test a specific set of configurations using the command-line option `--only-test` as follows:

```
irace --only-test configurations.txt
```

where `configurations.txt` has the same format as the set of initial configurations ([Section 5.5](#)).

8 Recovering irace runs

Problems like power cuts, hardware malfunction or the need to use computational power for other tasks may occur during the execution of **irace**, terminating a run before completion. At the end of each iteration, **irace** saves an R data file (`logFile`, by default `./irace.Rdata`) that not only contains information about the tuning progress ([Section 9.2](#)), but also internal information that allows recovering an incomplete execution.

To recover an incomplete **irace** run, set the option `recoveryFile` to the log file previously produced, and **irace** will continue the execution from the last saved iteration. The state of the random generator is saved and loaded, therefore, as long as the execution is continued in the same machine, the obtained results will be exactly the same as executing **irace** in one step (external factors, such as CPU load and disk caches, may affect the target algorithm and that may affect the results). You can specify the `recoveryFile` from the command-line or from the scenario file, and execute **irace** as described in [Section 4](#). For example, from the command-line use:

```
irace --recovery-file "./irace-backup.Rdata"
```



When recovering a previous run, **irace** will try to save data on the file specified by the `logFile` option. Thus, you must specify different files for `logFile` and `recoveryFile`. Before recovering, we strongly advise to rename the saved R data file as in the example above, which uses `"irace-backup.Rdata"`.



Do not change anything in the log file or the scenario file before recovering, as it may have unexpected effects on the recovered run of **irace**. In case of doubt, please contact us first ([Section 13](#)). In particular, it is not possible to continue a run of **irace** by recovering with a larger budget. Results will **not** be the same as running **irace** from the start with the largest budget. An alternative is to use the final configurations from one run as the initial configurations of a new run.



If your scenario uses `targetEvaluator` ([Section 5.3](#)) and `targetEvaluator` requires files created by `targetRunner`, then recovery will fail if those files are not present in the `execDir` directory. This can happen, for example, if you recover from a different directory than the one from which **irace** was initially executed, or when `execDir` is set to a temporary directory for every **irace** run. Thus, you need to copy the contents of the previous `execDir` into the new one.

```

...

# Testing of elite configurations: 5
# Testing iteration configurations: TRUE
# 2023-10-01 13:35:17 BST: Testing configurations (in no particular order): 2 29 3 20 50 34 47 86 74 111 106 92 123 134 130 119
  algorithm localsearch  alpha  beta  rho ants nnls  q0 dlb rasrank elitistants
2      acs              3 1.1275 3.3469 0.6471 36 43 0.9053 0 NA NA
29     ras              3 4.5152 5.6811 0.7818 11 43 NA 0 13 NA
3      eas              3 3.7246 5.5153 0.5998 75 29 NA 1 NA 91
20     acs              2 1.4734 0.1808 0.4304 5 39 0.5665 1 NA NA
50     ras              3 2.2804 4.5294 0.5508 18 32 NA 1 12 NA
34     acs              3 2.2044 2.4923 0.7243 12 29 0.4628 0 NA NA
47     acs              2 1.7046 6.3908 0.3256 5 36 0.2288 0 NA NA
86     ras              3 3.9808 3.4401 0.2191 20 25 NA 1 15 NA
74     mmas             3 1.1135 0.6356 0.3765 11 33 NA 1 NA NA
111    acs              2 1.2881 6.4311 0.5859 10 23 0.2126 0 NA NA
106    ras              3 3.2756 5.5035 0.9277 18 32 NA 0 10 NA
92     acs              2 1.6234 9.2153 0.1101 8 40 0.3412 0 NA NA
123    acs              2 1.0584 5.8121 0.5587 10 35 0.2575 0 NA NA
134    acs              2 1.6144 7.8972 0.1999 8 33 0.1415 1 NA NA
130    acs              2 1.5899 6.9391 0.5529 8 26 0.3796 0 NA NA
119    acs              2 1.4432 6.4746 0.4582 5 28 0.1931 0 NA NA
# 2023-10-01 13:48:55 BST: Testing results (column number is configuration ID in no particular order):
  seeds 2 29 3 20 50 34 47 86 74 111 106 92 123
1t 1385446146 23609115 23432130 23400390 23403445 23383391 23439897 23380458 23455774 23355888 23382496 23417159 23404000 23492199 23425
2t 1396979195 23327035 23208321 23186931 23288822 23181225 23228238 23265383 23201401 23279356 23188848 23171998 23314132 23244891 23207
3t 448912041 23111373 23246027 23092265 23130590 23016527 23056027 23070519 23084400 23041364 23118905 23063197 23115280 23069064 23094
4t 732530909 23128507 23159085 23119773 23290945 23063257 23095963 23110872 23151621 23053285 23133797 23139894 23148536 23101345 23084
5t 947545849 23340533 23347633 23298513 23274844 23236071 23274590 23246473 23292452 23232446 23242908 23264111 23219377 23258853 23234
6t 195435663 23532942 23470894 23496016 23533573 23423201 23538541 23526779 23500842 23497417 23525281 23464461 23510384 23454321 23478
7t 798649446 23440321 23430267 23331305 23422861 23375048 23454558 23491515 23405251 23509688 23386282 23508539 23438775 23432504 23438
8t 1692971486 23376183 23262794 23300201 23286312 23258955 23368034 23301552 23263607 23231022 23254721 23271971 23236607 23227995 23280
9t 896414478 23375100 23296796 23375905 23293303 23293631 23428815 23292599 23376861 23314826 23312995 23369543 23333975 23355279 23352
10t 1371337352 23222214 23163714 23110039 23126033 23089358 23198436 23080692 23146608 23106011 23092382 23087101 23108507 23096788 23126
# 2023-10-01 13:48:55 BST: Finished testing

```

Figure 5: Sample text output of **irace** when evaluating on test instances.

9 Output and results

During its execution, **irace** prints information about the progress of the tuning in the standard output. Additionally, after each iteration, an R data file is saved (**logFile** option) containing the state of **irace**.

9.1 Text output

Figure 6 shows the output, up to the end of the first iteration, of a run of elitist **irace** applied to the **ACOTSP** scenario with 1000 evaluations as budget.

First, **irace** gives the user a warning informing that it has found a file with the default scenario filename and it will use it. Then, general information about the selected **irace** options is printed:

- **nbIterations** indicates the minimum number of iterations **irace** has calculated for the scenario. Depending on the development of the tuning the final iterations that are executed can be more.
- **minNbSurvival** indicates the minimum number of alive configurations that are required to continue a race. When less configurations are alive the race is stopped and a new iteration begins.
- **nbParameters** is the number of parameters of the scenario.
- **seed** is the number that was used to initialize the random number generator in **irace**.
- **confidence level** is the confidence level of the statistical test.
- **budget** is the total number of evaluations available for the tuning.
- **time budget** is the maximum execution time available for the tuning.
- **mu** is a value used for calculating the minimum number of iterations.
- **deterministic** indicates if the target algorithm is assumed to be deterministic.

At each iteration, information about the progress of the execution is printed as follows:

- **experimentsUsed** is the number of experiments from the total budget that have been used up to the current iteration.
- **timeUsed** is the execution time used so far in the experiments. Only available when reported in the **targetRunner** (activate it with the **maxTime** option).
- **remainingBudget** is the number of experiments that have not been used yet.
- **boundEstimate** estimation of the mean execution time. This is used to calculate the remaining budget when **maxTime** is used.
- **currentBudget** is the number of evaluations **irace** has allocated to the current iteration.
- **nbConfigurations** is the number of configurations **irace** will use in the current iteration. In the first iteration, this number of configurations include the initial configurations provided; in later iterations, it includes the elite configurations from the previous iterations.

After the iteration information, a table shows the progress of the iteration execution. Each row of the table gives information about the execution of an instance in the race. The first column contains a symbol that describes the results of the statistical test:

- |x| No statistical test was performed for this instance. The options **firstTest** and **eachTest** control on which instances the statistical test is performed.
- |-| Statistical test performed and configurations have been discarded. The column **Alive** gives an indication of how many configurations have been discarded.
- |=| Statistical test performed and no configurations have been discarded. This means **irace** needs to evaluate more instances to identify the best configurations.
- ||| This indicator exists only for the elitist version of **irace**. It indicates that the statistical test was performed and some elite configurations appear to show bad performance and could be discarded but they are kept because of the elitist rules. See option **elitist** in [Section 11](#) for more information.

Other columns have the following meaning:

Instance: Index of (**instance**,**seed**) pair executed. This number corresponds to the row in the data frame returned by `get_instanceID_seed_pairs()`. See [Section 9.2](#) for more information. This index is different from the instance ID passed to **targetRunner**.

Bound: Only when **capping** is enabled. Execution time used as bound for the execution of new candidate configurations.

Alive: Number of configurations that have not been discarded after the statistical test was performed.

Best: ID of the best configuration according to the instances seen so far in this race (i.e., not including previous iterations).

Mean best: Mean cost value of the best configuration across the instances seen so far in this race (not globally). Equivalent to the concept of “iteration-best” in other algorithms.

Exp so far: Number of experiments performed so far.

W time: Wall-clock time spent on this instance.

rho, **KenW**, and **Qvar**: Spearman’s rank correlation coefficient ρ , Kendall’s concordance coefficient W , and a variance measure described in [17], respectively, of the configurations across the instances evaluated so far in this iteration. These measures evaluate how consistent is the performance of the configurations across the instances. Values close to 1 for **rho** and **KenW** and values close to 0 for **Qvar** indicate that the scenario is highly homogeneous. For heterogeneous scenarios, we provide advice in [Section 10.7](#).

Finally, **irace** outputs the best configuration found and a list of the elite configurations. The elite configurations are configurations that did not show statistically significant difference during the race; they are ordered according to their mean performance on the executed instances.

```

#-----
# irace: An implementation in R of (Elitist) Iterated Racing
# Version: 4.4.0.d5a98af
# Copyright (C) 2010-2025
# Manuel Lopez-Ibanez <manuel.lopez-ibanez@manchester.ac.uk>
# Jeremie Dubois-Lacoste
# Leslie Perez Caceres <leslie.perez.caceres@ulb.ac.be>
#
# This is free software, and you are welcome to redistribute it under certain
# conditions. See the GNU General Public License for details. There is NO
# WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
#
# irace builds upon previous code from the race package:
#   race: Racing methods for the selection of the best
#   Copyright (C) 2003 Mauro Birattari
#-----
# installed at: /home/manu/R/x86_64-pc-linux-gnu-library/4.3/irace
# called with: --parallel 2
# 2026-01-31 09:58:25 GMT: Reading parameter file '/home/manu/work/irace/git/devel-examples/vignette-example/parameters.txt'.
# 2026-01-31 09:58:25 GMT: 1 expression(s) specifying forbidden configurations read.
# 2026-01-31 09:58:25 GMT: Read 1 configuration(s) from file '/home/manu/work/irace/git/devel-examples/vignette-example/default.txt'
# 2026-01-31 09:58:25 GMT: Initialization
# Elitist race
# Elitist new instances: 1
# Elitist limit: 2
# nbIterations: 5
# minNbSurvival: 5
# nbParameters: 11
# seed: 687542627
# confidence level: 0.95
# budget: 1000
# mu: 5
# deterministic: FALSE

# 2026-01-31 09:58:25 GMT: Iteration 1 of 5
# experimentsUsed: 0
# remainingBudget: 1000
# currentBudget: 200
# nbConfigurations: 33
# Markers:
#   x No test is performed.
#   c Configurations are discarded only due to capping.
#   - The test is performed and some configurations are discarded.
#   = The test is performed but no configuration is discarded.
#   ! The test is performed and configurations could be discarded but elite configurations are preserved.
#   . Alive configurations were already evaluated on this instance and nothing is discarded.
#   : All alive configurations are elite, but some need to be evaluated on this instance.

+-----+-----+-----+-----+-----+-----+-----+-----+
| Instance| Alive| Best| Mean best| Exp so far| W time| rho|KenW| Qvar|
+-----+-----+-----+-----+-----+-----+-----+-----+
|x| 1| 33| 18| 33569063.00| 33|00:01:32| NA| NA| NA|
|x| 2| 33| 18| 33083049.00| 66|00:01:31|+0.98|0.99|0.0074|
|x| 3| 33| 31| 33159792.00| 99|00:01:50|+0.97|0.98|0.0062|
|x| 4| 33| 31| 33077667.25| 132|00:01:33|+0.98|0.98|0.0050|
|-| 5| 3| 31| 33040827.00| 165|00:01:32|-0.05|0.16|0.5124|
+-----+-----+-----+-----+-----+-----+-----+-----+
Best-so-far configuration: 31 mean value: 33040827.00
Description of the best-so-far configuration:
.ID. algorithm localsearch alpha beta rho ants nnls q0 dlb rasrank elitists time .PARENT.
31 31 acs 3 4.2717 0.2871 0.9362 6 17 0.918 1 NA NA 5 NA

# 2026-01-31 10:06:25 GMT: Elite configurations (first number is the configuration ID; listed from best to worst according to the sum of
algorithm localsearch alpha beta rho ants nnls q0 dlb rasrank elitists time
31 acs 3 4.2717 0.2871 0.9362 6 17 0.918 1 NA NA 5
18 eas 3 2.7092 5.9121 0.1318 17 20 NA 1 NA 531 5
9 mmas 3 3.4904 4.9746 0.5959 13 33 NA 0 NA NA 5

# 2026-01-31 10:06:25 GMT: Iteration 2 of 5

```

Figure 6: Sample text output of **irace**.

9.2 R data file (logfile)

The R data file created by **irace** (by default as `irace.Rdata`, see option `logFile`) contains an object called `iraceResults`. You can load this file in the R console with:

```
logfile <- system.file(package="irace", "exdata", "irace-acotsp.Rdata", mustWork=TRUE)
iraceResults <- read_logfile(logfile)
```

The `iraceResults` object is a list, and the elements of a list can be accessed in R by using the `$` or `[[ ]]` operators:

```
> iraceResults$irace_version
[1] "4.4.0.d5a98af"
```

The `iraceResults` list contains the following elements:

- **scenario**: The scenario R object containing the **irace** options used for the execution. See [Section 11](#) and the help of the **irace** package; open an R console and type: `?defaultScenario`. See [Section 11](#) for more information.
- **parameters**: The parameters R object containing the description of the target algorithm parameters. See [Section 5.1](#).
- **allConfigurations**: The target algorithm configurations generated by **irace**. This object is a **data frame**, each row is a candidate configuration; the first column (`.ID.`) indicates the internal identifier of the configuration; the final column (`.PARENT.`) is the identifier of the configuration from which the current configuration was sampled; and the remaining columns correspond to the parameter values; each column is named as the parameter name specified in the parameter object.

```
> head(iraceResults$allConfigurations)
```

	.ID.	algorithm	localsearch	alpha	beta	rho	ants	nnls	q0
1	1	as	0	1.0000	1.0000	0.9500	10	NA	NA
2	2	mmas	2	0.3654	6.2246	0.9672	19	50	NA
3	3	ras	0	2.8654	1.2246	0.4722	86	NA	NA
4	4	acs	3	1.6154	8.7246	0.2247	41	39	0.7617
5	5	eas	1	4.1154	3.7246	0.7197	9	16	NA
6	6	as	0	2.2404	7.4746	0.8434	6	NA	NA

	dlb	rasrank	elitistants	time	.PARENT.
1	<NA>	NA	NA	5	NA
2	1	NA	NA	5	NA
3	<NA>	42	NA	5	NA
4	0	NA	NA	5	NA
5	1	NA	602	5	NA
6	<NA>	NA	NA	5	NA

- **allElites**: A list that contains one element per iteration. Each element contains the internal identifier of the elite candidate configurations of the corresponding iteration (identifiers correspond to `allConfigurations$.ID.`).

```
> print(iraceResults$allElites)
```

```
[[1]]
[1] 31 18 9

[[2]]
[1] 31 18 9 43 49

[[3]]
[1] 88 43 49 31 69

[[4]]
[1] 104 110 95 96 111

[[5]]
[1] 121 119 139 125 141

[[6]]
[1] 119 121 139 125 141

[[7]]
[1] 119 121 139 125 141
```

The configurations are ordered by mean performance, that is, the ID of the best configuration corresponds to the first ID. To obtain the values of the parameters of all elite configurations found by **irace** use:

```
> logfile <- system.file(package="irace", "exdata", "irace-acotsp.Rdata", mustWork=TRUE)
> getFinalElites(logfile, n = 0)
```

	.ID.	algorithm	localsearch	alpha	beta	rho	ants	nnls	q0
119	119	acs	3	3.4033	3.8517	0.4882	7	11	0.7569
121	121	acs	3	4.8954	2.4081	0.0516	9	16	0.2240
125	125	acs	3	4.4088	1.6574	0.4104	7	11	0.6845
139	139	acs	3	3.7213	0.3944	0.8664	10	14	0.8420
141	141	acs	3	4.2378	2.1780	0.5210	8	12	0.6151
	dlb	rasrank	elitist	ants	time	.PARENT.			
119	1	NA	NA	5	110				
121	1	NA	NA	5	110				
125	1	NA	NA	5	110				
139	1	NA	NA	5	111				
141	1	NA	NA	5	110				

- **iterationElites**: A vector containing the best candidate configuration ID of each iteration. The best configuration found corresponds to the last one of this vector.

```
> print(iraceResults$iterationElites)
```

```
[1] 31 31 88 104 121 119 119
```

One can obtain the full configuration with:

```
> last <- length(iraceResults$iterationElites)
> id <- iraceResults$iterationElites[last]
> getConfigurationById(iraceResults, ids = id)
```

```

.ID. algorithm localsearch alpha beta rho ants nnls q0
119 119 acs 3 3.4033 3.8517 0.4882 7 11 0.7569
dlb rasrank elitistants time .PARENT.
119 1 NA NA 5 110
```

- **rejectedConfigurations**: A vector containing the rejected configurations IDs. These correspond to configurations that produced failed executions and were ignored by **irace** during the configuration process. See [Section 10.10](#) to enable the detection of such configurations.
- **experiments**: A matrix with configurations as columns and instances as rows. Column names correspond to the internal identifier of the configuration (**allConfigurations\$.ID.**). The results of a particular configuration can be obtained using:

```

> # As an example, we use the best configuration found
> best_config <- getFinalElites(iraceResults, n = 1)
> best_id <- as.character(best_config$.ID.)
> # Obtain the results of the best configuration
> all_exp <- iraceResults$experiments[, best_id]
> # all_exp is a vector and names(all_exp) is the (instance,seed) index.
> all_exp
```

```

      1      2      3      4      5      6      7
33406092 32551908 33101834 32864047 32896644 32778527 32765156
      8      9     10     11     12     13     14
32766978 32861100 32931457 32703895 32790518 32679546 32773935
     15     16     17
32865365 32737958 32674576
```

```

> # Obtain the results of the first and best configurations
> all_exp <- iraceResults$experiments[, c("1", best_id)]
> # all_exp is a matrix: colnames(all_exp) is configurationID and
> # rownames(all_exp) is the (instance,seed) index.
> all_exp
```

```

      1      119
1 42220233 33406092
2 39533155 32551908
3 41303115 33101834
```

```

4 40251195 32864047
5 39741584 32896644
6      NA 32778527
7      NA 32765156
8      NA 32766978
9      NA 32861100
10     NA 32931457
11     NA 32703895
12     NA 32790518
13     NA 32679546
14     NA 32773935
15     NA 32865365
16     NA 32737958
17     NA 32674576

```

When a configuration was not executed on an instance, its value is NA. A configuration may not be executed on an instance because: (1) it was not created yet when the instance was used, or (2) it was discarded by the statistical test and not executed on subsequent instances, or (3) the race terminated before this instance was considered.

Row names correspond to the row index of the `(instance,seed)` pairs in the data frame returned by `get_instanceID_seed_pairs()`. The instanceID and seed used for a particular experiment can be obtained with:

```

> # As an example, we get instanceID, seeds and instances of the experiments
> # of the best configuration.
> # We could get the indexes of the instances on which at least one
> # configuration was executed:
> pair_index <- which(apply(!is.na(all_exp), 1L, any))
> # or the instances on which all configurations were executed:
> pair_index <- which(apply(!is.na(all_exp), 1L, all))
> # but in this example we get the indexes of the instances executed for
> # the best configuration.
> pair_index <- which(!is.na(all_exp[, best_id]))
> instanceID <- get_instanceID_seed_pairs(iraceResults)[["instanceID"]][pair_index]
> # or get the seeds
> get_instanceID_seed_pairs(iraceResults)[["seed"]][pair_index]

[1] 1498426593 1324006684 156117387 2123556176 975149182 657774990
[7] 1688886839 1722597766 545710096 685987118 654417054 1203404683
[13] 1011189918 2070570017 1394746157 397287940 1165230845

> # or obtain the actual instances.
> iraceResults$scenario$instances[instanceID]

[1] "./instances/2000-918.tsp" "./instances/2000-919.tsp"
[3] "./instances/2000-519.tsp" "./instances/2000-813.tsp"
[5] "./instances/2000-811.tsp" "./instances/2000-512.tsp"
[7] "./instances/2000-715.tsp" "./instances/2000-819.tsp"

```

```

[9] "./instances/2000-712.tsp" "./instances/2000-816.tsp"
[11] "./instances/2000-814.tsp" "./instances/2000-620.tsp"
[13] "./instances/2000-812.tsp" "./instances/2000-717.tsp"
[15] "./instances/2000-611.tsp" "./instances/2000-516.tsp"
[17] "./instances/2000-616.tsp"

> # If the instances are of atomic type (integers, floating-point numbers or
> # character strings), the above is similar to:
> get_instanceID_seed_pairs(iraceResults, index = pair_index, instances=TRUE)

```

	instanceID <int>	seed <int>	instance <char>
1:	48	1498426593	./instances/2000-918.tsp
2:	49	1324006684	./instances/2000-919.tsp
3:	9	156117387	./instances/2000-519.tsp
4:	33	2123556176	./instances/2000-813.tsp
5:	31	975149182	./instances/2000-811.tsp
6:	2	657774990	./instances/2000-512.tsp
7:	25	1688886839	./instances/2000-715.tsp
8:	39	1722597766	./instances/2000-819.tsp
9:	22	545710096	./instances/2000-712.tsp
10:	36	685987118	./instances/2000-816.tsp
11:	34	654417054	./instances/2000-814.tsp
12:	20	1203404683	./instances/2000-620.tsp
13:	32	1011189918	./instances/2000-812.tsp
14:	27	2070570017	./instances/2000-717.tsp
15:	11	1394746157	./instances/2000-611.tsp
16:	6	397287940	./instances/2000-516.tsp
17:	16	1165230845	./instances/2000-616.tsp

- **experimentLog**: A matrix with columns **iteration**, **instance**, **configuration**. This matrix contains the log of all the experiments that **irace** performs during its execution. The **instance** column refers to the index of the data frame returned by **get_instanceID_seed_pairs()**. When **capping** is enabled a column **bound** is added to log the execution bound applied for each execution.
- **softRestart**: A logical vector that indicates if a soft restart was performed on each iteration. If **FALSE**, then no soft restart was performed. See option **softRestart** in [Section 11](#).
- **state**: A list that contains the state of **irace**, the recovery ([Section 8](#)) is done using the information contained in this object. The probabilistic model of the last elite configurations can be found here by doing:

```

> # As an example, we get the model probabilities for the
> # localsearch parameter.
> iraceResults$state$model["localsearch"]

NULL

```

```
> # The order of the probabilities corresponds to:
> iraceResults$scenario$parameters$domains$localsearch

[1] "0" "1" "2" "3"
```

The example shows a list that has one element per elite configuration (ID as element name). In this case, `localsearch` is a categorical parameter and it has a probability for each of its values.

- **testing:** A list that contains the testing results. The list contains the following elements:
 - **experiments:** Matrix of experiments in the same format as the `iraceResults$experiments` matrix. The column names indicate the candidate configuration identifier and the row names contain the name of the instances.

```
> # Get the results of the testing
> iraceResults$testing$experiments
```

	31	18	9	43	49	88	69
1t	32944784	33033100	33013371	32953399	32883380	32850397	32836413
2t	32623084	32778072	32794527	32618490	32616646	32691826	32620840
3t	33072782	33224743	33170187	32993969	33204526	33067747	33086766
4t	32672463	32944359	32982527	32862761	32914280	32813471	32774101
5t	32757113	32981971	32925115	32934641	32735675	32734899	32681930
6t	32631995	32915030	32826985	32659324	32690104	32605288	32612956
7t	32888179	33018877	33207141	32955898	32944549	32833837	32881356
8t	32873172	33065148	33075788	32912558	32885759	32954516	32893621
9t	32859003	32973628	33045932	32866447	33015793	32854516	32890008
10t	32746081	32809692	32787652	32795423	32626702	32600823	32623791
11t	32840529	32914141	33052005	32798435	32950640	32771228	32821320
12t	32473949	32750137	32776640	32532956	32538393	32526395	32566066
13t	32701713	32894063	32847192	32586664	32661430	32602247	32688903
14t	32741096	32821610	32790802	32694109	32629882	32490877	32561046
15t	32733327	32751283	32853776	32723787	32676051	32721583	32737507
16t	32906410	32984824	33047227	32972849	32835015	32837127	32924369
17t	33070622	33312383	33217866	33137165	33314440	33139859	33283712
18t	32516965	32595370	32610546	32541238	32620628	32574108	32559163
19t	32878686	33081250	33065079	33038391	32933055	32822238	32821817
20t	32632529	32827631	32710147	32767808	32691648	32756919	32634263
21t	32624329	32822815	32743914	32619144	32704497	32724659	32725717
22t	32777635	33032401	33028477	32870563	32952644	32758827	32737713
23t	32804089	32918477	32960861	32869443	32835720	32819481	32725978
24t	32745700	33075227	33042310	33018503	32785227	32857360	32876840
25t	32881834	33159015	33122645	32978013	32875967	32928238	32836964
26t	32980398	33051949	33220250	33004945	32999066	32929189	32979752
27t	32894079	32902528	32927781	32781225	32877412	32777994	32775437
28t	32773715	32877911	33012915	32799276	32712863	32817077	32691271
29t	33112621	33072562	33321607	33145214	33199569	32992175	33229556
30t	32750884	32874388	32785365	32881762	32627973	32764928	32760482
31t	33072169	33291079	33127452	33103101	33239656	32967407	33033713

32t	32744331	33048486	33035552	32895287	32984933	32890952	32805144
33t	32843222	32994813	33012232	32893855	32760142	32716120	32775835
34t	32542297	32934316	32817424	32803439	32716474	32599757	32741909
35t	32480265	32464958	32665891	32482302	32385408	32479373	32332709
36t	32542033	32721784	32548231	32485745	32593434	32560750	32534077
37t	32469979	32569734	32509174	32386739	32496212	32378501	32419146
38t	32617697	32794853	32939955	32778033	32719417	32855909	32632275
39t	32812910	33034531	32958733	32828209	32991367	32897088	32766338
40t	32386218	32402626	32472340	32416815	32361486	32279197	32408091
41t	32958559	33144787	33178444	32991417	32971859	33003562	32973532
42t	32977982	33045622	33027331	32927284	32792039	33022622	32910192
43t	32534343	32613470	32650868	32488704	32565168	32497804	32486155
44t	32632249	32887877	32809629	32714386	32702351	32704652	32803080
45t	32759089	32893564	32853351	32832488	32854953	32896084	32738493
46t	32658893	32805729	32940595	32688088	32759992	32709126	32697886
47t	32545768	32794207	32713342	32518210	32573298	32490196	32508126
48t	32475415	32705083	32693623	32526705	32548006	32510680	32535880
49t	33147711	33329072	33193178	33159756	33040712	33067038	33040020
50t	32394870	32688872	32547268	32443139	32473830	32368165	32465433
	104	110	95	96	111	121	119
1t	32790120	32834446	32804660	32839982	32858724	32866127	32734813
2t	32638387	32685541	32686548	32541240	32733603	32518245	32579425
3t	33143049	33015324	33018882	33181051	33098703	33114938	32989736
4t	32749451	32655721	32718760	32671676	32653936	32640947	32548128
5t	32798157	32720901	32779370	32860481	32825773	32701819	32621897
6t	32715870	32643563	32630839	32641541	32726137	32524943	32564336
7t	32980022	32942793	32897922	32898298	32907433	32889044	32831056
8t	32803108	32989210	32929281	32817124	32906884	32915516	32800993
9t	32825947	32924809	32852580	32900476	32796830	32737106	32816461
10t	32653232	32642155	32736098	32611916	32641298	32571196	32619049
11t	32779558	32755504	32892000	32847231	32877716	32777937	33009221
12t	32477508	32609462	32529985	32477137	32691561	32494935	32498288
13t	32487805	32649534	32617216	32574487	32656708	32590297	32602311
14t	32560134	32508649	32684249	32585635	32538501	32563785	32596737
15t	32732636	32701836	32783142	32754644	32821774	32752315	32732120
16t	32938182	32878081	32908345	32991591	32919861	32921417	32956191
17t	33137481	33138926	33194778	33144370	33226974	33114332	33143434
18t	32500511	32523816	32528386	32534071	32733066	32452661	32448494
19t	32926118	32945036	32866474	33066475	32955041	33001894	33014175
20t	32783033	32691942	32723033	32689457	32789478	32654302	32636514
21t	32620310	32686833	32709316	32601060	32799222	32634151	32650868
22t	32790637	32782065	32902874	32791809	32766312	32902500	32867937
23t	32718838	32629209	32767535	32836297	32735950	32932959	32662074
24t	32830291	32843535	33010142	32795347	32775106	32823284	32830178
25t	32840366	32829798	32928068	32970364	33079408	32978464	32952329
26t	32894263	33123057	32994790	32967446	33024724	32871465	33024672
27t	32739131	32683353	32752826	32752046	32777665	32751858	32710042
28t	32643308	32669369	32754245	32738079	32854458	32737872	32709174

29t	32952042	33072301	33089460	32954459	33137054	33012770	32928449
30t	32808097	32714995	32766631	32702662	32708095	32731366	32616750
31t	32969544	32975359	33113200	33105122	32975899	32921812	32932449
32t	32857638	32772980	32839378	32880910	32854917	32766457	32740835
33t	32884827	32847938	32732134	32724203	32797648	32841310	32728223
34t	32679917	32719534	32662479	32773825	32624663	32613044	32710122
35t	32331265	32400690	32430861	32389320	32481763	32340785	32326757
36t	32522528	32587133	32683073	32558096	32604279	32434859	32423257
37t	32289969	32348078	32291798	32328439	32460608	32380057	32346091
38t	32704806	32736340	32761532	32575554	32712979	32684904	32673467
39t	32931925	32872676	32730662	32829591	32835908	32796824	32739997
40t	32384774	32455874	32347746	32392590	32357899	32344571	32270459
41t	33059992	33062655	33058925	32943460	33081654	32970076	33008099
42t	33004205	32979669	32940961	33032349	32926611	32878984	32835607
43t	32493201	32445247	32507701	32444630	32450534	32376671	32428632
44t	32683988	32679739	32660842	32617750	32557059	32600546	32535982
45t	32752252	32912641	32760452	32782045	32602350	32725124	32630889
46t	32769680	32745893	32668300	32727266	32752889	32706814	32731733
47t	32481796	32609169	32576768	32459189	32494208	32563151	32475769
48t	32492824	32570289	32505750	32516142	32492597	32571647	32611805
49t	33128389	33136469	33082682	33174149	33237382	33063466	33058367
50t	32449225	32415846	32415526	32486407	32463392	32482025	32458084
	139	125	141				
1t	32846762	32752449	32729909				
2t	32555018	32589604	32553182				
3t	33064034	33112268	33115538				
4t	32772470	32670794	32654133				
5t	32733937	32745013	32895971				
6t	32628333	32589684	32629951				
7t	32781607	32826901	32902259				
8t	32770986	32894312	32738745				
9t	32755820	32920501	32778652				
10t	32657537	32679328	32650496				
11t	32763253	32780423	32855208				
12t	32534710	32620939	32551901				
13t	32583001	32556715	32601501				
14t	32729339	32574099	32626216				
15t	32834170	32850060	32792192				
16t	33060393	32861925	32969836				
17t	33170490	33164812	33069451				
18t	32491865	32431181	32509379				
19t	32966683	32833980	32863173				
20t	32690023	32695067	32602470				
21t	32683102	32710962	32778028				
22t	32718134	32873586	32848113				
23t	32820233	32768740	32745549				
24t	32847547	32738408	32986997				
25t	32848293	32929403	32910060				

```

26t 32990226 32943360 33043323
27t 32715615 32751083 32705106
28t 32872473 32789211 32876553
29t 32995389 33070447 32962657
30t 32717508 32706322 32756472
31t 32978190 32924854 32933044
32t 32815825 32849763 32825178
33t 32695351 32791363 32769161
34t 32598104 32670728 32520301
35t 32310648 32474803 32369318
36t 32349429 32554556 32457286
37t 32305824 32394922 32438578
38t 32626672 32658500 32710736
39t 32751224 32778882 32750117
40t 32277339 32293883 32234876
41t 32958408 33009392 32897575
42t 32813560 32859796 32912730
43t 32392053 32506237 32370579
44t 32611561 32511406 32594561
45t 32617455 32764935 32620028
46t 32652358 32705037 32619599
47t 32497579 32553437 32456268
48t 32620336 32643271 32507985
49t 33096371 33090146 33166183
50t 32364341 32432179 32515705

```

- **seeds**: The seeds used for the experiments, each seed corresponds to each instance in the rows of the test **experiments** matrix.

```

> # Get the seeds used for testing
> iraceResults$testing$seeds

```

	1t	2t	3t	4t	5t	6t
310098509	1191891037	1992920706	1556635314	660937021	627123935	
	7t	8t	9t	10t	11t	12t
421925618	1440918674	415630090	786655150	984347817	211089707	
	13t	14t	15t	16t	17t	18t
2043267046	911388040	1946490076	1569928577	1187926914	467003972	
	19t	20t	21t	22t	23t	24t
1143235327	2139328848	1336400162	197579087	1622161044	1015660437	
	25t	26t	27t	28t	29t	30t
1916222549	1933809701	1780651132	527334133	879873640	2067737294	
	31t	32t	33t	34t	35t	36t
1608280244	393822372	932726103	1590808222	1982372176	327392830	
	37t	38t	39t	40t	41t	42t
1143693767	120344916	1498426593	1324006684	156117387	2123556176	
	43t	44t	45t	46t	47t	48t
975149182	657774990	1688886839	1722597766	545710096	685987118	
	49t	50t				
654417054	1203404683					

In the example, instance 1000-1.tsp is executed with seed 310098509.

9.3 Analysis of results

The final configurations returned by **irace** are the elites of the final race. They are reported in decreasing order of performance, that is, the best configuration is reported first.

If testing is performed, you can further analyze the resulting best configurations by performing statistical tests in R:

```
> results <- iraceResults$testing$experiments
> # Wilcoxon paired test
> conf <- gl(ncol(results), # number of configurations
+           nrow(results), # number of instances
+           labels = colnames(results))
> pairwise.wilcox.test (as.vector(results), conf, paired = TRUE, p.adj = "bonf")
```

Pairwise comparisons using Wilcoxon signed rank test with continuity correction

data: as.vector(results) and conf

	31	18	9	43	49	88	69	104
18	1.6e-07	-	-	-	-	-	-	-
9	1.1e-07	1.00000	-	-	-	-	-	-
43	0.12640	5.3e-07	6.7e-07	-	-	-	-	-
49	1.00000	1.4e-06	3.9e-06	1.00000	-	-	-	-
88	1.00000	1.9e-07	2.6e-07	0.61728	0.99051	-	-	-
69	1.00000	4.0e-07	1.6e-07	0.33135	0.27309	1.00000	-	-
104	1.00000	1.1e-07	1.7e-07	0.01660	0.49845	1.00000	1.00000	-
110	1.00000	2.9e-07	1.9e-07	1.00000	1.00000	1.00000	1.00000	1.00000
95	1.00000	1.4e-07	3.5e-07	1.00000	1.00000	1.00000	1.00000	1.00000
96	1.00000	1.3e-07	1.9e-07	0.04317	0.53008	1.00000	1.00000	1.00000
111	1.00000	2.3e-06	1.9e-06	1.00000	1.00000	1.00000	1.00000	1.00000
121	1.00000	1.3e-07	1.1e-07	0.00037	0.04007	1.00000	1.00000	1.00000
119	0.63620	1.6e-07	1.1e-07	0.00014	0.00282	0.67564	0.17181	1.00000
139	1.00000	1.7e-07	1.1e-07	0.00037	0.01726	1.00000	1.00000	1.00000
125	1.00000	1.7e-07	1.2e-07	0.03860	0.09907	1.00000	1.00000	1.00000
141	1.00000	1.4e-07	1.4e-07	0.00432	0.33135	1.00000	1.00000	1.00000
	110	95	96	111	121	119	139	125
18	-	-	-	-	-	-	-	-
9	-	-	-	-	-	-	-	-
43	-	-	-	-	-	-	-	-
49	-	-	-	-	-	-	-	-
88	-	-	-	-	-	-	-	-
69	-	-	-	-	-	-	-	-
104	-	-	-	-	-	-	-	-
110	-	-	-	-	-	-	-	-
95	1.00000	-	-	-	-	-	-	-
96	1.00000	1.00000	-	-	-	-	-	-
111	1.00000	1.00000	1.00000	-	-	-	-	-

```

121 1.00000 0.29137 1.00000 0.09907 - - -
119 0.05793 0.00533 0.22436 0.00247 1.00000 - -
139 1.00000 0.12210 1.00000 0.03581 1.00000 1.00000 -
125 1.00000 1.00000 1.00000 0.16058 1.00000 0.44028 1.00000 -
141 1.00000 0.58095 1.00000 0.08913 1.00000 1.00000 1.00000 1.00000

```

P value adjustment method: bonferroni

The Kendall concordance coefficient (W) and the Spearman's rho can be applied over data that has the characteristics of the data obtained in the testing, that is a full matrix where all configurations are executed in all instances. W can show if the configurations tested have an homogeneous performance on the used instances set. If evidence of an heterogeneous scenario found we recommend to make some adjustments in the **irace** options as described in [Section 10.7](#).

```

> irace:::concordance(iraceResults$testing$experiments)

$kendall.w
[1] 0.3458941

$spearman.rho
[1] 0.332545

```

It is also possible, as shown in [Fig. 7](#), to plot the performance on the test set of the best-so-far configuration over the number of experiments as follows:

```

# Get summary data from the logfile.
irs <- irace_summarise(iraceResults)
# Get number of iterations
iters <- irs$n_iterations
# Get number of experiments (runs of target-runner) up to each iteration
fes <- cumsum(table(iraceResults$state$experiment_log[["iteration"]]))
# Get the mean value of all experiments executed up to each iteration
# for the best configuration of that iteration.
elites <- as.character(iraceResults$iterationElites)
values <- colMeans(iraceResults$testing$experiments[, elites])
stderr <- function(x) sqrt(var(x)/length(x))
err <- apply(iraceResults$testing$experiments[, elites], 2L, stderr)
plot(fes, values, type = "s",
     xlab = "Number of runs of the target algorithm",
     ylab = "Mean value over testing set", ylim=c(min(values-err), max(values+err)))
points(fes, values, pch=19)
arrows(fes, values - err, fes, values + err, length=0.05, angle=90, code=3)
text(fes, values, elites, pos = 1)

```

The **irace** package also provides an implementation of the ablation method [\[6\]](#). See [Section 10.11](#).

Finally, more advanced visualizations of the behavior of **irace** are provided by the ACVIZ software package [\[4\]](#), which is available at <https://github.com/souzamarcelo/acviz>. See an example in [Fig. 8](#).

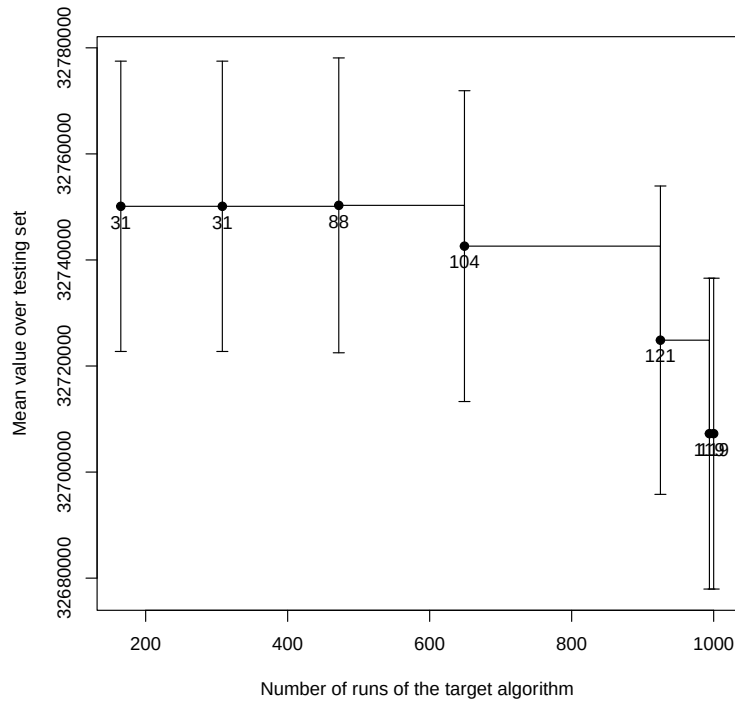


Figure 7: Testing set performance of the best-so-far configuration over number of experiments. Label of each point is the configuration ID.

10 Advanced topics

10.1 Monitoring iterations with a callback

The scenario option `iterationCallback` may be set to an R function that is called after every completed `irace` iteration. For example, the following callback reports the consumed budget and the identifier of the best configuration found in each iteration:

```
report_progress <- function(iteration, elites, progress, ...)
{
  if (progress$maxTime > 0) {
    used <- progress$timeUsed
    total <- progress$maxTime
    budget_type <- "time"
  } else {
    used <- progress$experimentsUsed
    total <- progress$maxExperiments
    budget_type <- "experiments"
  }
  message("Iteration ", iteration, "/", progress$nbIterations,
```

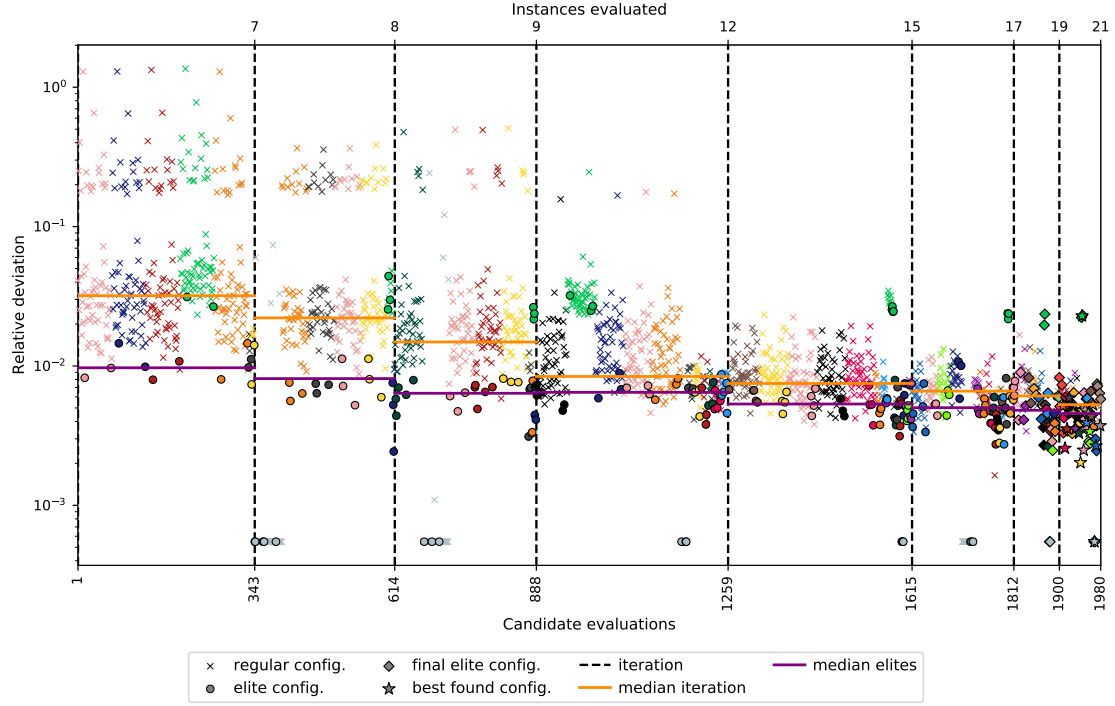


Figure 8: Visualization produced by ACVIZ [4].

```

    ": ", used, "/", total, " ", budget_type,
    " used; best configuration: ", elites[["ID. "]] [1L])
}
scenario$iterationCallback <- report_progress
irace(scenario)

```

The callback is called with the named arguments `iteration`, `elites`, and `progress`. User-defined callbacks should use these exact argument names and take `...` as their last argument so they remain compatible if future versions of **irace** provide more information. The `elites` data frame is ordered from best to worst and is copied before it is passed to the callback.

The `progress` list contains `nbIterations`, `maxExperiments`, `experimentsUsed`, `remainingBudget`, `remainingBudgetEstimated`, `currentBudget`, `currentBudgetUsed`, `maxTime`, `timeUsed`, `remainingTime`, and `boundEstimate`. The budgets `remainingBudget`, `currentBudget`, and `currentBudgetUsed` are numbers of target-algorithm executions. When `maxTime` is used, `remainingBudget` is estimated from `boundEstimate`, and `remainingBudgetEstimated` is `TRUE`. In that mode, `remainingTime` uses the same unit as `maxTime`; otherwise, `remainingTime` and `boundEstimate` are `NA`.

The callback return value is ignored. Errors raised by the callback stop **irace**. Random-number generation within the callback does not affect the random-number sequence of the tuning run.

10.2 Tuning budget

Before setting the budget for a run of **irace**, please consider the number of parameters that need to be tuned, available processing power and available time. The optimal budget depends on the

difficulty of the tuning scenario, the size of the parameter space and the heterogeneity of the instances. Typical values range from 1 000 to 100 000 runs of the target algorithm, although smaller and larger values are also possible. Currently, **irace** does not detect whether the given budget allows generating all possible configurations. In such a case, the use of *iterated* racing is unnecessary: One can simply perform a single race of all configurations (see FAQ in Section 12.11).

Irace provides two options for setting the total tuning budget (**maxExperiments** and **maxTime**). The option **maxExperiments** limits the number of executions of **targetRunner** performed by **irace**. The option **maxTime** limits the total time of the **targetRunner** executions. When this latter option is used, **targetRunner** must return the evaluation cost together with the execution time ("**cost time**").



When the goal is to minimize the computation time of an algorithm, and you wish to use **maxTime** as the tuning budget, **targetRunner** must return the time also as the evaluation cost, that is, return the time twice as "**time time**".



When using **targetEvaluator** and using **maxTime** as tuning budget, **targetRunner** just returns the time ("**time**") and **targetEvaluator** returns the cost.

When using **maxTime**, **irace** estimates the execution time of each **targetRunner** execution before the configuration. The amount of budget used for the estimation is set with the option **budgetEstimation** (default is 2%). The obtained estimation is adjusted after each iteration using the obtained results and it is used to estimate the number of experiments that can be executed. Internally, **irace** uses the number of remaining experiments to adjust the number of configurations tested in each race.

10.3 Multi-objective tuning

Currently, **irace** only optimizes one cost value at a time, which can be solution cost, computation time or any other objective that is returned to **irace** by the **targetRunner**. If the target algorithm is multi-objective, it will typically return not a single cost value, but a set of objective vectors (typically, a Pareto front). For tuning such a target algorithm with **irace**, there are two alternatives. If the algorithm returns a single vector of objective values, they can be aggregated into one single number by using, for example, a weighted sum. Otherwise, if the target algorithm returns a set of objective vectors, a unary quality metric (e.g., the hypervolume) may be used to evaluate the quality of the set.³

The use of aggregation or quality metrics often requires normalizing the different objectives. If normalization bounds are known a priori for each instance, normalized values can be computed by **targetRunner**. Otherwise, the bounds may be dynamically computed while running **irace**, by using **targetEvaluator**. In this case, **targetRunner** will save the output of the algorithm, then the first call to **targetEvaluator** will examine the output produced by all calls to **targetRunner** for the same instance, update the normalization bounds and return the normalized output. Subsequent calls to **targetEvaluator** for the same instance will simply return the normalized output. A similar approach can be used to dynamically compute the reference points or sets often required by unary quality metrics.

For more information about defining a **targetEvaluator**, see Section 5.3. Examples of tuning a multi-objective target algorithm using the hypervolume can be found in the examples at [\\$IRACE_HOME/examples/hypervolume](#) and [\\$IRACE_HOME/examples/moaco](#).

³Multiple metrics are implemented in C, R and Python in the **moocore** package (<https://github.com/multi-objective/moocore>).

10.4 Improving anytime behavior

In some scenarios, we may be unsure when the end-user may terminate the target-algorithm, and we would like to tune the parameters of the algorithm so that it finds a solution as good as possible for any given termination criterion. That is, we would like to obtain a parameter configurations with good *anytime performance*.

Following the approach proposed by López-Ibáñez and Stützle [10], **irace** can tune an algorithm to improve its anytime performance. For that purpose, the algorithm must output a (time, solution cost) pair every time that the best-so-far solution within a run is improved. Such pairs form a matrix with two columns and as many rows as improvements were made. We can define a *reference point*, where the first component is value slightly higher than the maximum runtime of a single run and the second component is a maximum cost value such that differences above that maximum are irrelevant for evaluating performance. Using this reference point and the matrix as an input, one can calculate the hypervolume of the matrix, as described above in Section 10.3. The negative of this hypervolume value becomes the cost returned by the **targetRunner** to **irace**.

It is also possible to specify preferences over the trade-off between runtime and solution cost by using the weighted hypervolume. Further details and examples can be found in López-Ibáñez and Stützle [10].

Although there are other ways to measure anytime performance, most of them are equivalent or an approximation of the measure described above [14].

10.5 Tuning for minimizing computation time

When using **irace** for tuning algorithms that only report computation time to reach a target, **targetRunner** should return the execution time of a configuration instead of solution cost. When using **maxTime** as the budget, this means that **targetRunner** must return twice the execution time since the first value is the minimization objective and the second value is used to track the budget consumed.

Starting from version 3.0, **irace** includes an elitist racing procedure that implements an **adaptive capping mechanism** [16]. Adaptive capping [7] is a configuration technique that avoids the execution of long runs of the target algorithm, focusing the configuration budget in the evaluation of the best configurations found. This is done by bounding the execution time of each configuration based on the best performing candidate configurations.

To use adaptive capping, the **capping** option must be enabled and the **elitist** irace option must be selected. When evaluating candidate configurations on an instance, **irace** calculates an execution bound based on the execution times of the elite configurations. The **boundType** option defines how the performance of the elite configurations is defined to obtain the execution bound. The default value of **boundType** calculates the performance (p_i^s) of each elite configuration (s) as the mean execution time of the instances already executed in the race and the currently executed instance (i). The **cappingType** option specifies the measure used to obtain the elite configurations bound. By default, the execution bound is calculated as the median of the execution times of the elite configurations:

$$b_i = \text{Median}_{\theta_s \in \Theta^{\text{elite}}} \{p_i^s\} \quad (1)$$

The execution bound for new configurations (j) is calculated by multiplying the elite configurations bound by the number of instances (i) in the execution list and subtracting the mean execution time of the instances executed by the candidate:

$$k_i'^j = b_i \cdot i + b^{\min} - p_{i-1}^j \cdot (i - 1) \quad (2)$$

A small constant $b^{\min}$ is added to account for time measurements errors. These settings are also used to apply a dominance elimination criterion together with the statistical test elimination. The domination criterion is defined as:

$$b_i + b^{\min} < p_i^j \quad (3)$$

When elite configurations dominate new configurations, these are eliminated from the race.



The default statistical test when **capping** is enabled is **t-test**. This test is more appropriate to configure algorithms for optimizing runtime (see [Section 10.8](#)).

The execution bound is constantly adjusted by **irace** based on the best configurations times, nevertheless, a maximum execution time ($b^{\max}$) is never exceeded. This maximum execution time must be defined in the configuration scenario when **capping** is enabled. To specify the maximum execution bound for the target runner executions use the **boundMax** option. The final execution bound (k_i^j) is calculated by:

$$k_i^j = \begin{cases} b^{\max} & \text{if } k_i'^j > b^{\max}, \\ \min\{b_i, b^{\max}\} & \text{if } k_i'^j \leq 0, \\ k_i'^j & \text{otherwise;} \end{cases} \quad (4)$$

Additionally, the **boundDigits** option defines the precision of the time bound provided by **irace**, the default setting is 0.

Timed out executions occur when the maximum execution bound (**boundMax**) is reached and the algorithm has not achieved successful termination or a defined quality goal. In this case, it is a common practice to apply a penalty known as PARX, in which timeouts are penalized by multiplying **boundMax** by a constant X . The constant X may be set using the **boundPar** option. Bounded executions are executions that do not achieve successful termination or a defined quality goal in the execution bound (k_i^j) set by **irace**, which is smaller than **boundMax**. The **boundAsTimeout** option replaces the evaluation of bounded executions by the **boundMax** value. More details about the implementation of adaptive capping can be found in Pérez Cáceres et al. [16].



Note that bounded executions are not timed out executions and thus, they will not be penalized by PARX.



Penalized evaluations of timed out and bounded executions are only used for the elimination tests and the comparison between the quality of configurations. To calculate execution bounds and computation budget consumed, **irace** uses only unpenalized execution times. The unpenalized execution time must be provided by the target runner or target evaluator as described in [Section 5.2](#) and [Section 5.3](#).



More advanced capping methods that are applicable to minimizing solution cost are available when combining **irace** with the **capopt** package described by De Souza et al. [5].

10.6 Hyper-parameter optimization of machine learning methods

The **irace** package can also be used for model selection and hyper-parameter optimization of machine learning (ML) methods. We will next explain a possible setup for one given dataset and

using 10-fold cross-validation (CV). Generalizing to multiple datasets and different resampling strategies, e.g. leave-one-out, is straightforward.

First, split the dataset into training, to be used by **irace**, and testing, to be used for evaluating the performance of the configuration returned by **irace**. A typical split could be 70% and 30%, respectively.

The training set is used by **irace** to perform 10-fold CV, that is, the data is split into 10 folds. A single run of the **targetRunner** will use 9 folds for training and the remaining fold for validation. Splitting the data into folds can be done at each call of **targetRunner** or before running **irace**, however, it is important that the split is always the same for every call of the **targetRunner**, i.e., the content of the folds does not change, only which folds are used for training and validation will change.

The setup of **irace** should be as follows:

- **trainInstancesFile**="train-instances.txt", where this file contains one number per line from 1 to 10. This number will tell the **targetRunner** which fold should be used for validation.
- **trainInstancesDir**="", because the folds are the "instances" and you do not have actual instance files. If you want to pass the name of the dataset to the **targetRunner**, you can specify it either at each line of "train-instances.txt", directly in the **targetRunner**, or as a fixed parameter in the **parameterFile**.
- **deterministic**=1 unless it really makes sense to train more than once the same ML model on the same data. If it makes sense, then your **targetRunner** should use the seed passed by **irace** to seed the ML model before training.
- **sampleInstances**=0 because the folds should already be generated by randomly sampling the dataset.
- **testType**="t-test because the performance metrics in ML are typically the mean of the CV results, which assumes that the performance are close to normally distributed.
- **firstTest**=2 because **irace** should discard configurations very aggressively looking for maximum generality.

Finally, your **targetRunner** needs to be able to do the following:

- Receive from **irace** the hyper-parameter settings, the dataset name and a fold number (the "instance"). Let us use fold 3 as an example.
- Train the ML model on the whole training set minus fold 3, then validate (score) the model on fold 3 and return the score to **irace** (negated if the score must be maximized, because **irace** assumes minimization). Since each fold is different, each instance should give a different result. Each row in the table printed by **irace** should print something different; otherwise, something is wrong in your setup.

The above is actually 10 times faster than doing 10-fold CV for each call to **targetRunner**, thus, you should assign to **irace** 10 times the budget than what would be assigned to other methods that do a complete 10-fold CV at each step.

10.7 Heterogeneous scenarios

We classify a scenario as homogeneous when the target algorithm has a consistent performance regarding the instances; roughly speaking, good configurations tend to perform well and bad configurations tend to perform poorly on all instances of the problem. By contrast, in heterogeneous

scenarios, the target algorithm has an inconsistent performance on different instances, that is, some configurations perform well for a subset of the instances, while they perform poorly for a different subset.

When facing a heterogeneous scenario, the first question should be whether the objective of tuning is to find configurations that perform reasonably well over all instances, even if that configuration is not the best ones in any particular instance (a generalist). If this is not the goal, then it would be better to partition instances into more similar subsets and execute **irace** separately on each subset. This will lead to a portfolio of algorithm configurations, one for each subset, and algorithm selection techniques can be used to select the best configuration from the portfolio when facing a new instance.

To make sure **irace** is not misled by results on few instances, it may be useful to increase the number of instances executed before doing a statistical test using the option **firstTest**, e.g., `--first-test 10` (default value is 5), in order to see more instances before discarding configurations. The option **elitistNewInstances** in elitist **irace** (option **elitist**) can be used to increase the number of new instances executed in each iteration, e.g., `--elitist-new-instances 5` (default value is 1).

If finding an overall good configuration for all the instances is the objective, then we recommend that instances are randomly sampled (option **sampleInstances**), unless one can provide the instances in a particular order that does not bias the tuning towards any subset.

If instances are easily categorized in different classes, then we recommend to create “blocks” of instances in **trainInstancesFile**, where each block should contain one instance from each class. Then set the option **blockSize** to the number of classes within each block, so that **irace** will always see a complete block of instances before eliminating configurations. The value of **blockSize** will multiply the values of **firstTest**, **eachTest** and **elitistNewInstances**. Randomly sampling instances (**sampleInstances=1**) will randomly sample the blocks but not break the blocks.

While executing **irace**, the homogeneity of the scenario can be observed by examining the values of Spearman’s rank correlation coefficient and Kendall’s concordance coefficient in the text output of **irace**. See [Section 9.1](#) for more information.

10.8 Choosing the statistical test

The statistical test used in **irace** identifies statistically bad performing configurations that can be discarded from the race in order to save budget. Different statistical tests use different criteria to compare the cost of the configurations, which has an effect on the tuning results.

Irace provides two types of statistical tests (option **testType**). Each test has different characteristics that are beneficial for different goals:

- **Friedman test (F-test)**: This test uses the ranking of the configurations to analyze the differences between their performance. This makes the test suitable for scenarios where the scale of the performance metric is not as important to assess configurations as their relative ranking. This test is also indicated when the distribution of the mean performances deviates greatly from a normal distribution. For example, the ranges of the performance metric on different instances may be completely difference and comparing the performance of different configurations using the mean over multiple instances may be deceiving. We recommend to use the **F-test** (default when **capping** is not enabled) when tuning for solution cost and whenever the best performing algorithm should be among the best in as many instances as possible.
- **Student’s t-test (t-test)**: This test uses the mean performance of the configurations to

analyze the differences between the configurations.⁴ This makes the test suitable for scenarios where the differences between values obtained for different instances are relevant to assess good configurations. We recommend using t-test, in particular, when the target algorithm is minimizing computation time and, in general, whenever the best configurations should obtain the best average solution cost.

The confidence level of the tests may be adjusted by using the option `confidence`. Increasing the value of `confidence` leads to a more strict statistical test. Keep in mind that a stricter test will require more budget to identify which configurations perform worse. A less strict test discards configurations faster by requiring less evidence against them and, therefore, it is more likely to discard good configurations.

10.9 Complex parameter space constraints

Some parameters may have complex dependencies. Ideally, parameters should be defined in the way that is more likely to help the search performed by **irace**. For example, when tuning a branch and bound algorithm, one may have the following parameters:

- branching (**b**) that takes values in $\{0,1,2,3\}$, where 0 indicates no branching will be used and the rest are different types of branching.
- stabilization (**s**) that takes values in $\{0,1,2,3,4,5,6,7,8,9,10\}$, of which for **b**=0 only $\{0,1,2,3,4,5\}$ are relevant.

In this case, it is not possible to describe the parameter space by defining only two parameters for **irace**. An extra parameter must be introduced as follows:

#	name	label	type	range	condition
b		"-b "	c	(0,1,2,3)	
s1		"-s "	c	(0,1,2,3,4,5)	b == "0"
s2		"-s "	c	(0,1,2,3,4,5,6,7,8,9,10)	b != "0"

Parameters whose values depend on the value of other parameters may also require using extra parameters or changing the parameters and processing them in `targetRunner`. For example, given the following parameters:

- Population size (**p**) takes the integer values $[1,100]$.
- Selection size (**s**) takes the same values but no more than the population size, that is $[1,p]$.

In this case, it is possible to describe the parameters **p** and **s** using surrogate parameters for **irace** that represent a ratio of the original interval as follows:

#	name	label	type	range
p		"-p "	i	(1,100)
s_f		"-s "	r	(0.0,1.0)

and `targetRunner` must calculate the actual value of **s** as $\min(\max(\text{round}(\text{s_f} \cdot \text{p}, 1)), 100)$. For example, if the parameter **p** has value 50 and the surrogate parameter **s_f** has value 0.3, then **s** will have value 15.

The processing within `targetRunner` can also split and join parameters. For example, assume the following parameters:

⁴The t-test does not require that the performance values follow a normal distribution, only that the distribution of sample means does. In practice, the t-test is robust despite large deviations from the assumptions.

#	name	label	type	range
m		"-m "	i	(1,250)
e		"-e "	r	(0.0,2.0)

These parameters could be used to define a value $m \cdot 10^e$ for another parameter (`--strength`) not known by **irace**. Then, **targetRunner** takes care of parsing `-m` and `-e`, computing the strength value and passing the parameter `--strength` together with its value to the target algorithm.

More complex parameter space constraints may be implemented by means of the **repairConfiguration** function (Section 5.6).

10.10 Unreliable target algorithms and immediate rejection

There are some situations in which the target algorithm may fail to execute correctly. By default, **irace** stops as soon as a call to **targetRunner** or **targetEvaluator** fails, which helps to detect bugs in the target algorithm. Sometimes the failure cannot be fixed because it is due to system problems, network issues, memory limits, bugs for which no fix is available, or fixing them is impossible because there is no access to the source code.

In those cases, if the failure is caused by random errors or transient system problems, one may wish to ignore the error and try again the same call in the hope that it succeeds. The option **targetRunnerRetries** indicates the number of times a **targetRunner** execution is repeated if it fails. Use this option only if you know additional repetitions could be successful.

If the target algorithm consistently fails for a particular set of configurations, these configurations may be declared as forbidden (Section 5.1.5) so that **irace** avoids them. On the other hand, if the configurations that cause the problem are unknown, the **targetRunner** should return **Inf** so that **irace** immediately rejects the failing configuration. This immediate rejection should be used with care according to the goals of the tuning. For example, a configuration that crashes on a particular instance, e.g., by running out of memory, might still be considered acceptable if it gives very good results on other instances. The configurations which were rejected during the execution of **irace** are saved in the Rdata output file (see Section 9.2).



If the configuration budget is specified in total execution time (**maxTime** option), immediate rejected executions must provide the cost and time (which must be **Inf** 0). Nevertheless, rejected configurations will be excluded from the execution time estimation and the execution bound calculation.

10.11 Ablation Analysis

The ablation method [6] takes two configurations (source and target) and generates a sequence of configurations that differ between each other just in one parameter, where parameter values in source are replaced by values from target. The sequence can be seen as a “path” from the source to the target configuration. This can be used to find new better “intermediate” configurations or to analyse the impact of the parameters in the performance.

To perform ablation, you can use the **ablation()** R function

```
ablog <- ablation("irace.Rdata", src = 1)
plotAblation(ablog)
```

or the **ablation** command-line executable (see more details below).

```
ablation -l irace.Rdata --src 1 --plot plot-ablation.pdf --plot-type rank,boxplot
```

You may specify the IDs of the source and target configurations. By default, the source is taken as the first configuration evaluated by **irace** and the target as the best overall configuration found. Use the function `plotAblation` to visualize the ablation results (Fig. 9).

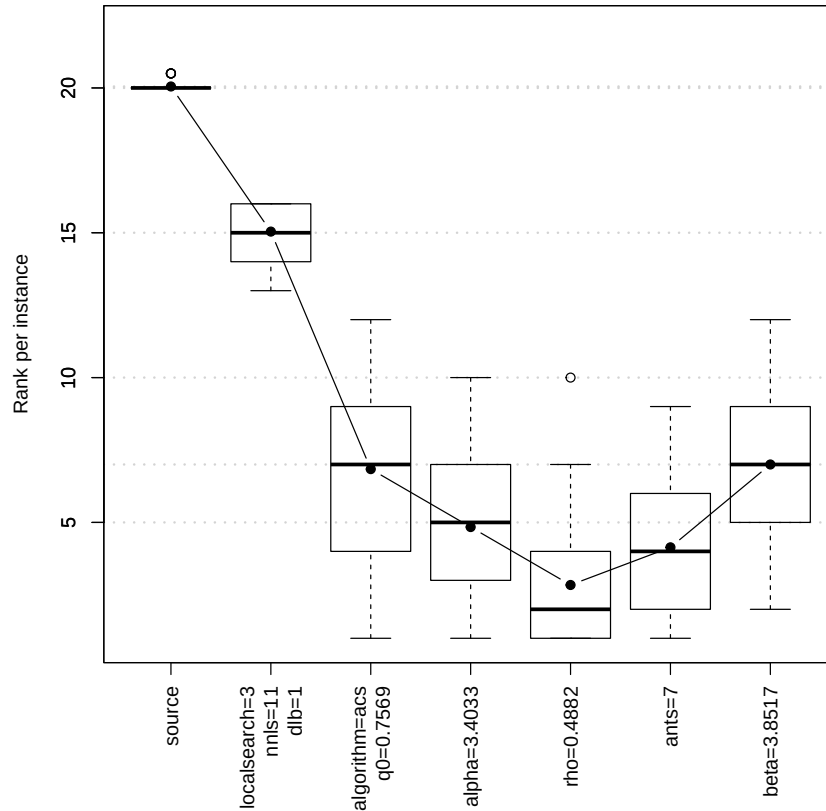


Figure 9: Example of plot generated by `plotAblation()`.

The function returns a list containing the following elements:

configurations: A dataframe of configurations tested during ablation.

instances: The instances used for the ablation.

scenario: Scenario options provided by the user.

trajectory: Best configuration IDs at each step of the ablation.

best: Best overall configuration found.

We also provide a command-line executable (`ablation.exe` in Windows) that allows you to perform ablation without launching R. It is installed in the same location as the **irace** command-line executable and has the following options:

```
#-----
# ablation: An implementation in R of Ablation Analysis
```

```

# Version: unknown
# Copyright (C) 2020--2025
# Manuel Lopez-Ibanez      <manuel.lopez-ibanez@manchester.ac.uk>
# Leslie Perez Caceres    <leslie.perez.caceres@ulb.ac.be>
#
# This is free software, and you are welcome to redistribute it under certain
# conditions. See the GNU General Public License for details. There is NO
# WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
#-----
# installed at: /tmp/Rtmp5lPEEx/Rinst1ecc7aeb481b/irace
# called with: --help
-l,--log-file          Path to the (.Rdata) file created by irace from which
                        the "iraceResults" object will be loaded.
-S,--src               Source configuration ID or the path to a file
                        containing the configuration. Default: 1.
-T,--target            Target configuration ID (by default the best
                        configuration found by irace) or the path to a file
                        containing the configuration.
-P,--params            Specific parameter names to be used for the ablation
                        (separated with commas). By default use all
-t,--type              Type of ablation to perform: "full" will execute each
                        configuration on all "--n-instances" to determine the
                        best-performing one; "racing" will apply racing to
                        find the best configurations. Default: full.
-n,--nrep              Number of replications per instance used in "full"
                        ablation. Default: 1.
    --seed             Integer value to use as seed for the random number
                        generation. Default: 1234567.
-o,--output-file       Log file to save the ablation log. If "", the results
                        are not saved to a file. Default: log-ablation.Rdata.
    --instances-file   Instances file used for ablation: "train", "test" or a
                        filename containing the list of instances. Default:
                        train.
-p,--plot              Output filename (.pdf) for the plot. If not given, no
                        plot is created.
-O,--plot-type         Type of plot. Supported values are "mean", "boxplot",
                        "rank" or "rank,boxplot". Default: mean.
    --old-path         Old path found in the log-file (.Rdata) given as input
                        to be replaced by --new-path.
    --new-path         New path to replace the path found in the log-file
                        (.Rdata) given as input.
-e,--exec-dir          Directory where the target runner will be run.
-s,--scenario           Scenario file to override the scenario given in the
                        log-file (.Rdata)
    --parallel         Number of calls to targetRunner to execute in
                        parallel. Values 0 or 1 mean no parallelization.

```

10.12 Post-selection race

At the end of the configuration process, **irace** will automatically run a post-selection race [18] to try to consume any remaining budget. This post-selection race will not generate new configurations. It will only do additional runs on configurations already evaluated in the current run of **irace**. First, it will try to evaluate the best configurations found in new instances not seen yet. If this is not possible due to the budget available, then it will evaluate configurations that have been evaluated on fewer instances than the best configuration found. Currently, this automatic post-selection is not supported when the budget is measured in runtime (**maxTime**). The automatic post-selection can be disabled using the option **postselection**.

It is also possible to manually run a post-selection race after completing a run of **irace** using the R function **psRace()**.

```
# Execute all elite configurations in the iterations
psRace("irace.Rdata", max_experiments = 0.5, iteration_elites=TRUE)
# Execute a set of configurations IDs providing budget
psRace("irace.Rdata", conf_ids = c(34, 87, 102, 172, 293), max_experiments = 500)
```

The execution of the post-selection race adds an element (**psrace_log**) to the **iraceResults** object saved in the **irace** log file (**logFile**). The post-selection log consists of a list with the following elements:

configurations: Configurations used in the post-selection race.

instances: Data frame with the instances and seeds used in the post-selection race.

max_experiments: Configuration budget assigned to the post-selection race.

experiments: Matrix of results generated by the post-selection race, in the same format as the matrix **iraceResults\$experiments**. Column names are the configuration IDs and row names are the instance IDs.

elites: Best configuration(s) found in the post-selection race.

10.13 Parameter importance analysis using PyImp

The **PyImp**⁵ tool developed by the AutoML group⁶ supports various parameter importance analysis methods using surrogate models. Given a performance dataset of an algorithm configuration scenario, a Random Forest is built to predict performance of all algorithm configurations. Parameter importance analyses are then applied on the prediction model. The model serves as a surrogate for the original target algorithm, so that the algorithm does not need to be executed during the analyses. Three analysis methods are supported, namely fANOVA [9] (functional analysis of variance), forward selection [8], and ablation analysis with surrogates [1]. Note that the **irace** package directly supports ablation (without surrogate models) analysis with and without racing (Section 10.11). Although ablation analysis without surrogates may be more time-consuming, results of the surrogate version may be less accurate than the non-surrogate one.

The **.Rdata** dataset generated by **irace** can be used as input for **PyImp**. The package **irace2pyimp**⁷ is able to convert an **irace.Rdata** file into the input format required by **PyImp**. The conversion can be accessed either through the R console (function **irace2pyimp**),

⁵<https://github.com/automl/ParameterImportance>

⁶<https://www.automl.org/>

⁷<https://github.com/ndangtt>

or via command line executable `irace2pyimp` (or `irace2pyimp.exe` in Windows) provided by the package. You can find the location of the executable by running this command in R: `system.file(file.path("bin", "irace2pyimp"), package="irace2pyimp", mustWork=TRUE)` or `system.file(file.path("bin", "irace2pyimp.exe"), package="irace2pyimp", mustWork=TRUE)` in Windows.

To see the usage of the executable, please run: `irace2pyimp --help`. For more information on the R function `irace2pyimp`, type in the R console: `?irace2pyimp`.

Given as input an `irace.Rdata` file, the script will generate the following output files:

- `params.pcs`: a text file containing the parameter space definition.
- `runhistory.json`: a JSON file containing the list of algorithm configurations evaluated during the tuning and the performance data obtained.
- `traj_aclib2.json`: a JSON file containing the best configurations after each iteration of `irace`. The last configuration will be used as the target configuration in ablation analysis.
- `scenario.txt`: a text file containing the definition of the tuning scenario.
- `instances.txt`: a text file containing the list of instances.
- `features.csv`: a .csv file containing instance features. If no instance features are provided, the index of each instance will be used as a feature.

PyImp can then be called using the files listed above as input. Several examples on how to use the script and call **PyImp** can be found at `system.file("/examples/", package="irace2pyimp")`.

11 List of command-line and scenario options

Most **irace** options can be specified in the command line using a flag or in the **irace** scenario file using the option name (or setting their value in the `scenario` list passed to the various R functions exported by the package). This section describes the various **irace** options that can be specified by the user in this way.



Relative filesystem paths (e.g., `../scenario/`) given in the command-line are relative to the current working directory (the directory at which **irace** is invoked). However, paths given in the scenario file are relative to the directory containing the scenario file. See also Table 1.

11.1 General options

`--help` *flag: -h or --help default:*

Show the list of command-line options of **irace**.

`--version` *flag: -v or --version default:*

Show the version of **irace**.

`--check` *flag: -c or --check default:*

Check that the scenario and parameter definitions are correct and test the execution of the target algorithm. See Section 4.

`--init` *flag: -i or --init default:*

Initialize the working directory with the template config files. This copies the files in `$IRACE_HOME/templates` to the working directory without overwriting the files with the same names as those of the template files.

scenarioFile *flag: -s or --scenario default: ./scenario.txt*
File that contains the scenario setup and other irace options. All options listed in this section can be included in this file. See `$IRACE_HOME/templates/` for an example. Relative file-system paths specified in the scenario file are relative to the scenario file itself.

execDir *flag: --exec-dir default: ./*
Directory where the target runner will be run. The default execution directory is the current directory.



The execution directory must exist before executing **irace**, it will not be created automatically.

logFile *flag: -l or --log-file default: ./irace.Rdata*
File to save tuning results as an R dataset. The provided path must be either an absolute path or relative to **execDir**. See Section 9.2 for details on the format of the R dataset.

quiet *flag: -q or --quiet default: 0*
Reduce the output generated by irace to a minimum.

debugLevel *flag: --debug-level default: 0*
Level of information to display in the text output of **irace**. A value of 0 silences all debug messages. Higher values (1, 2 or 3) provide more verbose debug messages. Details about the text output of **irace** are given in Section 9.1.

seed *flag: --seed default:*
Seed to initialize the random number generator. The seed must be a positive integer. If the seed is "" or NULL, a random seed will be generated.

repairConfiguration *default:*
User-defined R function that takes a configuration generated by **irace** and repairs it. See Section 5.6 for details.

iterationCallback *default:*
Optional R function called after each **irace** iteration with the iteration number, ordered elite configurations, and progress information. See Section 10.1 for details.

postselection *flag: --postselection default: 1*
Perform a postselection race after the execution of **irace** to consume all remaining budget (requires elitist racing). Value 0 disables the postselection race. See Section 10.12.

acLib *flag: --acLib default: 0*
Enable/disable AClib mode. This option enables compatibility with **GenericWrapper4AC** (<https://github.com/automl/GenericWrapper4AC/>) as **targetRunner** script.

11.2 Elitist irace

elitist *flag: -e or --elitist default: 1*
Enable/disable elitist **irace**.

In the **elitist** version of **irace** [12], elite configurations are not discarded from the race until non-elite configurations have been executed on the same instances as the elite configurations. Each race begins by evaluating all configurations on a number of new instances. This number is defined by the option **elitistNewInstances**. After the new instances have

been evaluated, configurations are evaluated on instances seen in the previous race. Elite configurations already have results for most of these previous instances and, therefore, do not need to be re-evaluated. Finally, after configurations have been evaluated on all these instances, the race continues by evaluating additional new instances.

The statistical tests can be performed at any moment during the race according to the setting of the options `firstTest` and `eachTest`. The elitist rule forbids discarding elite configurations, even if the show poor performance, until the last of the previous instances is seen in the race.

The **non-elitist** version of **irace** can discard elite configurations at any point of the race, instances are not re-used from one race to the next, and new instances are sampled for each race.

`elitistNewInstances` *flag: --elitist-new-instances* *default: 1*
 Number of new instances added to each race before evaluating instances from previous races (only for elitist **irace**).



If `deterministic` is `TRUE` then the number of `elitistNewInstances` will be reduced or set to 0 once all instances have been evaluated.

`elitistLimit` *flag: --elitist-limit* *default: 2*
 Maximum number of statistical tests performed without successful elimination after all instances from the previous race have been evaluated. If the limit is reached, the current race is stopped. Only valid for elitist **irace**. Use 0 to disable the limit.

11.3 Internal irace options

`sampleInstances` *flag: --sample-instances* *default: 1*
 Enable/disable the sampling of the training instances. If the option `sampleInstances` is disabled, the instances are used in the order provided in the `trainInstancesFile` or in the order they are read from the `trainInstancesDir` when `trainInstancesFile` is not provided. For more information about training instances see [Section 5.4](#).

`softRestart` *flag: --soft-restart* *default: 1*
 Enable/disable the soft-restart strategy that avoids premature convergence of the probabilistic model. When a sampled configuration is *similar* to its parent configuration, the probabilistic model of these configurations is soft restarted. The soft-restart mechanism is explained in the **irace** paper [12]. The similarity of categorical and ordinal parameters is given by the hamming distance, and the option `softRestartThreshold` defines the similarity of numerical parameters.

`softRestartThreshold` *flag: --soft-restart-threshold* *default: 1e-04*
 Soft restart threshold value for numerical parameters.

`nbIterations` *flag: --iterations* *default: 0*
 Maximum number of iterations to be executed. Each iteration involves the generation of new configurations and the use of racing to select the best configurations. By default (with 0), **irace** calculates a *minimum* number of iterations as $N^{\text{iter}} = \lfloor 2 + \log_2 N^{\text{param}} \rfloor$, where N^{param} is the number of non-fixed parameters to be tuned. Setting this parameter may make **irace** stop sooner than it should without using all the available budget. We recommend to use the default value.

nbExperimentsPerIteration *flag: --experiments-per-iteration default: 0*
 Number of runs of the target algorithm per iteration. By default (when equal to 0), this value changes for each iteration and depends on the iteration index and the remaining budget. Further details are provided in the **irace** paper [12]. We recommend to use the default value.

minNbSurvival *flag: --min-survival default: 0*
 Minimum number of configurations needed to continue the execution of each race (iteration). If the number of configurations alive in the race is not larger than this value, the current iteration will stop and a new iteration will start, even if there is budget left to continue the current race. By default (when equal to 0), the value is calculated automatically as $\lfloor 2 + \log_2 N^{\text{param}} \rfloor$, where N^{param} is the number of non-fixed parameters to be tuned.

nbConfigurations *flag: --num-configurations default: 0*
 The number of configurations that will be raced at each iteration. By default (when equal to 0), this value changes for each iteration and depends on **nbExperimentsPerIteration**, the iteration index and **mu**. The precise details are given in the **irace** paper [12]. We recommend to use the default value.

mu *flag: --mu default: 5*
 Parameter used to define the number of configurations to be sampled and evaluated at each iteration. The number of configurations will be calculated such that there is enough budget in each race to evaluate all configurations on at least $\mu + \min(5, j)$ training instances, where j is the index of the current iteration. The value of μ will be adjusted to never be lower than the value of **firstTest**. We recommend to use the default value and, if needed, adjust **firstTest** and **eachTest**, instead.

11.4 Target algorithm parameters

parameterFile *flag: -p or --parameter-file default: ./parameters.txt*
 File that contains the description of the parameters of the target algorithm. See Section 5.1.

11.5 Target algorithm execution

targetRunner *flag: --target-runner default: ./target-runner*
 Executable or R function that evaluates a configuration of the target algorithm on a particular instance. See Section 5.2 for details.

targetRunnerLauncher *flag: --target-runner-launcher default:*
 Executable that will be used to launch the target runner, when **targetRunner** cannot be executed directly (e.g., a Python script in Windows).

targetCmdline *flag: --target-cmdline default: {configurationID} {instanceID} {seed} {instance} {bound} {targetRunnerArgs}*
 Command-line arguments provided to **targetRunner** (or **targetRunnerLauncher** if defined). The substrings **{configurationID}**, **{instanceID}**, **{seed}**, **{instance}**, and **{bound}** will be replaced by their corresponding values. The substring **{targetRunnerArgs}** will be replaced by the concatenation of the switch and value of all active parameters of the particular configuration being evaluated. The substring **{targetRunner}**, if present, will be replaced by the value of **targetRunner** (useful when using **targetRunnerLauncher**). Example:

```
targetRunner="./real_target_runner.py"
targetRunnerLauncher="python"
targetCmdLine="-m {targetRunner} {configurationID} {instanceID}\
--seed {seed} -i {instance} --cutoff {bound} {targetRunnerArgs}"
```

- targetRunnerRetries** *flag: --target-runner-retries default: 0*
 Number of times to retry a call to **targetRunner** if the call failed.
- targetRunnerTimeout** *flag: --target-runner-timeout default: 0*
 Timeout in seconds of any **targetRunner** call (only applies to **target-runner** executables not to R functions), ignored if 0.
- targetRunnerData** *default:*
 Optional data passed to **targetRunner**. This is ignored by the default **targetRunner** function, but it may be used by custom **targetRunner** functions to pass persistent data around.
- targetRunnerParallel** *default:*
 Optional R function to provide custom parallelization of **targetRunner**. See [Section 6](#) for more information.
- targetEvaluator** *flag: --target-evaluator default:*
 Optional script or R function that returns a numerical value for an experiment after all configurations have been executed on a given instance using **targetRunner**. See [Section 5.3](#) for details.
- deterministic** *flag: --deterministic default: 0*
 Enable/disable deterministic target algorithm mode. If the target algorithm is deterministic, configurations will be evaluated only once per instance. See [Section 5.4](#) for more information.



If the number of instances provided is less than the value specified for the option **firstTest**, no statistical test will be performed.

- parallel** *flag: --parallel default: 0*
 Number of calls of the **targetRunner** to execute in parallel. Values 0 or 1 mean no parallelization. For more information on parallelization, see [Section 6](#).
- loadBalancing** *flag: --load-balancing default: 1*
 Enable/disable load-balancing when executing experiments in parallel. Load-balancing makes better use of computing resources, but increases communication overhead. If this overhead is large, disabling load-balancing may be faster. See [Section 6](#).
- mpi** *flag: --mpi default: 0*
 Enable/disable use of **Rmpi** to execute the **targetRunner** in parallel using MPI protocol. When **mpi** is enabled, the option **parallel** is the number of slave nodes. See [Section 6](#).
- batchmode** *flag: --batchmode default: 0*
 Specify how irace waits for jobs to finish when **targetRunner** submits jobs to a batch cluster: **sge**, **pbs**, **torque**, **slurm** or **htcondor** (**targetRunner** must submit jobs to the cluster using, for example, **qsub**). See [Section 6](#).

11.6 Initial configurations

`configurationsFile` *flag: --configurations-file default:*

File containing a table of initial configurations. If empty or NULL, **irace** will not use initial configurations. See [Section 5.5](#).



The provided configurations must not violate the constraints described in **parameterFile** and **forbiddenFile**.

11.7 Training instances

`trainInstancesDir` *flag: --train-instances-dir default:*

Directory where training instances are located; either absolute path or relative to current directory. See [Section 5.4](#).

`trainInstancesFile` *flag: --train-instances-file default:*

File that contains a list of instances and optionally additional parameters for them. See [Section 5.4](#).



The list of instances in **trainInstancesFile** is interpreted as file-system paths relative to **trainInstancesDir**. When using an absolute path or instances that are not files, set **trainInstancesDir=""**.

`blockSize` *flag: --block-size default: 1*

Number of training instances, that make up a 'block' in **trainInstancesFile**. Elimination of configurations will only be performed after evaluating a complete block and never in the middle of a block. Each block typically contains one instance from each instance class (type or family) and the block size is the number of classes. The value of **blockSize** will multiply **firstTest**, **eachTest** and **elitistNewInstances**.

11.8 Tuning budget

`maxExperiments` *flag: --max-experiments default: 0*

The maximum number of runs (invocations of **targetRunner**) that will be performed. It determines the maximum budget of experiments for the tuning. See [Section 10.2](#).

`minExperiments` *flag: --min-experiments default:*

The minimum number of runs (invocations of **targetRunner**) that will be performed. If this option is set, then **maxExperiments** is ignored and the actual budget will depend on the number of parameters and **minNbSurvival**, but it will not be smaller than this value. See [Section 10.2](#).

`maxTime` *flag: --max-time default: 0*

The maximum total time for the runs of **targetRunner** that will be performed. The mean execution time of each run is estimated in order to calculate the maximum number of experiments (see option **budgetEstimation**). When **maxTime** is positive, then **targetRunner** must return the execution time as its second output. This value and the one returned by **targetRunner** must use the same units (seconds, minutes, iterations, evaluations, ...). See [Section 10.2](#).

budgetEstimation *flag: --budget-estimation default: 0.05*
Fraction (smaller than 1) of the budget used to estimate the mean execution time of a configuration. Only used when **maxTime** > 0. See [Section 10.2](#).

minMeasurableTime *flag: --min-measurable-time default: 0.01*
Minimum time unit that is still (significantly) measurable.

11.9 Statistical test

testType *flag: --test-type default:*
Specifies the statistical test used for elimination:

- F-test** (Friedman test)
- t-test** (pairwise t-tests with no correction)
- t-test-bonferroni** (t-test with Bonferroni's correction for multiple comparisons)
- t-test-holm** (t-test with Holm's correction for multiple comparisons).

We recommend to not use corrections for multiple comparisons because the test typically becomes too strict and the search stagnates. See [Section 10.8](#) for details about choosing the statistical test most appropriate for your scenario.



The default setting of **testType** is **F-test** unless the **capping** option is enabled in which case, the default setting is **t-test**.

firstTest *flag: --first-test default: 5*
Specifies how many instances are evaluated before the first elimination test.



The value of **firstTest** must be a multiple of **eachTest**.

eachTest *flag: --each-test default: 1*
Specifies how many instances are evaluated between elimination tests.

confidence *flag: --confidence default: 0.95*
Confidence level for the elimination test.

11.10 Adaptive capping

capping *flag: --capping default:*
Enable the use of adaptive capping. Capping is enabled by default if **elitist** is active, **maxTime** > 0 and **boundMax** > 0. When using this option, **irace** provides an execution bound to each target algorithm execution (See [Section 5.2](#)). For more details about this option See [Section 10.5](#).

cappingAfterFirstTest *flag: --capping-after-first-test default: 0*
If set to 1, elimination due to capping only happens after **firstTest** instances are seen.

cappingType *flag: --capping-type default: median*
Specifies the measure used to define the execution bound:

- median** (the median of the performance of the elite configurations)

mean (the mean of the performance of the elite configurations)

best (the best performance of the elite configurations)

worst (the worst performance of the elite configurations).

boundType *flag: --bound-type default: candidate*

Specifies how to calculate the performance of elite configurations for the execution bound:

candidate (performance of candidates is aggregated across the instances already executed)

instance (performance of candidates on each instance).

boundMax *flag: --bound-max default: 0*

Maximum execution bound for **targetRunner**. It must be specified when capping is enabled.

boundDigits *flag: --bound-digits default: 0*

Precision used for calculating the execution time. It must be specified when capping is enabled.

boundPar *flag: --bound-par default: 1*

Penalty used for PARX. This value is used to penalize timed out executions by multiplying **boundMax**, see [Section 10.5](#).

boundAsTimeout *flag: --bound-as-timeout default: 1*

Replace the configuration cost of bounded executions with **boundMax**. See [Section 10.5](#).

11.11 Recovery

recoveryFile *flag: --recovery-file default:*

Previously saved **irace** log file that should be used to recover the execution of **irace**; either absolute path or relative to the current directory. If empty or NULL, recovery is not performed. For more details about recovery, see [Section 8](#).

11.12 Testing

--only-test *flag: --only-test default:*

Run the configurations contained in the file provided as argument on the test instances. See [Section 7](#).

testInstancesDir *flag: --test-instances-dir default:*

Directory where testing instances are located, either absolute or relative to current directory.

testInstancesFile *flag: --test-instances-file default:*

File containing a list of test instances and optionally additional parameters for them.

testNbElites *flag: --test-num-elites default: 1*

Number of elite configurations returned by irace that will be tested if test instances are provided. For more information about the testing, see [Section 7](#).

testIterationElites *flag: --test-iteration-elites default: 0*

Enable/disable testing the elite configurations found at each iteration.

12 FAQ (Frequently Asked Questions)

12.1 Is irace minimizing or maximizing the output of my algorithm?

By default, **irace** considers that the value returned by `targetRunner` (or by `targetEvaluator`, if used) should be **minimized**. In case of a maximization problem, one can simply multiply the value by -1 before returning it to irace. This is done, for example, when maximizing the hypervolume (see the last lines in `$IRACE_HOME/examples/hypervolume/target-evaluator`).

12.2 Are experiments with irace reproducible?

Short answer: Yes, under some conditions.

Long answer: According to the terminology described by López-Ibáñez et al. [13], we define *repeatability* as “*exactly repeating the original experiment, generating precisely the same results*”. Following this definition, a run of **irace** is repeatable under the following conditions:

- Same version of **irace**.
- Same version of R (different versions of R may change the behavior of functions used by **irace**).
- The behavior of `targetRunner` is deterministic or exactly reproducible for the same instance, parameter configuration and random seed. Make sure that `targetRunner` uses the seed provided by **irace** to initialize all random number generators used. If the result of `targetRunner` depends on CPU-time, wall-clock time or system load in any way, then `targetRunner` is not reproducible and neither will be **irace**.
- Same random seed (`seed`) given to **irace**.
- Same scenario options (Section 11). Although some options should not affect reproducibility (e.g., `debugLevel`), maintaining a list of such options will be a huge effort, thus the safest assumption is that any change may break reproducibility.
- Same parameter space (Section 5.1), including types, domains, conditions and forbidden configurations. The order of the parameters may also affect reproducibility (the name of the parameters should not) because it affects the order in which random numbers are used.
- Same training instances provided and in the same order (Section 5.4). Even if the instances are sampled randomly (`sampleInstances`), a different initial order will produce a different sample even with the same random seed.
- Same initial configurations (Section 5.5), if any.

12.3 Is it possible to configure a MATLAB algorithm with irace?

Definitely. There are three main ways to achieve this:

1. Edit the `targetRunner` script to call MATLAB in a non-interactive way. See the MATLAB documentation, or the following links.⁸ You would need to pass the parameter received by `targetRunner` to your MATLAB script.⁹ There is a minimal example in

⁸See [suppress-start-message-of-matlab \(stackoverflow.com\)](https://stackoverflow.com/questions/1048442/suppress-start-message-of-matlab) and [how-to-call-matlab-from-command-line-and-print-to-stdout-before-exiting \(stackoverflow.com\)](https://stackoverflow.com/questions/1048442/how-to-call-matlab-from-command-line-and-print-to-stdout-before-exiting)

⁹See [how-can-i-pass-input-parameters-when-running-matlab-in-batch-mode-in-windows \(mathworks.com\)](https://mathworks.com/help/matlab/matlab_external/how-can-i-pass-input-parameters-when-running-matlab-in-batch-mode-in-windows.html) and [how-can-i-pass-command-line-arguments-to-a-standalone-matlab-executable-running \(mathworks.com\)](https://mathworks.com/help/matlab/matlab_external/how-can-i-pass-command-line-arguments-to-a-standalone-matlab-executable-running.html)

`$IRACE_HOME/examples/matlab/`, where you can find an example `target-runner` script calling the `matlab` executable.

2. Call MATLAB code directly from R using the `matlabr` package (<https://cran.r-project.org/package=matlabr>). This is a better option if you are experienced in R. Define `targetRunner` as an R function instead of a path to a script. The function should call your MATLAB code with appropriate parameters. There is a minimal example in `$IRACE_HOME/examples/matlabr/`, where you can find the `targetRunner` function in `scenario.txt` and the MATLAB code in `Main.m`.
3. Another possibility is calling MATLAB directly from a different programming language and write `targetRunner` in that programming language, for example, in Python (see examples in `$IRACE_HOME/examples/target-runner-python/`).¹⁰

12.4 My program works perfectly on its own, but not when running under irace. Is irace broken?

Every time this was reported, it was a difficult-to-reproduce bug, i.e., a [Heisenbug](#), in the program (target algorithm), not in `irace`. To detect such bugs, we recommend that you use, within `targetRunner`, a memory debugger (e.g., [valgrind](#)) to run your program. For example, if your program is executed by `targetRunner` as:

```
${EXE} ${FIXED_PARAMS} -i ${INSTANCE} ${CONFIG_PARAMS} 1> ${STDOUT} 2> ${STDERR}
```

then replace that line with:

```
valgrind --error-exitcode=1 ${EXE} ${FIXED_PARAMS} -i ${INSTANCE} \
  ${CONFIG_PARAMS} 1> ${STDOUT} 2> ${STDERR}
```

If there are bugs in your program, they will appear in `${STDERR}`, thus do not delete those files. Memory debuggers will significantly slowdown your code, so use them only as a means to find what is wrong with your target algorithm. Once you have fixed the bugs, you should remove the use of `valgrind`.

12.5 irace seems to run forever without any progress, is this a bug?

Every time this problem was reported, the issue was in the target algorithm and not in `irace`. Some ideas for debugging this problem:

- Check that the target algorithm is really not running nor paused nor sleeping nor waiting for input-output.
- Use `debugLevel=3` to see how `irace` calls `target-runner`, run the same command outside `irace` and verify that it terminates.
- Add some output to your algorithm that reports at the very end the runtime and exit code. Verify that this output is printed when `irace` calls your algorithm.
- In `target-runner`, print something to a log file *after* calling your target algorithm. Verify that this output appears in the log file when `irace` is running.
- Set a maximum timeout when calling your target algorithm from `target-runner` (see [FAQ 12.6](#)).

¹⁰See [call-matlab-functions-from-python \(mathworks.com\)](#) and [call-user-script-and-function-from-python \(mathworks.com\)](#)

12.6 My program may be buggy and run into an infinite loop. Is it possible to set a maximum timeout?

We are not aware of any way to achieve this using R. However, in GNU/Linux, it is easy to implement by using the `timeout` command¹¹ in `targetRunner` when invoking your program.

12.7 When using the `mpi` option, `irace` stops with an error message indicating that a function is not defined. How to fix this?

Rmpi does not work the same way when called from within a package and when called from a script or interactively. When **irace** creates the slave nodes, the slaves will load a copy of **irace** automatically. If the slave nodes are on different machines, they must have **irace** installed. If **irace** is not installed system-wide, R needs to be able to find **irace** on the slave nodes. This is usually done by setting `R_LIBS`, `.libPaths()` or by loading **irace** using `library()` or `require()` with the argument `"lib.loc"`. The settings on the master are not applied to the slave nodes automatically, thus the slave nodes may need their own settings. After spawning the slaves, it is too late to modify those settings, thus modifying the shell variable `R_LIBS` seems the only valid way to tell the slaves where to find **irace**.

If the path is set correctly and the problem persists, please check these instructions:

1. Test that **irace** and **Rmpi** work. Run **irace** on a single machine (submit node), without calling `qsub`, `mpirun` or a similar wrapper around **irace** or R.
2. Test loading **irace** on the slave nodes. However, jobs submitted by `qsub`/`mpirun` may load R packages using a different mechanism from the way it happens if you log directly into the node (e.g., with `ssh`). Thus, you need to write a little R program such as:

```
library(Rmpi)
mpi.spawn.Rslaves(nslaves = 10)
paths <- mpi.applyLB(1:10, function(x) {
  library(irace); return(path.package("irace")) })
print(paths)
```

Submit this program to the cluster like you would submit **irace** (using `qsub`, `mpirun` or whatever program is used to submit jobs to the cluster).

3. In the script `bin/parallel-irace-mpi`, the function `irace_main()` creates an MPI job for our cluster. You may need to speak with the admin of your cluster and ask them how to best submit a job for MPI. There may be some particular settings that you need. **Rmpi** normally creates log files; but **irace** suppresses those files unless `debugLevel > 0`.

Please contact us (Section 13) if you have further problems.

12.8 Error: 4 arguments passed to `.Internal(nchar)` which requires 3

This is a bug in R 3.2.0 on Windows. The solution is to update your version of R.

¹¹<https://man7.org/linux/man-pages/man1/timeout.1.html>

12.9 Warning: In `read.table(filename, header = TRUE, colClasses = "character", : incomplete final line found by ...`

This is a warning given by R when the last line of an input file does not finish with the newline character. The warning is harmless and can be ignored. If you want to suppress it, just open the file and press the ENTER key at the end of the last line of the file to end the final line with a newline.

12.10 How are relative filesystem paths interpreted by **irace**?

The answer depends on where the path appears. Relative paths may appear as the argument of command-line options, as the value of options given in the scenario file, or within various scripts, functions or instance files. Table 1 summarizes how paths are translated from relative to absolute.

Table 1: Translation of relative to absolute filesystem paths.

Relative path appears as ...	...is relative to ...
a string within <code>trainInstancesFile</code>	<code>trainInstancesDir</code>
a string within <code>testInstancesFile</code>	<code>testInstancesDir</code>
code within <code>targetRunner</code> or <code>targetEvaluator</code>	<code>execDir</code>
the value of <code>logFile</code> or <code>--log-file</code>	<code>execDir</code>
the value of other options in the scenario file	the directory containing the scenario file
the value of other command-line options	invocation (working) directory of irace

12.11 My parameter space is small enough that **irace** could generate all possible configurations; however, **irace** generates repeated configurations and/or does not generate some of them. Is this a bug?

Typically, **irace** is applied to parameter spaces that are much larger than what can be explored within the budget given. Thus, **irace** does not try to detect whether all possible configurations can be evaluated for the given budget and it does not waste computation time to check for repeated configurations. Thus, if the parameter space is actually very small, the initial random sampling performed by **irace** may generate repeated configurations and/or never generate some configurations, which is not ideal. If you still want to use (non-iterated) racing, the recommended approach is to provide all configurations explicitly to **irace** (Section 5.5) and execute a single race (`nbIterations=1`) with exactly the number of configurations provided (e.g., `nbConfigurations=240`). A future version of **irace** may automatically detect this case and switch to non-iterated racing without having to set additional options. Future versions may also implement computationally cheap checks for repeated configurations.¹²

12.12 On Windows and using `target-runner.py` (a Python file), I get the error “`target-runner.py` is not executable”

The issue is that `.py` files are not executable on their own and you need `python.exe` to read the `.py` file and execute it. Linux knows how to do this if the first line of the file is “`#!/usr/bin/python`”, however, Windows doesn’t know how to do it. In Windows you have 2 options:

¹²If you are interested in implementing this, please contact us!

- Create a `target-runner.bat` file that contains a line similar to (see `templates/windows/target-runner.bat`):

```
C:\path\to\python.exe C:\path\to\target-runner.py %instance% %seed% \
%candidate_parameters% 1>%stdout% 2>%stderr%
```

- Or convert `target-runner.py` into an `.exe` file, for example, using `auto-py-to-exe`¹³, so that you do not need a `.bat` file.

12.13 Error in socketConnection("localhost", port = port, server = TRUE, lock = TRUE, : can not open the connection

This error may arise if you activate the `parallel` option of `irace` and your `targetRunner` or `targetEvaluator` tries to setup a parallel cluster or execute code in parallel in a way that interacts badly with the parallel mechanism in R. In this case, you need to either investigate yourself if there is a way for the two parallel mechanisms to co-exist or, if that is not possible, disable parallelism in `irace` or in your code. Note that packages or software used by your `targetRunner` may have a parallel mechanism enabled by default and unknown to you. This is definitely NOT a bug in `irace`.

12.14 irace does not print the call to the targetRunner with debugLevel=2 when using the parallel

This is a limitation of Windows or Rstudio. Running without `parallel` should work. Unfortunately, we cannot fix this limitation in `irace`. If you need to understand how `irace` calls `targetRunner` when running in parallel, you can implement a logging mechanism able to handle parallelism directly inside the `targetRunner`.

12.15 Why does the column Bound in the output sometimes shows NA?

This should only happen when the symbol in the first column is '.', which means that all alive configurations are elite and have already been evaluated on this instance, so nothing is evaluated nor discarded.

13 Resources and contact information

More information about the package can be found on the `irace` webpage:

<https://mlopez-ibanez.github.io/irace/>

For questions and suggestions please contact the development team through the `irace` package Google group:

<https://groups.google.com/d/forum/irace-package>

or by sending an email to:

irace-package@googlegroups.com

¹³<https://pypi.org/project/auto-py-to-exe/>

14 Acknowledgements

We would like to thank all the people that directly or indirectly have collaborated in the development and improvement of **irace**: Prasanna Balaprakash, Zhi (Eric) Yuan, Franco Mascia, Alberto Franzin, Anthony Antoun, Esteban Diaz Leiva, Federico Caselli, Pablo Valledor Pellicer, André de Souza Andrade, and Nguyen Dang (nttd@st-andrews.ac.uk).

Bibliography

- [1] A. Biedenkapp, M. T. Lindauer, K. Eggensperger, F. Hutter, C. Fawcett, and H. H. Hoos. Efficient parameter importance analysis via ablation with surrogates. In S. P. Singh and S. Markovitch, editors, *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Feb. 2017. doi: [10.1609/aaai.v31i1.10657](https://doi.org/10.1609/aaai.v31i1.10657).
- [2] M. Birattari. On the estimation of the expected performance of a metaheuristic on a class of instances. how many instances, how many runs? Technical Report TR/IRIDIA/2004-001, IRIDIA, Université Libre de Bruxelles, Belgium, 2004.
- [3] M. Birattari, Z. Yuan, P. Balaprakash, and T. Stützle. F-race and iterated F-race: An overview. In T. Bartz-Beielstein, M. Chiarandini, L. Paquete, and M. Preuss, editors, *Experimental Methods for the Analysis of Optimization Algorithms*, pages 311–336. Springer, Berlin/Heidelberg, 2010. doi: [10.1007/978-3-642-02538-9_13](https://doi.org/10.1007/978-3-642-02538-9_13).
- [4] M. De Souza, M. Ritt, M. López-Ibáñez, and L. Pérez Cáceres. ACVIZ: A tool for the visual analysis of the configuration of algorithms with irace. *Operations Research Perspectives*, 8: 100186, 2021. doi: [10.1016/j.orp.2021.100186](https://doi.org/10.1016/j.orp.2021.100186).
- [5] M. De Souza, M. Ritt, and M. López-Ibáñez. Capping methods for the automatic configuration of optimization algorithms. *Computers & Operations Research*, 139:105615, 2022. doi: [10.1016/j.cor.2021.105615](https://doi.org/10.1016/j.cor.2021.105615).
- [6] C. Fawcett and H. H. Hoos. Analysing differences between algorithm configurations through ablation. *Journal of Heuristics*, 22(4):431–458, 2016.
- [7] F. Hutter, H. H. Hoos, K. Leyton-Brown, and T. Stützle. ParamILS: an automatic algorithm configuration framework. *Journal of Artificial Intelligence Research*, 36:267–306, Oct. 2009. doi: [10.1613/jair.2861](https://doi.org/10.1613/jair.2861).
- [8] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Identifying key algorithm parameters and instance features using forward selection. In P. M. Pardalos and G. Nicosia, editors, *Learning and Intelligent Optimization, 7th International Conference, LION 7*, volume 7997 of *Lecture Notes in Computer Science*, pages 364–381. Springer, Heidelberg, Germany, 2013. doi: [10.1007/978-3-642-44973-4_40](https://doi.org/10.1007/978-3-642-44973-4_40).
- [9] F. Hutter, H. H. Hoos, and K. Leyton-Brown. An efficient approach for assessing hyperparameter importance. In E. P. Xing and T. Jebara, editors, *Proceedings of the 31st International Conference on Machine Learning, ICML 2014*, volume 32, pages 754–762. PMLR, 2014. URL <https://proceedings.mlr.press/v32/hutter14.html>.
- [10] M. López-Ibáñez and T. Stützle. Automatically improving the anytime behaviour of optimisation algorithms. *European Journal of Operational Research*, 235(3):569–582, 2014. doi: [10.1016/j.ejor.2013.10.043](https://doi.org/10.1016/j.ejor.2013.10.043).

- [11] M. López-Ibáñez, J. Dubois-Lacoste, T. Stützle, and M. Birattari. The irace package, iterated race for automatic algorithm configuration. Technical Report TR/IRIDIA/2011-004, IRIDIA, Université Libre de Bruxelles, Belgium, 2011. URL <http://iridia.ulb.ac.be/IridiaTrSeries/link/IridiaTr2011-004.pdf>. Published in *Operations Research Perspectives* [12].
- [12] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, T. Stützle, and M. Birattari. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016. doi: [10.1016/j.orp.2016.09.002](https://doi.org/10.1016/j.orp.2016.09.002).
- [13] M. López-Ibáñez, J. Branke, and L. Paquete. Reproducibility in evolutionary computation. *ACM Transactions on Evolutionary Learning and Optimization*, 1(4):1–21, 2021. doi: [10.1145/3466624](https://doi.org/10.1145/3466624).
- [14] M. López-Ibáñez, D. Vermetten, J. Dreó, and C. Doerr. Using the empirical attainment function for analyzing single-objective black-box optimization algorithms. *IEEE Transactions on Evolutionary Computation*, 29(5):1774–1782, 2025. doi: [10.1109/TEVC.2024.3462758](https://doi.org/10.1109/TEVC.2024.3462758).
- [15] C. C. McGeoch. Analyzing algorithms by simulation: Variance reduction techniques and simulation speedups. *ACM Computing Surveys*, 24(2):195–212, 1992. doi: [10.1145/130844.130853](https://doi.org/10.1145/130844.130853).
- [16] L. Pérez Cáceres, M. López-Ibáñez, H. H. Hoos, and T. Stützle. An experimental study of adaptive capping in irace. In R. Battiti, D. E. Kvasov, and Y. D. Sergeyev, editors, *Learning and Intelligent Optimization, 11th International Conference, LION 11*, volume 10556 of *Lecture Notes in Computer Science*, pages 235–250. Springer, Cham, Switzerland, 2017. doi: [10.1007/978-3-319-69404-7_17](https://doi.org/10.1007/978-3-319-69404-7_17).
- [17] M. Schneider and H. H. Hoos. Quantifying homogeneity of instance sets for algorithm configuration. In Y. Hamadi and M. Schoenauer, editors, *Learning and Intelligent Optimization, 6th International Conference, LION 6*, volume 7219 of *Lecture Notes in Computer Science*, pages 190–204. Springer, Heidelberg, Germany, 2012. doi: [10.1007/978-3-642-34413-8_14](https://doi.org/10.1007/978-3-642-34413-8_14).
- [18] Z. Yuan, M. A. Montes de Oca, T. Stützle, H. C. Lau, and M. Birattari. An analysis of post-selection in automatic configuration. In C. Blum and E. Alba, editors, *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2013*, pages 1557–1564. ACM Press, New York, NY, 2013. ISBN 978-1-4503-1963-8.

Appendix A Installing R

This section gives a quick R installation guide that will work in most cases. The official instructions are available at <https://cran.r-project.org/doc/manuals/r-release/R-admin.html>

A.1 GNU/Linux

You should install R from your package manager. On a Debian/Ubuntu system it will be something like:

```
sudo apt-get install r-base
```

Once R is installed, you can launch R from the Terminal and from the R prompt install the **irace** package (see [Section 3.2](#)).

A.2 OS X

You can install R directly from a CRAN mirror.¹⁴ Alternatively, if you use homebrew, you can just brew the R formula from the science tap (unfortunately it does not come already bottled so you need to have Xcode¹⁵ installed to compile it):

```
brew tap homebrew/science  
brew install r
```

Once R is installed, you can launch R from the Terminal (or from your Applications), and from the R prompt install the **irace** package (see [Section 3.2](#)).

A.3 Windows

You can install R from a CRAN mirror.¹⁶ We recommend that you install R on a filesystem path without spaces, special characters or long names, such as **C:\R**. Once R is installed, you can launch the R console and install the **irace** package from it (see [Section 3.2](#)).

Appendix B targetRunner troubleshooting checklist

If the **targetRunner** script fails to return the output expected by **irace**, it can be sometimes difficult to diagnose where the problem lies. The more descriptive errors provided by your script, the easier it will be to debug it. If **targetRunner** enters an infinite loop, **irace** will wait indefinitely (see FAQ in [Section 12.6](#)). If you are using temporary files to redirect the output of your algorithm, check that these files are properly created. We recommend to follow the structure of the example file (**target-runner**) provided in **\$IRACE_HOME/templates**. The following error examples are based on that example file.

In case of failure of **targetRunner**, **irace** will print an error on its output describing which execution of **targetRunner** failed. Follow this checklist to detect where the problem is:

1. Make sure that your **targetRunner** script or program is at the specified location. If you see this error:

¹⁴<https://cran.r-project.org/bin/macosx/>

¹⁵Xcode download webpage: <https://developer.apple.com/xcode/download/>

¹⁶<https://cran.r-project.org/bin/windows/>

```
Error: == irace == target runner './tuning/target-runner' does not exist
```

it means that **irace** cannot find the **target-runner** file. Check that the file is at the path specified by the error.

2. Make sure that your **targetRunner** script is an executable file and the user running **irace** has permission to execute it. The following errors:

```
Error: == irace == target runner './tuning/target-runner' is a directory,  
not a file
```

or

```
Error: == irace == target runner './tuning/target-runner' is not executable
```

mean that your **targetRunner** is not an executable file. In the first case, the script is a folder and therefore there must be a problem with the name of the script. In the second case, you must make the file executable, which in GNU/Linux can be done by:

```
chmod +x ./tuning/target-runner
```

3. If your **targetRunner** script calls another program, make sure it is at the location described in the script (variable **EXE** in the examples and templates). A typical output for such an error is:

```
Error: == irace == running command './tuning/target-runner' 1 8 676651103  
./tuning/Instances/1000-16.tsp --ras --localsearch 2 --alpha 4.03 --beta 1.89  
--rho 0.02 --ants 37 --nnls 48 --dlb 0 --rasranks 15 2>\&1' had status 1  
== irace == The call to target_runner_default was:  
./tuning/target-runner 1 8 676651103 ./tuning/Instances/1000-16.tsp --ras  
--localsearch 2 --alpha 4.03 --beta 1.89 --rho 0.02 --ants 37 --nnls 48  
--dlb 0 --rasranks 15  
== irace == The output was:  
Tue May 3 19:00:37 UTC 2016: error: ./bin/acotsp: not found or not executable  
(pwd: ./tuning/acotsp-arena)
```

You may test your script by copying the command line shown in the error and executing **target-runner** directly on the execution directory (**execDir**). In this case, the command line is:

```
./tuning/target-runner 1 8 676651103 ./tuning/Instances/1000-16.tsp --ras \  
--localsearch 2 --alpha 4.03 --beta 1.89 --rho 0.02 --ants 37 --nnls 48 \  
--dlb 0 --rasranks 15
```

This executes the **targetRunner** script as **irace** does. The output of this script must be only one number.

4. If your **targetRunner** file is an executable script in Python, R, Perl, Bash or some other interpreted programming language, make sure that the interpreter specified in the first line of the file exists at the correct location. For example, if the first line of **target-runner.py** is:

```
#!/usr/bin/python
```

Then make sure that **/usr/bin/python** exists and it is executable. Otherwise, you will get an error such as:

Error: == irace == error in running command

5. Check that your `targetRunner` script is actually returning one number as output. For example:

```
Error: == irace == The output of './tuning/target-runner 1 25 365157769
./tuning/Instances/1000-31.tsp --ras --localsearch 1 --alpha 0.26 --beta
6.95 --rho 0.69 --ants 56 --nnls 10 --dlb 0 --rasranks 7' is not numeric!
== irace == The output was:
Solution: 24479793
```

In the example above, the output of `target-runner` is “Solution: 24479793”, which is not a number. If `target-runner` is parsing the output of the target algorithm, you need to verify that the code only parses the solution cost value.

6. Check that your `targetRunner` script is creating the output files for your algorithm. If you see an error as:

```
== irace == The output was: Tue May 3 19:41:40 UTC 2016:
error: c1-9.stdout: No such file or directory
```

The output file of the execution of your algorithm has not been created (check permissions) or has been deleted before the result can be read.

7. Other errors can produce the following output:

```
== irace == The output was: Tue May 3 19:49:06 UTC 2016:
error: c1-23.stdout: Output is not a number
```

This might be because your `targetRunner` script is not executing your algorithm correctly. To further investigate this issue, comment out the line that eliminates the temporary files that saves the output of your algorithm. Similar to this one

```
rm -f "${STDOUT}" "${STDERR}"
```

Execute directly the `targetRunner` command-line that is provided in the error message, look in your execution directory for the files that are created. Check the `.stderr` file for errors and the `.stdout` file to see the output that your algorithm produces.

8. Some command within `targetRunner` may not be working correctly. In that case, you must debug the commands individually exactly as `irace` executes them. In order to find where the problem is, print the commands to a log file before executing them. For example:

```
echo "$EXE ${FIXED_PARAMS} -i $INSTANCE ${CONFIG_PARAMS}" >> ${STDERR}.log
$EXE ${FIXED_PARAMS} -i $INSTANCE ${CONFIG_PARAMS} 1> ${STDOUT} 2> ${STDERR}
```

then look at the `${STDERR}.log` file corresponding to the `targetRunner` call that failed and execute/debug the last command there.

9. If the language of your operating system, the `target-runner` or the target algorithm is not English, `irace` may not be able to recognize the numbers generated by `target-runner`. We recommend that you run `irace`, the `target-runner` and the target algorithm under an English locale (or make sure that their languages and number format are compatible).

10. It is possible that **transient bugs** in the target algorithm are only visible when running within **irace**, and all commands within **targetRunner** appear to work fine when executed directly in the command-line outside **irace**. See FAQ in [Section 12.4](#)) for suggestions on how to detect such bugs.
11. If your **targetRunner** script works when running irace with **parallel=0** but it fails when using higher number of cores, this may be due to any number of reasons:
 - If you submit jobs through a queuing system, the running environment when using the queuing system may not be the same as when you launch **irace** yourself. The queuing system may also send the job to different machines depending on the number of CPUs requested. One way to test this is to submit the failing execution of **targetRunner** to the queuing system, and specifically to any problematic machine.
 - When using MPI, some calls to **targetRunner** may run on different computers than the one running the master **irace** process. See FAQ in [Section 12.7](#).
 - Does **targetRunner** read or create intermediate files? These files may cause a race condition when two calls to **targetRunner** happen at the same time. You have to make sure that parallel runs of **targetRunner** do not interfere with each other's files.
 - Maybe these files consume too much memory or fill the filesystem when there are simultaneous **targetRunner** calls? Moreover, queuing systems have stricter limits for computing nodes than for the submit/host node.
 - Does the machine or the queuing system impose any limits on number of processes or CPU/memory/filesystem usage per job? Such limits may only trigger when more than one process is executed in parallel, killing the **targetRunner** process before it has a chance to print anything useful. In that case, **irace** may not detect the the program finished unexpectedly, only that the expected output was not printed.

Appendix C **targetEvaluator** troubleshooting checklist

Even if **targetRunner** appears to work, the use of **targetEvaluator** may lead to other problems. The same checklist of **targetRunner** can be followed here. In addition, we list here other potential problems unique to **targetEvaluator**:

1. If **targetEvaluator** fails only in the second or later iteration, this may because output files or data generated by a previous call to **targetRunner** are missing. Elite configurations are never re-executed on the same instance and seed pair, that is, **irace** will call only once **targetRunner** for each pair of configuration ID and instance ID. However, **targetEvaluator** is always re-executed, which takes into account any updated information (normalization bounds, reference sets/points, best-known values, etc.). Thus, any files or data generated by **targetRunner** for a given configuration must remain available to **targetEvaluator** as long as that configuration is alive. The list of alive configurations is passed to **targetEvaluator**, which may decide then which data to keep or remove.

Appendix D Glossary

Parameter tuning: Process of searching good settings for the parameters of an algorithm under a particular tuning scenario (instances, execution time, etc.).

Scenario: Settings that define an instance of the tuning problem. These settings include the algorithm to be tuned (target), budget for the execution of the target algorithm (execution time, evaluations, iterations, etc.), set of problem instances and all the information that is required to perform the tuning.

Target algorithm: Algorithm whose parameters will be tuned.

Target parameter: Parameter of the target algorithm that will be tuned.

irace option: Configurable option of **irace**.

Elite configurations: Best configurations found so far by **irace**. New configurations for the next iteration of **irace** are sampled from the probabilistic models associated to the elite configurations. All elite configurations are also included in the next iteration.

\$IRACE_HOME: The filesystem path where **irace** is installed. You can find this information by opening an R console and executing:

```
system.file(package = "irace")
```

Appendix E NEWS

NEWS

**If you are viewing this file on CRAN, please check [latest news on the irace website](https://mlopez-ibanez.github.io/

irace 4.5

New features and improvements

* #98 irace callback: Add an optional `iterationCallback` scenario function that is invoked after each completed irace. This allows irace users to monitor tuning progress or collect intermediate results without modifying irace internals.

irace 4.4.3.9000

* Fix #94: `sample_model.ParamOrd()` now samples uniformly when the model mean is `NA`, avoiding `NA` values for active ordinal parameters. Reported by @Saethox.

irace 4.4.3

* Do not run postselection race unless elitist racing is enabled.

irace 4.4.2

* Fix failing assert with `elitist=FALSE` (issue #90).

irace 4.4.1

* Minor release to skip failing test on CRAN.

irace 4.4.0

New features and improvements

* The [User Guide](https://mlopez-ibanez.github.io/irace/irace-package.pdf) now contains a section on tuning for anytim

Fixes

- * ``irace`` could error when tuning a single parameter and initial configurations are given for this single parameter in an input file. (Reported by Xiaolu Liu)

- * If ``targetRunner`` returns the expected output in separate lines, ``irace`` could error with: ``Error in set(target_output, j = "configuration", value = unlist_element(experiments, "id_configuration"))``

- * `irace` will now call the RNG of R before evaluating a new instance. This change should make no difference to well-behaved ``targetRunners`` that use the seed provided by `irace` to reseed the RNG, as recommended. However, ``targetRunners`` written in R that fail to follow this recommendation could end up with the same sequence of random numbers in different instances. Now, different instances should get different random numbers.

irace 4.3

New features and improvements

- * Updated MATLAB examples and instructions in the User Guide.

Fixes

- * Add extra whitespace before the first and after the last boxplots in ``plotAblation()`` so that boxes will not overlap with plot margins.

- * Fix bug in ``readConfigurationsFile()`` when a fixed parameter that is both conditional and disabled does not appear in the input file.

- * Fix #87: Using a dependent parameter may produce an error when sampling a different parameter.

irace 4.2.0

Major breaking changes

- * Recovery (``--recover-file``) has been completely re-implemented. This version of `irace` cannot recover files generated by previous versions and vice versa.

- * Now ``blockSize`` multiplies ``elitistNewInstances``, which should ensure that `irace` evaluates instances in blocks.

- * Several improvements in ``ablation()`` may result in different ablation results using this version of `irace`. Also, the log generated by ``ablation()`` has some important changes to record more detailed information and it is more consistent with the log generated by ``irace()``.

New features and improvements

- * When ``targetRunner`` or ``targetEvaluator`` are implemented in R, using random numbers within those functions does not affect the sequence of random numbers within `irace`.

- * ``psRace()`` gains a ``psrace_logFile`` argument to avoid overwriting ``scenario$logFile``.

- * ``psRace()`` makes better use of the budget remaining.

- * New scenario option ``cappingAfterFirstTest``

```

    (``--capping-after-first-test``). If set to 1, elimination due to capping only
    happens after `firstTest` instances are seen (issue #78, suggested by Nguyen Dang).

* `getConfigurationById()` now returns configurations in the same order
  (including repeated values) of the IDs passed as argument.

* `ablation(type="full")` will evaluate all intermediate configurations on all
  instances at once. This makes better use of parallel resources but will not
  report any information about progress after each instance.

* `ablation()` now works better with dependent-domain parameters and with
  complicated conditions. Ablation will not generate intermediate
  configurations that are forbidden.

* The ACOTSP example in the user-guide now uses the ACOTSPQAP code and provides more guidance in case of errors.

## Fixes

* Fix #79: irace.exe did not encode the `libPath` correctly in R >= 4.0 on Windows, producing 'unrecognized escape in c

* Fix #76: Recovery (`--recover-file`) is working again with a completely new implementation.

* Fixed documentation of `psRace()`.

* Fix `psRace()` not saving `psrace_log` in `scenario$logFile`.

* Fix bug in `psRace()` using more budget than the maximum given.

* Handle the case when all elites are rejected before starting the race and
  `sum(alive) <= minSurvival`.

* Fix bug in `sampleSobol()` with parameters whose domain depends on a conditional parameter.

# irace 4.1

## Major breaking changes

* Recovery (`--recover-file`) is currently broken (issue #76).

* The `experiment` list passed to the `targetRunner` R function (see
  `target_runner_default()`) does not contain an element `switches`.
  This element can be obtained from `scenario$parameters$switches`.

## New features and improvements

* Conditions that are always true because they depend on fixed parameters are
  replaced by a `TRUE` value to speed up their evaluation. This speed-up is only
  measurable with very large and complicated parameter spaces.

* The options `src` and `target` of `ablation()` (or `--src` and `--target` of
  the command-line `ablation` tool) now accept a character string that points
  to a file containing a configuration that will be read with
  `readConfigurationsFile()`. This allows performing ablation between
  configurations not explored before.

## Fixes

* Fixed some typos in the user guide. (@sbomsdorf)

* Fixed several bugs in post-selection including `Error in if (any(left >= 0L)) { : missing value where TRUE/FALSE need

```

```

* Fix bug in `ablation()` not calling `stop_parallel()`.

# irace 4.0

## Major breaking changes

* Requires R version >= 4.0

* Logfiles `*.Rdata` use format version 3, which can only be read by R version >= 3.5.

* The scenario options `forbiddenFile` and `forbiddenExps` have been removed
  and will give an error if present. Forbidden configurations are now
  specified in the parameter space description. See the example in
  `readParameters()`.

* The scenario option `digits` has been removed and will give an error if
  present. The number of `digits` for real-valued parameters is now specified
  in the parameter space description. See the example in `readParameters()`.

* The default value of the scenario option `trainInstancesDir` is now `""`.
  The previous default value of `"./Instances"` often caused confusion to
  users not using files as training instances.

* The `scenario` object now includes the `parameters` object. Thus
  functions such as `irace()`, which previously took as arguments both
  `scenario` and `parameters`, now only take `scenario`. This also means that
  the log file `irace.Rdata` does not contain a separate `parameters` element
  since this element can now be found within `scenario`.

* The default value of the scenario option `softRestartThreshold` is now
  0.0001 and does not depend on `digits`.

* The command-line executables `irace` and `ablation` (`irace.exe` and
  `ablation.exe` in Windows) will load the version of the `irace` package that
  is found in the same path where the executables are. In earlier versions,
  the executables will always load the version found via `base::libPaths()`.
  This change allows installing multiple versions of the irace package in
  different locations and each executable will use its corresponding version.
  The correct location can be verified by looking at the line `"installed at:"`
  printed in the output.

* Adaptive capping is now enabled by default if `maxTime > 0` and `boundMax > 0`.
  It can be disabled with `--capping 0` in the command-line options or `capping=0` in the scenario options. See [Pérez-
  (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* The scenario option `targetRunnerLauncherArgs`, introduced in version 3.5,
  was removed and replaced by `targetCmdline`, which is more flexible (fixes #38).
  Please see the user-guide for details.

* Command-line options in joined form, given as `--log-file= --check`, without
  any argument after the `=` will be interpreted as an empty argument,
  equivalent to using `logFile=""` in `scenario.txt`.

* irace will now give an error if you try to recover from a file generated by
  a different version of irace, since such attempts typically end up in errors
  that are difficult to understand.

* irace warns about using `&&` and `||` instead of `&` and `|` in
  parameter conditions and forbidden expressions. A future version of irace
  will reject those uses as errors.

```

- * The internal function ``irace.reload.debug()`` has been removed.
Use `[`devtools::reload()`](https://devtools.r-lib.org/reference/reload.html)` instead.
- * The column `"instance"` of the ``instancesList`` data frame stored in the logFile has been renamed to `"instanceID"`. This data frame should not be accessed directly. Instead use the new function ``get_instanceID_seed_pairs()``.
- * Using ``maxTime > 0`` with ``elitist=0`` now gives a clear error rather than fail later.
(fix #65, reported by @DEOCH)
- * ``path_rel2abs()`` will not expand symlinks to avoid problems with Python's venv.
(fix #64, reported by @DEOCH)
- * Expansion of ``~`` in Windows now follows the definition of ``fs::path_expand()`` rather than ``base::path.expand()``.
- * `irace` is now more strict in enforcing runtime bounds given with ``scenario$boundMax`` and will stop with an error if the ``target-runner`` reports a runtime larger than the given bound.
- * All functions that contained a period (``.'``) in the name have been renamed to use ``_'`` instead.
- * The periods (``.'``) in the arguments of ``scenario$targetRunnerParallel`` and ``scenario$targetEvaluator`` have also been replaced by ``_'``.
- * The environment ``irace`` that was available in the log file under ``iraceResults$state$.irace`` is replaced directly by ``iraceResults$state``. It contains similar information but some entries have been renamed. For example, the ``experimentLog`` data frame is now called ``experiment_log`` and it is a `[`data.table`](https://cran.r-project.org/package=data.table)`.
- * The interface of ``psRace()`` has been simplified.
- * ``irace`` will automatically execute a post-selection race (``psRace()``) using any remaining budget (currently only when ``maxTime == 0``). To disable this behavior, set the scenario option ``postselection`` (``--postselection``) to ``0``.

New features and improvements

- * ``sampleUniform()`` and ``sampleModel()`` are significantly faster thanks to using `[`data.table`](https://cran.r-project.org/package=data.table)`.
- * Initial configurations are sampled using Sobol low-discrepancy sequences using ``spacefillr::generate_sobol_set()``. The number of configurations is controlled by ``n``.
- * Parameter spaces can be constructed programmatically using ``parametersNew()``. See the documentation for details.
- * Ablation will report configurations that produced the same results, which indicates parameter values that have the same effect on the target algorithm, possibly indicating a bug in the target algorithm.
- * New option ``instancesFile`` of ``ablation()`` for using either the training instances, the test instances or reading instances from a given file.
- * New option ``nrep`` of ``ablation()`` specifies the number of replications per instance used in ``"full"`` ablation. It replaces the previous parameter ``n_instances``, whose definition was more difficult to use correctly.
- * Matrix operations are faster thanks to ``matrixStats``.
- * New scenario option ``blockSize`` for defining blocks of instances. Configurations are only eliminated after evaluating a complete block and never in the middle of a block. This is useful for scenarios when there are

clearly defined instance classes and the best configuration should be balanced among them. In that case, ``trainInstancesFile`` should be written so that each block contains one instance of each class and ``blockSize`` is set to the number of classes.

- * New scenario option ``targetRunnerTimeout``: Timeout in seconds of any ``targetRunner`` call (only applies to ``target-runner`` executables not to R functions).
- * ``plotAblation()`` has several new options:
 - ``type='rank'`` to plot ranks per instance instead of raw cost values.
 - ``n`` to limit the number of parameters shown in the plot.
 - ``width`` replaces ``pdf.width``.
 - ``height`` sets the height of the plot in the PDF file.
- * The previously internal function ``check.output.target.runner`` is renamed to ``check_output_target_runner`` and exported to allow users who write their own ``targetRunnerParallel`` to check the output and repair it if possible. (Deyao Chen)
- * New functions ``read_ablogfile()``, ``has_testing_data()``, ``irace_summarise()``.
- * New functions ``get_random_seed()``, ``set_random_seed()``, ``restore_random_seed()`` useful for writing ``targetRunner`` functions in R.
- * New function ``get_instanceID_seed_pairs()`` to get the pairs of instanceID and random seed used during the races (and optionally the actual instances).
- * The ``parameters`` object now stores the number of ``digits`` (decimal places after the point) for each parameter of type ``r``. As a result, the ``repairConfiguration`` function (see ``defaultScenario()``) only needs two arguments: ``configuration`` and ``parameters``. See examples in the user-guide.
- * ``readScenario()`` (and command-line `irace`) do not require a ``scenario.txt`` file. (Contributed by @DEOCH)
- * ``read_pcs_file()`` now supports forbidden configurations.
- * When testing, ``irace`` now prints the random seed used for each instance as an additional column.
- * The package provides a new executable ``target-runner-dummy`` (or ``target-runner-dummy.exe`` in Windows) for the purposes of testing. It may also be useful for understanding the typical setup of ``irace``.
- * New scenario option ``minExperiments`` to set a minimum budget of runs. (proposed by @Saethox, fixes #58)
- * New function ``multi_irace()`` for executing multiple runs of `irace` with the same or different scenarios and parameters, possibly in parallel. (Contributed by @Saethox)

Fixes

- * ``ablation_cmdline()`` and ``plotAblation()`` no longer create an empty ``Rplots.pdf`` file when specifying an output PDF file.
- * Fix #66: when using ``maxTime > 0``, `irace` estimates the time per run by executing 2 configurations on ``firstTest`` instances and adjusts ``boundMax`` to not go over ``budgetEstimation``. This may result in a smaller ``boundMax`` than before. To reduce this impact, the default value of ``budgetEstimation`` is now ``0.05`` instead of ``0.02``. (Manuel López-Ibáñez, reported by @DEOCH)

- * Fix #55: Configurations provided may use ``<NA>`` in addition to ``NA`` to denote the missing value of a disabled parameter.
(Manuel López-Ibáñez, reported by @TheIronBorn)
- * Fix #44: irace now will give an error if the domain of real-valued (r) parameters would change depending on the value of ``'digits'``. The solution is to increase the value of ``'digits'`` or adjust the domain.
(Manuel López-Ibáñez, reported by @mb706)
- * If scenario option ``targetRunnerParallel`` is set, irace no longer tries to initialize a parallel environment or setup MPI. It is now the responsibility of the user to do that before calling irace or within the function assigned to ``targetRunnerParallel``.
- * irace no longer sets ``option(error=utils::recover())`` in debug mode to avoid issues when calling irace from Python. The user can set this if desired.
- * Fix bug failing to restart with parameters that have dependent domains.
- * Fix bug with ``sampleInstances=FALSE`` that could re-evaluate the same (instance, seed) pair more than once.
- * Fix bug when using ``targetRunnerLauncher`` and ``targetRunner`` contain whitespace.
- * Fix bug in ``ablation_cmdline()`` about missing ``scenario`` object.
- * ``ablation()`` will now save and restore the previous random seed.
- * ``ablation()`` will detect if the logfile (e.g., ``irace.Rdata``) is incomplete.
- * ``readConfigurationsFile()`` now handles parameters with dependent domains.
- * Fix #71: Ensure ``".ID."`` is the first column in ``checkTargetFiles()`` (Manuel López-Ibáñez, reported by @ivan1arriola)

irace 3.5

New features and improvements

- * Handling of dependent parameter domains: These should be specified in the parameter domain definition and, for now, only numerical parameter can define dependent domains. A numerical domain can be dependent on one bound, e.g. ``(1, "param1*2")``, where the dependent bound can include basic arithmetic operators.
(Leslie Pérez Cáceres, Manuel López-Ibáñez)
- * The package now provides an ``ablation`` executable (`ablation.exe` in Windows) that makes easier to perform ablation analysis without having any R knowledge.
- * The interface to functions ``ablation()`` and ``plotAblation()`` has been simplified. The ``ablation()`` function now allows overriding scenario settings. The ``plotAblation()`` function will not create the plot if the ablation log does not contain a complete ablation.
(Manuel López-Ibáñez)
- * The argument ``n.instances`` of ``ablation()`` has been renamed to ``n_instances`` and it is now a factor that multiplies ``scenario$firstTest``.
(Manuel López-Ibáñez)
- * New command-line option ``--quiet`` to run without producing any output except errors (also available as a scenario option).
(Manuel López-Ibáñez)

- * New command-line option `--init` to initialize a scenario. (Deyao Chen)
- * Added support for HTCondor cluster framework to `--batchmode`.
(Filippo Bistaffa)
- * `--check` now also check the contents of `configurationsFile` and runs configurations provided via `initConfigurations`.
(Manuel López-Ibáñez, reported by Andreea Avramescu)
- * New scenario options `targetRunnerLauncher` and `targetRunnerLauncherArgs` to help in cases where the target-runner must be invoked via another software with particular options (such as `python.exe` in Windows).
(Manuel López-Ibáñez)
- * New scenario option `minMeasurableTime`.
(Manuel López-Ibáñez)
- * An error is produced if a variable set in the scenario file is not known to irace. If your scenario file contains R code, then use variable names beginning with a dot ``.``, which will be ignored by irace.
(Manuel López-Ibáñez)
- * Plotting functions have been moved to the new package [iraceplot](https://auto-optimization.github.io/iraceplot/). In particular, `configurationsBoxplot()` is replaced by `iraceplot::boxplot_training()` and `iraceplot::boxplot_test()`; `parallelCoordinatesPlot()` is replaced by `iraceplot::parallel_cat()` and `iraceplot::parallel_coord()`; and `parameterFrequency()` is replaced by `iraceplot::sampling_frequency()`.
(Leslie Pérez Cáceres, Manuel López-Ibáñez)
- * The user-guide now contains a detailed section on "Hyper-parameter optimization of machine learning methods".
(Manuel López-Ibáñez)
- * When `testType="F-test"` and only two configurations remain, the elimination test now uses the pseudo-median estimated by the Wilcoxon signed-rank test to decide which configuration is the best one instead of comparing the median difference.
(Manuel López-Ibáñez)
- * New functions `testing_fromlog()` and `testing_fromfile()` for independently executing the testing phase. The function `testing.main()` was removed as it is superseded by the new ones.
(Manuel López-Ibáñez)
- * New function `read_logfile()` to easily read the log file produced by irace.
(Manuel López-Ibáñez)
- * New function `printParameters()` that prints a parameters R object as a valid input text.
(Manuel López-Ibáñez)
- * `irace2pyimp` moved to its own R package.
(Manuel López-Ibáñez)
- * Generating the file `irace.Rdata` may be disabled by setting `logFile=""`.
(Manuel López-Ibáñez, reported by Johann Dreö)
- * `path_rel2abs()` and `checkParameters()` are now exported so that other packages may use them.
(Manuel López-Ibáñez)

- * ``path_rel2abs()`` also searches in system paths. (Manuel López-Ibáñez)
- * ``readConfigurationsFile()`` will now detect duplicated configurations and error. (Manuel López-Ibáñez)
- * The interface to functions ``getFinalElites()``, ``getConfigurationById()`` and ``getConfigurationByIteration()`` has been simplified.
- * The package provides a ``irace.sindef`` file that may be used for building a standalone container of irace using Singularity. See the ``README.md`` file for instructions. (Contributed by Johann Dreo)
- * New example ``examples/target-runner-python/target-runner-python-win.bat`` contributed by Levi Ribeiro.
- * New helper script in ``bin/parallel-irace-slurm`` to launch ``irace`` in [SLURM](https://slurm.schedmd.com/) computer cluster. (Manuel López-Ibáñez)
- * Rename ``scenario.update.paths()`` to ``scenario_update_paths()``. The old name is deprecated. (Manuel López-Ibáñez)

Fixes

- * Correctly handle clear out-performance cases despite strong bi-modality. (Reported by Nguyen Dang, fixed by Manuel López-Ibáñez)
- * Fix error when recovering from a parallel run on Windows. (Manuel López-Ibáñez, reported by Tarek Gamal)
- * ``testNbElites`` now controls how many iteration elites are tested when ``testIterationElites=1``. This is the documented behavior in the user guide. (Manuel López-Ibáñez, reported by Marcelo de Souza)
- * Fixes to the Matlab example. (Manuel López-Ibáñez)
- * The default of ``testType`` is now set to ``t-test`` when capping is enabled. (Manuel López-Ibáñez, reported by Jovana Radjenovic)
- * Fix various issues in the user guide. (Manuel López-Ibáñez, reported by Jovana Radjenovic)
- * Remove duplicated elites. (Manuel López-Ibáñez, reported by Federico Naldini)
- * Fix (#7): warnings with partial matched parameters. (Manuel López-Ibáñez, reported by Marc Becker)
- * Fix (#10): wrong assert with ``elitist=0``. (Manuel López-Ibáñez)
- * Fix (#12): irace can be run with [FastR](https://www.graalvm.org/22.1/docs/getting-started/#run-r).
- * Fix (#13): Maximum number configurations immediately rejected reached. (Manuel López-Ibáñez)
- * Fix: when setting the scenario file in the command-line, ``scenarioFile`` was not set correctly. The correct scenario was used, however, the debug output and the value stored in the log / recovery file was wrong. (Manuel López-Ibáñez, reported by Richard Schoonhoven)
- * With ``sampleInstances = FALSE``, ``elitist`` irace does not change the order of instances already seen. However, if you want to make sure that the order of the instances is enforced, you also need to set ``elitistNewInstances=0``.

```

* The function `irace.usage()` was removed. It was not really useful for R
  users as the same result can be obtained by calling
  `irace.cmdline("--help")`.
                                                    (Manuel López-Ibáñez)

# irace 3.4.1 (31/03/2020)

* `NEWS` converted to markdown.

* Fix CRAN error on Solaris.

# irace 3.4 (30/03/2020)

* `irace2pyimp` function and executable (`irace2pyimp.exe` on Windows) to
  convert .Rdata files generated by irace to the input files required by the
  parameter importance analysis tool PyImp
  (https://github.com/automl/ParameterImportance).
                                                    (Nguyen Dang, Manuel López-Ibáñez)

* Initial configurations may also be provided directly in R using
  `scenario$initConfigurations`
                                                    (Manuel López-Ibáñez)

* Rdata files are saved in version 2 to keep compatibility with older R
  versions.
                                                    (Manuel López-Ibáñez)

* Fix invalid assert with ordered parameters:
                                                    (Leslie Pérez Cáceres)

  ```
 value >= 1L && value <= length(possibleValues) is not TRUE
  ```

* The `irace` executable (`irace.exe` on Windows) is a compiled binary instead
  of a script. On Windows, `irace.exe` replaces `irace.bat`
                                                    (Manuel López-Ibáñez)

* `inst/examples/Spear` contains the Spear (SAT solver) configuration scenario.
                                                    (Manuel López-Ibáñez)

* Fixed bug when reporting minimum `maxTime` required.
  (Reported by Luciana Salete Buriol,
  fixed by Manuel López-Ibáñez)

* Fixed bug detected by assert:

  ```R
 all(apply(!is.na(elite.data$experiments), 1, any)) is not TRUE
  ```

  (Reported by Maxim Buzdalov, fixed by Manuel López-Ibáñez)

# irace 3.3 (26/04/2019)

* Fix buggy test that breaks CRAN.
                                                    (Manuel López-Ibáñez)

* Do not print "23:59:59" when wall-clock time is actually close to zero.
                                                    (Manuel López-Ibáñez)

```

```

# irace 3.2 (24/04/2019)

* Fix `irace --check --parallel 2` on Windows. (Manuel López-Ibáñez)

* Values of real-valued parameter are now printed with sufficient precision to
  satisfy `digits` (up to `digits=15`). (Manuel López-Ibáñez)

* It is possible to specify `boundMax` without capping.
  (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* `irace --check` will exit with code 1 if the check is unsuccessful
  (Manuel López-Ibáñez)

* Print where irace is installed with `--help`. (Manuel López-Ibáñez)

* irace will now complain if the output of `target-runner` or `target-evaluator`
  contains extra lines even if the first line of output is correct. This is to
  avoid parsing the wrong output. Unfortunately, this may break setups that
  relied on this behavior. The solution is to only print the output that irace
  expects. (Manuel López-Ibáñez)

* Completely re-implement `log` parameters to fix several bugs. Domains that
  contain zero or negative values are now rejected.
  (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* New option `aclib=` (`--aclib 1`) enables compatibility with the
  GenericWrapper4AC (https://github.com/automl/GenericWrapper4AC/) used by
  AClib (https://aclib.net/). This is EXPERIMENTAL. `--aclib 1` also sets
  digits to 15 for compatibility with AClib defaults.
  (Manuel López-Ibáñez)

* Fix printing of output when capping is enabled. (Manuel López-Ibáñez)

* `checkTargetFiles()` (`--check`) samples an instance unless
  `sampleInstances` is FALSE. (Manuel López-Ibáñez)

* Fix symbol printed in elimination test. (Manuel López-Ibáñez)

* Use `dynGet()` to find `targetRunner` and `targetEvaluator`.
  As a result, we now require R >= 3.2. (Manuel López-Ibáñez)

* All tests now use `testthat`. (Manuel López-Ibáñez)

* New function `scenario.update.paths()` (Manuel López-Ibáñez)

* Fix assert failure that may happen when `elitistNewInstances` is larger than
  `firstTest`. Reported by Jose Riveaux. (Manuel López-Ibáñez)

* Fix bug in `checkTargetFiles()` (`--check`) with capping.
  (Leslie Pérez Cáceres)

* Clarify a few errors/warnings when `maxTime > 0`.
  (Manuel López-Ibáñez, suggested by Haroldo Gambini Santos)

# irace 3.1 (12/07/2018)

* Use testthat for unit testing. (Manuel López-Ibáñez)

```

```

* Allow instances to be a list of arbitrary R objects (`mlr` bugfix).
                                     (Manuel López-Ibáñez)

# irace 3.0 (05/07/2018)

* irace now supports adaptive capping for computation time minimization.
  The default value of the `testType` option is t-test when adaptive capping
  is enabled. Please see the user-guide for details.
                                     (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* The package contains an `ablation()` function implementing the ablation
  method for parameter importance analysis by Fawcett and Hoos (2016).
                                     (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* New option `postselection` executes a post-selection race.
                                     (Leslie Pérez Cáceres)

* At the end of each race, if the race stops before evaluating all instances
  seen in previous races, then the best overall may be different than the best
  of the race. We now print the best overall (best-so-far). Elites evaluated
  on more instances are considered better than those evaluated on fewer.
                                     (Manuel López-Ibáñez, Leslie Pérez Cáceres)

* Last active parameter values of numerical parameters (`i` and `r`) are carried
  by the sampling model. When a value must be assigned and the parameter was
  previously not active, the sampling is performed around the last value.
                                     (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* R help pages are now generated with Roxygen2.
                                     (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* The user guide documents `--version`, `--help`, and `--check`.
                                     (Manuel López-Ibáñez)

* A return value of `Inf` from `targetRunner`/`targetEvaluation` results in
  the immediate rejection of the configuration without any further evaluation.
  This is useful for handling unreliable or broken configurations that should
  not stop irace.
                                     (Manuel López-Ibáñez)

* Numerical parameters may be sampled on a logarithmic scale using `i,log`
  or `r,log`.
                                     (Alberto Franzin)

* New `target-runner.bat` for Windows contributed by André de Souza Andrade.

* Fixed all shell scripts calling functions before defining them, which is not
  portable.
                                     (Manuel López-Ibáñez)

* Fixed `--parallel` bug in Windows that resulted in
  `Error in checkForRemoteErrors(val)`.
                                     (Manuel López-Ibáñez)

* Improve error message when no training instances are given.
                                     (Manuel López-Ibáñez)

# irace 2.4 (03/08/2017)

* The output of irace now specifies in which order, if any, configurations are
  printed.
                                     (Manuel López-Ibáñez, suggested by Markus Wagner)

```

* Several fixes for handling paths in Windows.
(Manuel López-Ibáñez)

* ``readConfigurationsFile()`` now has a ``text=`` argument, which allows reading configurations from a string.
(Manuel López-Ibáñez)

* User-provided functions (`targetRunner`, `targetEvaluator` and `repairConfiguration`) and user-provided conditions for forbidden configurations are now byte-compiled when read, which should make their evaluation noticeably faster.
(Manuel López-Ibáñez)

* The argument ``experiment`` passed to the R function ``targetRunner`` does not contain anymore an element ``extra.params``. Similarly, the ``scenario`` structure does not contain anymore the elements ``instances.extra.params`` and ``testInstances.extra.params``. Any instance-specific parameters values now form part of the character string that defines an instance and it is up to the user-defined ``targetRunner`` to parse them appropriately. These changes make no difference when `targetRunner` is an external script, or when instances and instance-specific parameter values are read from a file.
(Manuel López-Ibáñez)

irace 2.3

* Fix bug that will cause ``iraceResults$experimentLog`` to count calls to ``targetEvaluator`` as experiments, even if no call to ``targetRunner`` was performed. This does not affect the computation of the budget consumed and, thus, it does not affect the termination criteria of irace. The bug triggers an assertion that terminates irace, thus no run that was successful with version 2.2 is affected.
(Manuel López-Ibáñez)

irace 2.2

* Command-line parameters are printed to stdout (useful for future replications). (Manuel López-Ibáñez, suggested by Markus Wagner)

* Users may provide a function to repair configurations before being evaluated. See the scenario variable `repairConfiguration`.
(Manuel López-Ibáñez)

* The option ``--sge-cluster`` (``sgeCluster``) was removed and replaced by ``--batchmode`` (``batchmode``). It is now the responsibility of the target-runner to parse the output of the batch job submission command (e.g., ``qsub`` or ``squeue``), and return just the job ID. Values supported are: "sge", "torque", "pbs" and "slurm".
(Manuel López-Ibáñez)

* The option ``--parallel`` can now be combined with ``--batchmode`` to limit the number of jobs submitted by irace at once. This may be useful in batch clusters that have a small queue of jobs.
(Manuel López-Ibáñez)

* New examples under ``inst/examples/batchmode-cluster/``.
(Manuel López-Ibáñez)

* It is now possible to include scenario definition files from other scenario files by using:

```
```R
eval.parent(source("scenario-common.txt", chdir = TRUE, local = TRUE))
```

```

...

 This feature is VERY experimental and the syntax is likely to change in the
 future. (Manuel López-Ibáñez)

* Fix a bug that re-executed elite results under some circumstances.
 (Leslie Pérez Cáceres)

* Restrict the number of maximum configurations per race to 1024.
 (Leslie Pérez Cáceres)

* Do not warn if the last line in the instance file does not terminate with a
 newline. (Manuel López-Ibáñez)

* Fix bug when `deterministic == 1`.
 (Manuel López-Ibáñez, Leslie Pérez Cáceres)

* Update manual and vignette with details about the expected arguments and
 return value of `targetRunner` and `targetEvaluator`. (Manuel López-Ibáñez)

* Many updates to the User Guide vignette. (Manuel López-Ibáñez)

* Fix `dontrun` example in `irace-package.Rd` (Manuel López-Ibáñez)

* Fix bug: If testInstances contains duplicates, results of testing are not
 correctly saved in `iraceResults$testing$experiments` nor reported correctly
 at the end of a run. Now unique IDs of the form `1t, 2t, ...` are used for
 each testing instance. These IDs are used for the rownames of
 `iraceResults$testing$experiments` and the names of the
 `scenario$testInstances`
 and `iraceResults$testing$seeds` vectors. (Manuel López-Ibáñez)

* Fix bug where irace keeps retrying the `target-runner` call even if it
 succeeds. (Manuel López-Ibáñez)

* New command-line parameter
...
 --only-test FILE
...
 which just evaluates the configurations given in FILE on the testing
 instances defined by the scenario. Useful if you decide on the testing
 instances only after running irace. (Manuel López-Ibáñez)

* Bugfix: When using `maxTime != 0`, the number of experiments performed may be
 miscounted in some cases. (Manuel López-Ibáñez)

irace 2.1

* Fix CRAN errors in tests. (Manuel López-Ibáñez)

* Avoid generating too many configurations at once if the initial time
 estimation is too small. (Manuel López-Ibáñez)

irace 2.0

* Minimum R version is 2.15.

* Elitist irace by default, it can be disabled with parameter `--elitist 0`.
 (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* The parameter `--test-type` gains two additional values: (Manuel López-Ibáñez)

```

```

- `t-test-bonferroni` (t-test with Bonferroni's correction for multiple
 comparisons),
- `t-test-holm` (t-test with Holm's correction for multiple comparisons)

* MPI does not create log files with `--debug-level 0`.
 (Manuel López-Ibáñez)

* For simplicity, the `parallel-irace-*` scripts do not use an auxiliary
 `tune-main` script. For customizing them, make a copy and edit them
 directly.
 (Manuel López-Ibáñez)

* New parameters: (Manuel López-Ibáñez)
...

--target-runner-retries : Retry target-runner this many times in case of error.
...

* We print diversity measures after evaluating on each instance:
 (Leslie Pérez Cáceres)

- Kendall's W (also known as Kendall's coefficient of concordance) If 1,
 all candidates have ranked in the same order in all instances. If 0, the
 ranking of each candidate on each instance is essentially random.

$$W = \text{Friedman} / (m * (k-1))$$

- Spearman's rho: average (Spearman) correlation coefficient computed on the
 ranks of all pairs of raters. If there are no repeated data values, a
 perfect Spearman correlation of +1 or -1 occurs when each of the variables
 is a perfect monotone function of the other.

* Many internal and external interfaces have changed. For example, now we
 consistently use 'scenario' to denote the settings passed to irace and
 'configuration' instead of 'candidate' to denote the parameter settings
 passed to the target algorithm. Other changes are:
...R
parameters$boundary -> parameters$domain
hookRun -> targetRunner
hookEvaluate -> targetEvaluator
tune-conf -> scenario.txt
instanceDir -> trainInstancesDir
instanceFile -> trainInstancesFile
testInstanceDir -> testInstancesDir
testInstanceFile -> testInstancesFile
...

* Minimal example of configuring a MATLAB program
 (thanks to Esteban Diaz Leiva)

* Paths to files or directories given in the scenario file are relative to the
 scenario file (except for `--log-file`, which is an output file and it is
 relative to `--exec-dir`). Paths given in the command-line are relative to the
 current working directory. Given
...bash
$ cat scenario/scenario.txt
targetRunner <- "./target-runner"
$ irace -s scenario/scenario.txt
...

irace will search for `./scenario/target-runner`, but given
...bash

```

```

$ irace -s scenario/scenario.txt --target-runner ./target-runner
...
irace will search for `./target-runner`. (Manuel López-Ibáñez)

* New command-line wrapper for Windows installed at
`system.file("bin/irace.bat", package="irace)`
(thanks to Anthony Antoun)

* Budget can be specified as maximum time (`maxTime`, `--max-time`) consumed by
the target algorithm. See the documentation for the details about how this
is handled.
(Leslie Pérez Cáceres, Manuel López-Ibáñez)

irace 1.07

* The best configurations found, either at the end or at each iteration of an
irace run, can now be applied to a set of test instances different from the
training instances. See options `testInstanceDir`, `testInstanceFile`,
`testNbElites`, and `testIterationElites`. (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* The R interfaces of `hookRun`, `hookEvaluate` and `hookRunParallel` have changed.
See `help(hook.run.default)` and `help(hook.evaluate.default)` for examples of
the new interfaces.

* Printing of race progress now reports the actual configuration and instance
IDs, and numbers are printed in a more human-readable format.
(Leslie Pérez Cáceres, Manuel López-Ibáñez)

* Reduce memory use for very large values of `maxExperiments`.
(Manuel López-Ibáñez, thanks to Federico Caselli for identifying the issue)

* New option `--load-balancing` (`loadBalancing`) for disabling load-balancing
when executing jobs in parallel. Load-balancing makes better use of
computing resources, but increases communication overhead. If this overhead
is large, disabling load-balancing may be faster.
(Manuel López-Ibáñez, thanks to Federico Caselli for identifying the issue)

* The option `--parallel` in Windows now uses load-balancing by default.
(Manuel López-Ibáñez)

* The wall-clock time after finishing each task is printed in the output.
(Manuel López-Ibáñez, thanks to Federico Caselli for providing an initial
patch)

irace 1.06

* Fix bug that could introduce spurious whitespace when printing the
final configurations. (Manuel López-Ibáñez)

* Fix bug if there are more initial candidates than needed for the
first race. (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* New configuration options, mainly for R users:

- `hookRunParallel`: Optional R function to provide custom
parallelization of `hook.run`.

- `hookRunData`: Optional data passed to `hookRun`. This is ignored by the
default `hookRun` function, but it may be used by custom `hookRun` R
functions to pass persistent data around. (Manuel López-Ibáñez)

```

```

irace 1.05

* New option `--version`. (Manuel López-Ibáñez)

* Terminate early if there is no sufficient budget to run irace with
 the given settings. (Manuel López-Ibáñez)

* The option `--parallel` (without `--mpi`) now works under Windows.
 (Manuel López-Ibáñez, thanks to Pablo Valledor Pellicer for testing
 it)

* Improved error handling when running under Rmpi. Now irace will
 terminate as soon as the master node detects at least one failed
 slave node. This avoids irace reporting two times the same error.
 Also, irace will print all the unique errors returned by all slaves
 and not just the first one.
 (Manuel López-Ibáñez)

* Forbidden configurations may be specified in terms of constraints
 on their values. Forbidden configurations will never be evaluated by irace.
 See `--forbidden-file` and `inst/templates/forbidden.tmpl`.
 (Manuel López-Ibáñez)

* New option `--recovery-file` (`recoveryFile`) allows resuming a
 previous irace run. (Leslie Pérez Cáceres)

* The confidence level for the elimination test is now
 configurable with parameter `--confidence`. (Leslie Pérez Cáceres)

* Much more robust handling of relative/absolute paths. Improved support
 for Windows. (Leslie Pérez Cáceres, Manuel López-Ibáñez)

* Provide better error messages for incorrect parameter
 descriptions. (Manuel López-Ibáñez)
 Examples:
 ...
 x "" i (0, 0) # lower and upper bounds are the same
 x "" r (1e-4, 5e-4) # given digits=2, ditto
 x "" i (-1, -2) # lower bound must be smaller than upper bound
 x "" c ("a", "a") # duplicated values
 ...

* Print elapsed time for calls to hook-run if `debugLevel >=1`.
 (Manuel López-Ibáñez)

* `examples/hook-run-python/hook-run`: A multi-purpose `hook-run` written
 in Python. (Franco Mascia)

* Parallel mode in an SGE cluster (`--sge-cluster`) is more
 robust. (Manuel López-Ibáñez)

irace 1.04

* Replace obsolete package multicore by package parallel
 (requires R >= 2.14.0)

* Use load-balancing (`mc.preschedule = FALSE`) in `mclapply`.

irace 1.03

* Use `reg.finalizer` to finish Rmpi properly without clobbering
 `.Last()`.

```

```
* Remove uses of deprecated `as.real()`.

* Nicer error handling in `readParameters()`.

* Add hypervolume (multi-objective) example.

* Fix several bugs in the computation of similar candidates.

irace 1.02

* More concise output.

* The parameters `expName` and `expDescription` are now useless and they
 were removed.

* Faster computation of similar candidates (Jeremie Dubois-Lacoste
 and Leslie Pérez Cáceres).

* Fix bug when saving instances in `tunerResults$experiments`.

* `irace.cmdline ("--help")` does not try to quit R anymore.

irace 1.01

* Fix bug caused by file.exists (and possibly other functions)
 not handling directory names with a trailing backslash or slash on
 Windows.

* Fix bug using per-instance parameters (Leslie Pérez Cáceres).

* Fix bug when reading initial candidates from a file.
```

---